

6 传输层

(6 学时左右)

目录

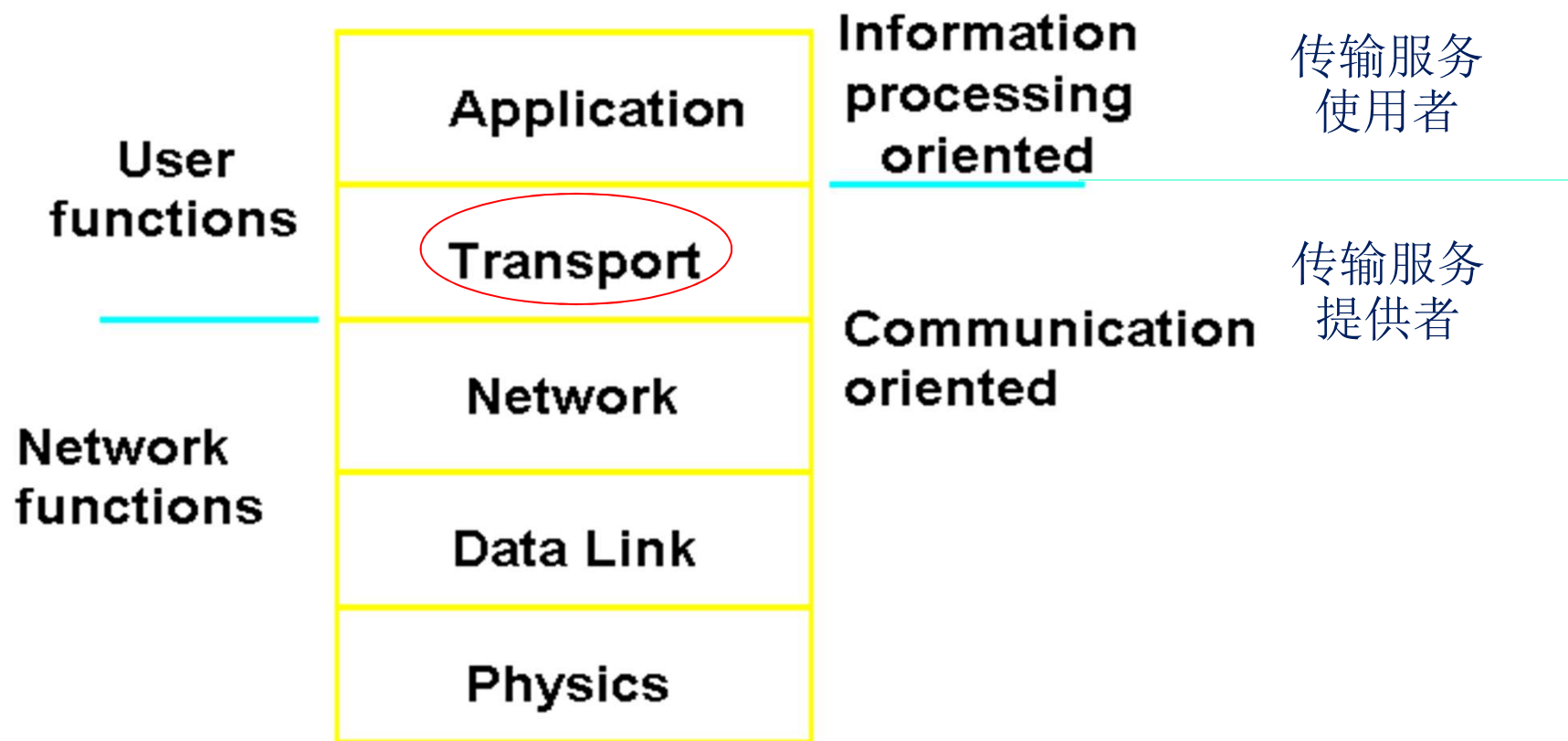
- 传输层的功能与服务
- 传输协议要素
 - ◆ 寻址
 - ◆ 连接建立与释放
 - ◆ 流量控制和缓存
 - ◆ 拥塞控制
- 因特网的传输层协议
 - ◆ 用户数据报协议 (UDP)
 - ◆ 传输控制协议 (TCP)

提出传输层的动机

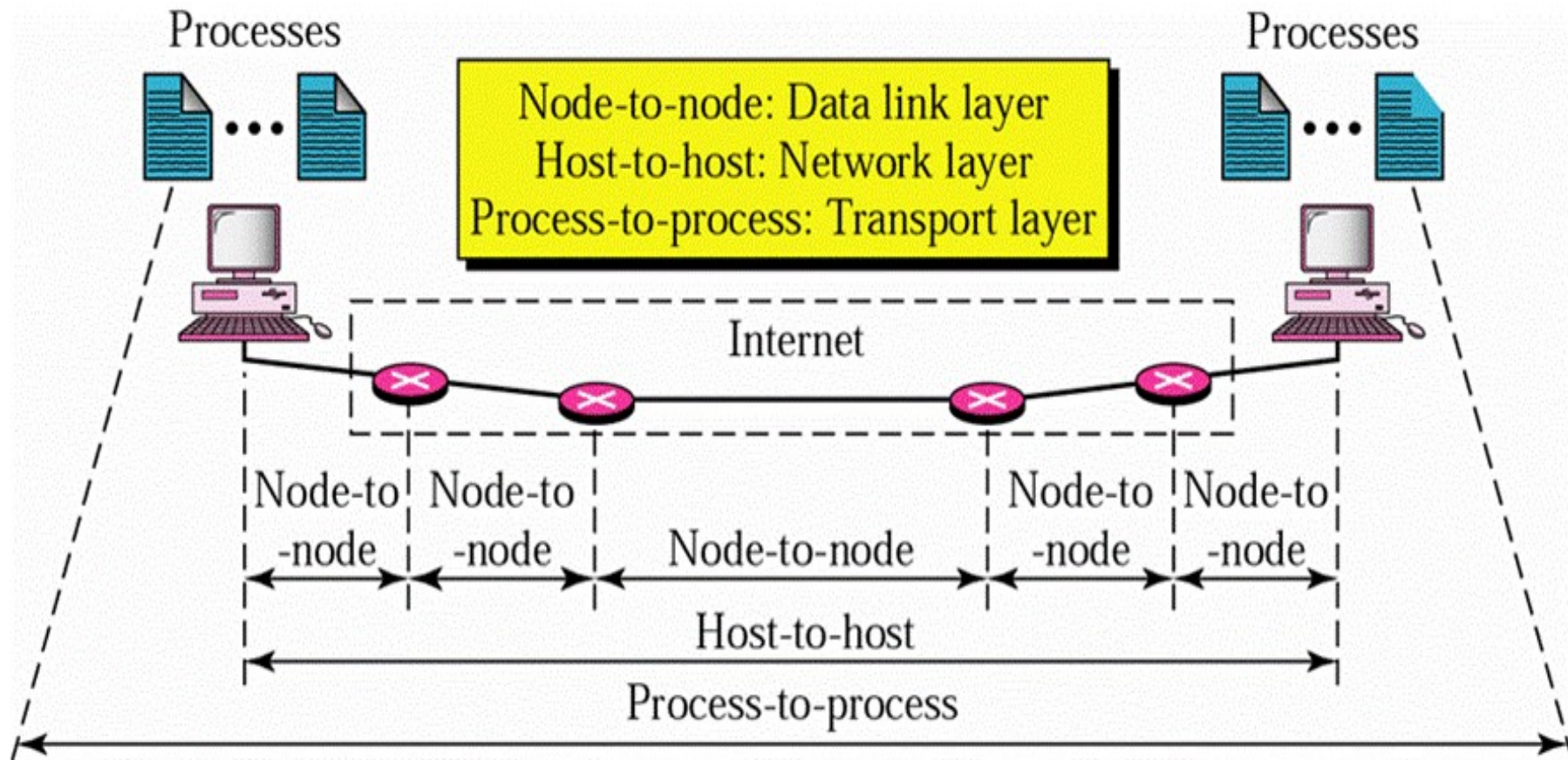
- **IP**提供了较弱但高效的服务模型 (*尽力而为*)
 - ◆ **IP**包可能被延误、被丢弃、乱序或者重复
 - ◆ 包的长度受到限制
- **IP**包只能寻址到主机
 - ◆ 如何确定这个包内的数据应该交给哪一个网络应用？
- 主机应该以怎样的速率向网络中发送包？
 - ◆ 如果太快，可能淹没网络
 - ◆ 太慢则低效

传输层在协议栈中的位置

■ 整个协议层次结构的核心



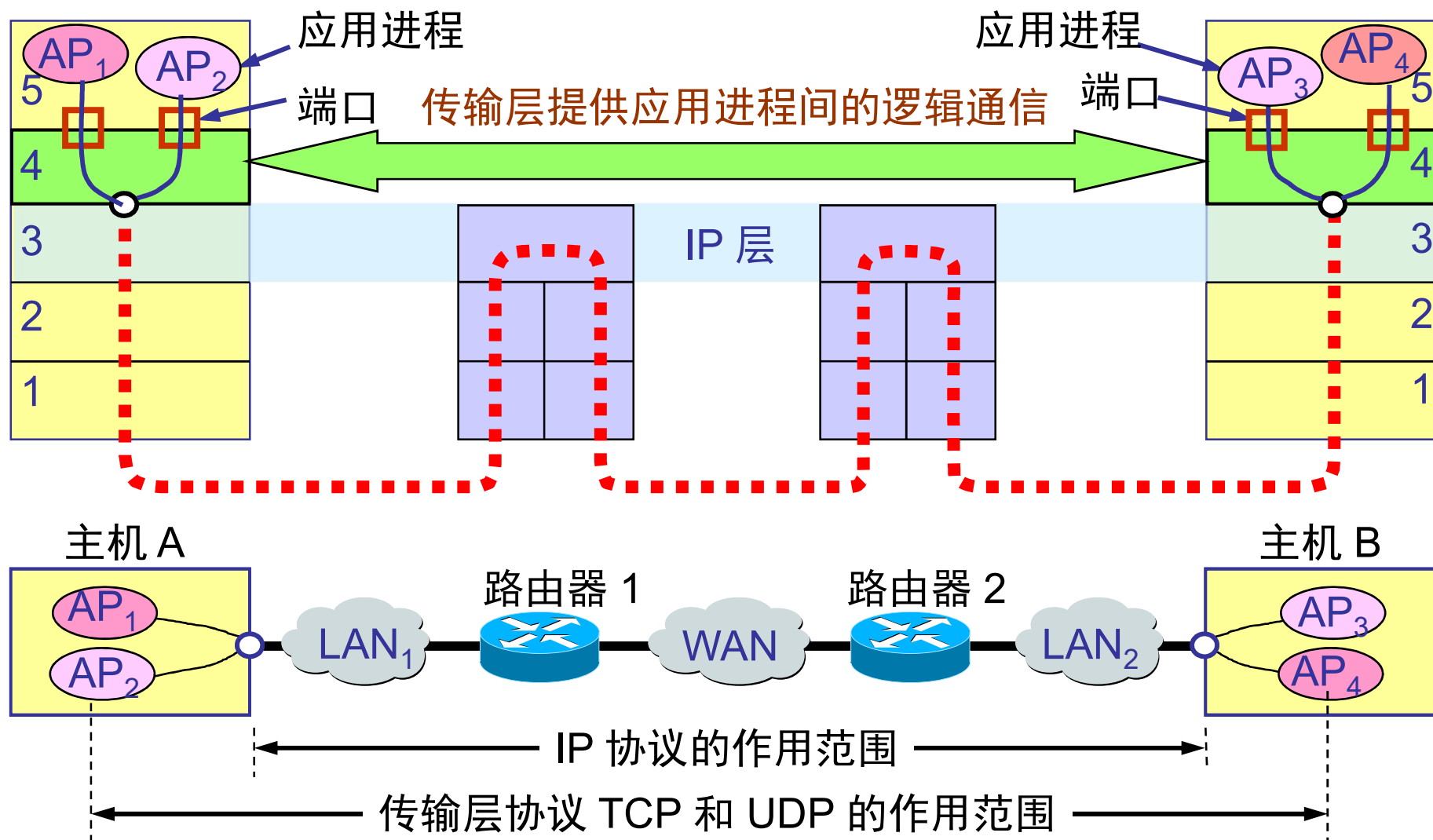
进程-进程的交付



进程-进程

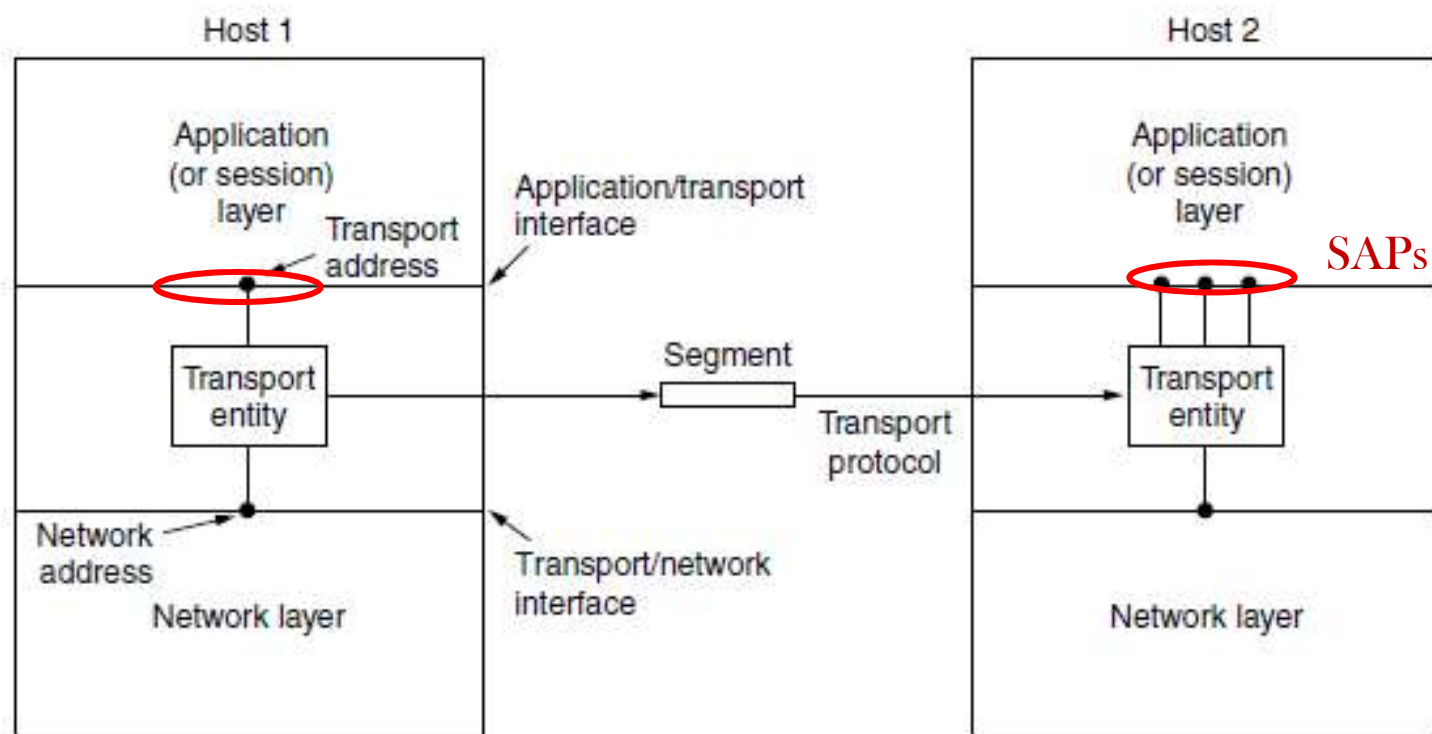
[Forouzan]

传输层的作用范围



传输层提供给上层的服务

- 屏蔽通信子网的多样性，为上层提供**公共接口**
- 提供**共享**同一个网络链路的多个服务访问点 (**SAP**)
- 提供两类服务：无连接的传输服务, 面向连接的传输服务



客户端-服务器模式

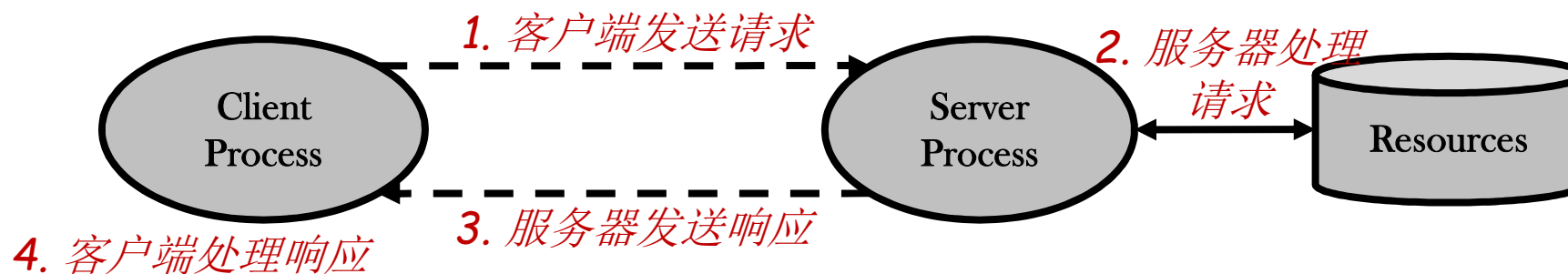
- 几乎所有网络应用程序都使用

- 服务器

- ◆ 管理资源并为客户端提供服务
- ◆ 最先开始执行
- ◆ 在预定位置**被动**等待客户端的请求

- 客户端

- ◆ 稍后开始执行
- ◆ **主动发起**与服务器的联系



传输服务原语

- 提供给应用进程的接口（应用编程接口 **API**）
- 一个简单的面向连接的服务示例

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	Request a release of the connection

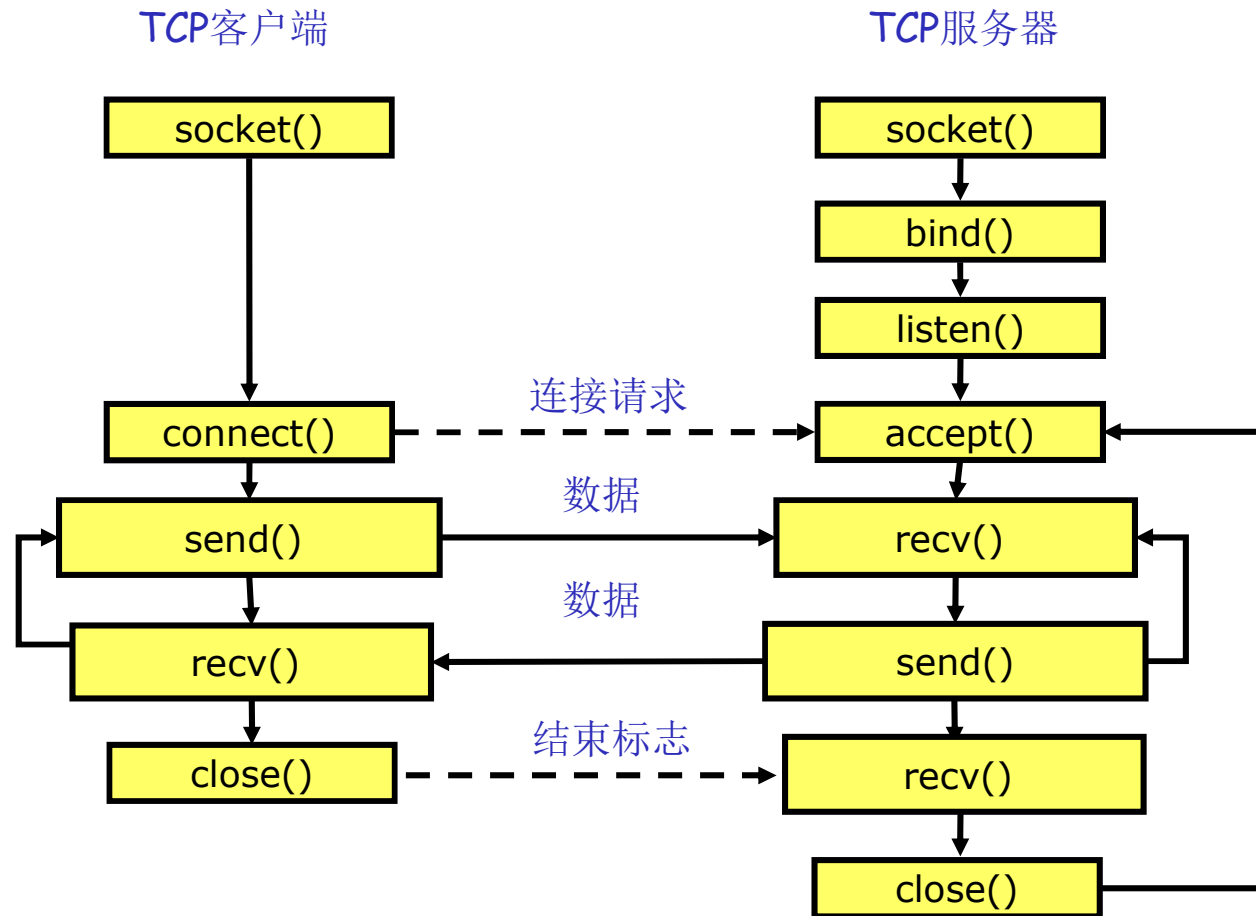
套接字 (Socket)

■ 因特网中广泛使用的API

Primitive	Meaning
SOCKET	Create a new communication endpoint
BIND	Associate a local address with a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Passively establish an incoming connection
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

Figure 6-5. The socket primitives for TCP.

基于TCP的Socket程序流程

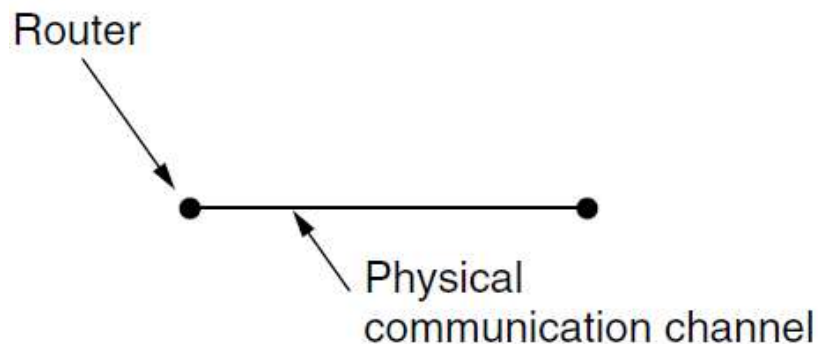


目录

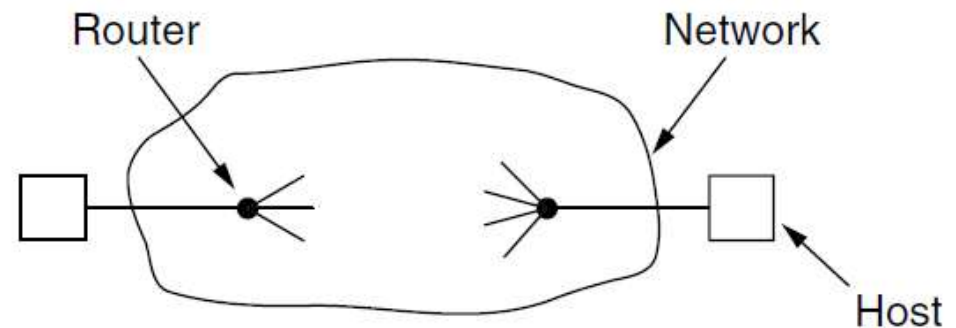
- 传输层的功能与服务
- 传输协议要素
 - ◆ 寻址
 - ◆ 连接建立与释放
 - ◆ 流量控制和缓存
 - ◆ 拥塞控制
- 因特网的传输层协议
 - ◆ 用户数据报协议 (UDP)
 - ◆ 传输控制协议 (TCP)

传输协议与数据链路协议

- 两种协议都要处理差错控制、排序、流量控制等问题（使用**ARQ**）
- 点-点交付 与 进程-进程交付
- 两种协议的运行环境不同
 - ◆ 目的地址
 - ◆ 初始连接的建立
 - ◆ 网络的存储能力
 - ◆ 缓冲区的分配（如缓冲和流量控制）



数据链路层：中间是一条链路



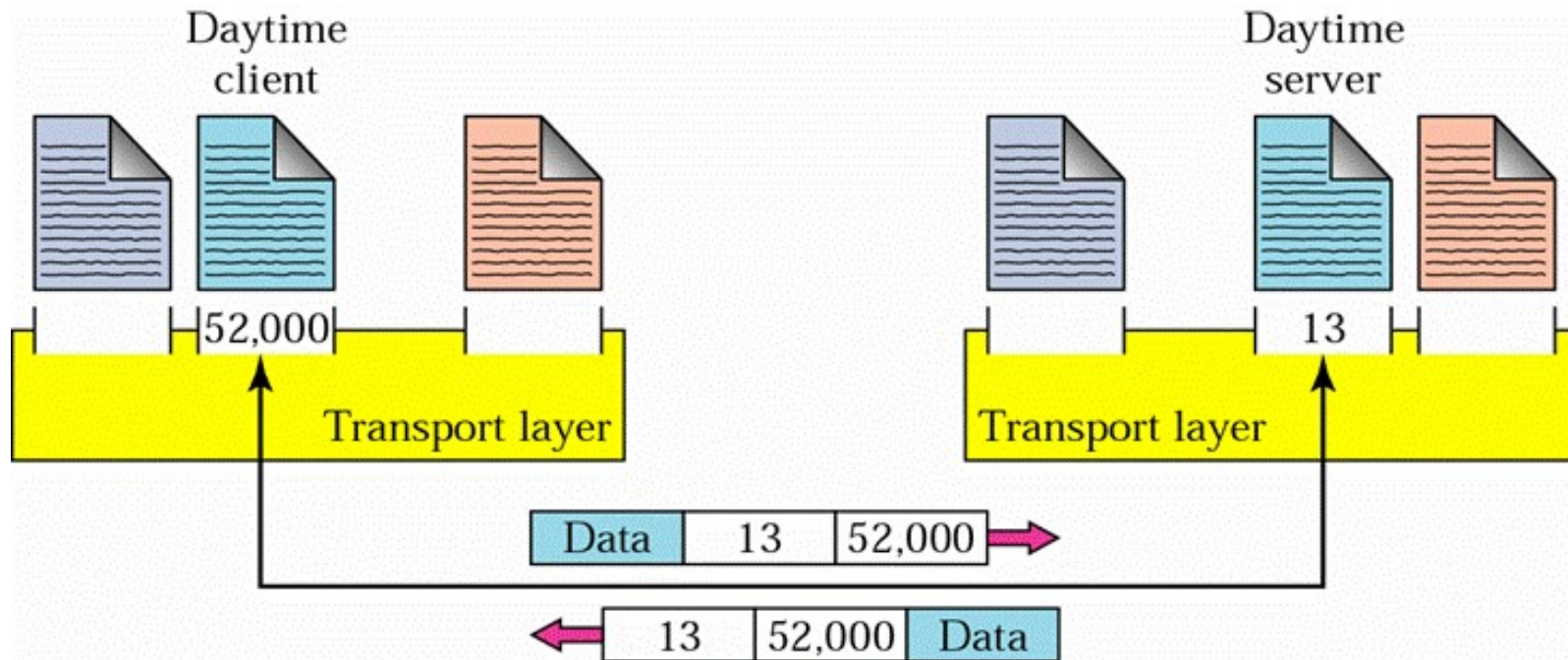
传输层：中间是网络（或互联网）

进程-进程通信涉及下列问题

- 寻址
- 多路复用与解复用
- 连接建立与释放
- 差错控制
- 流量控制与缓存
- 拥塞控制

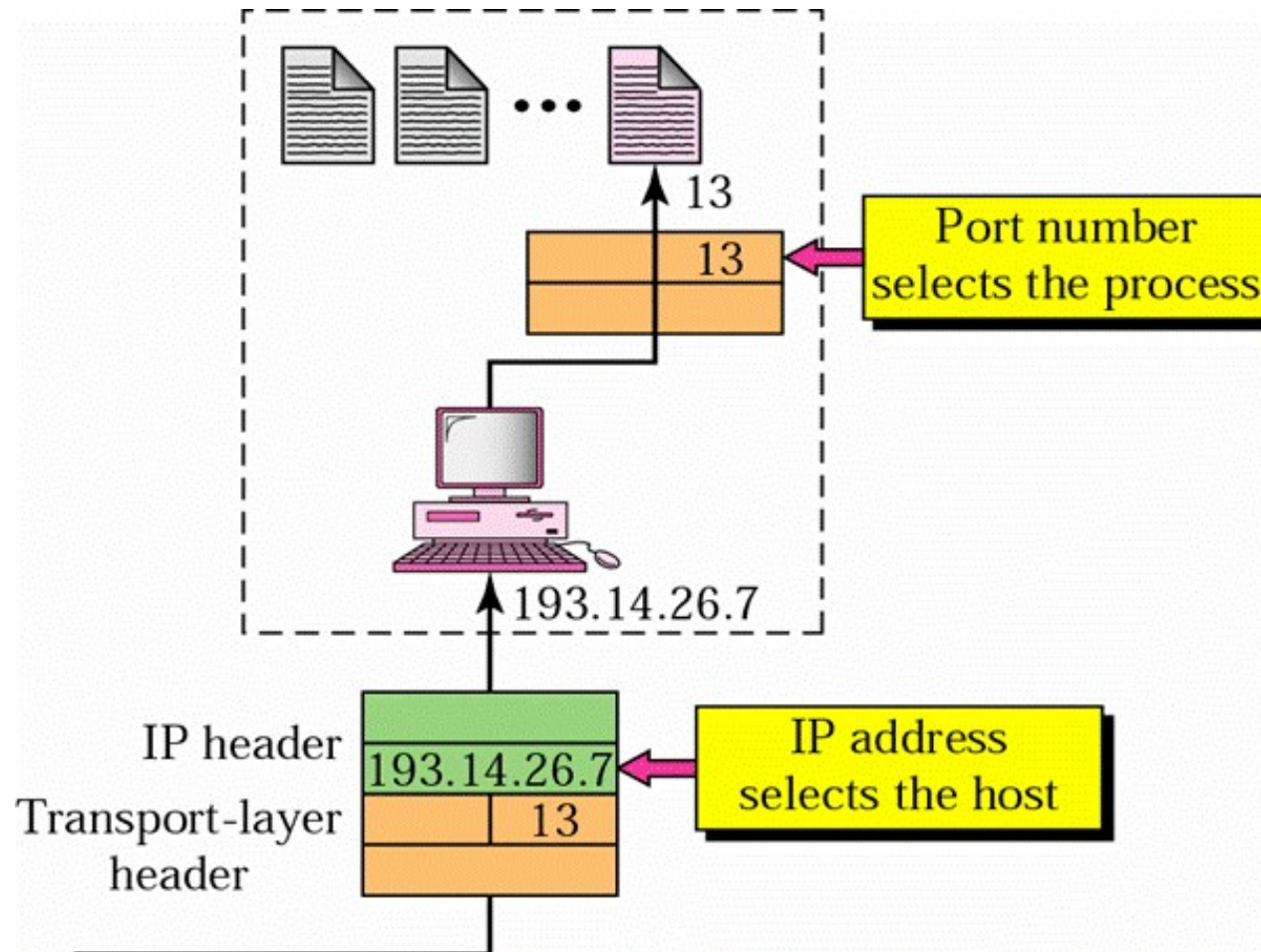
寻址：端口号

- 是应用进程的服务访问点(SAP)
- 决定哪个应用进程获得哪个消息



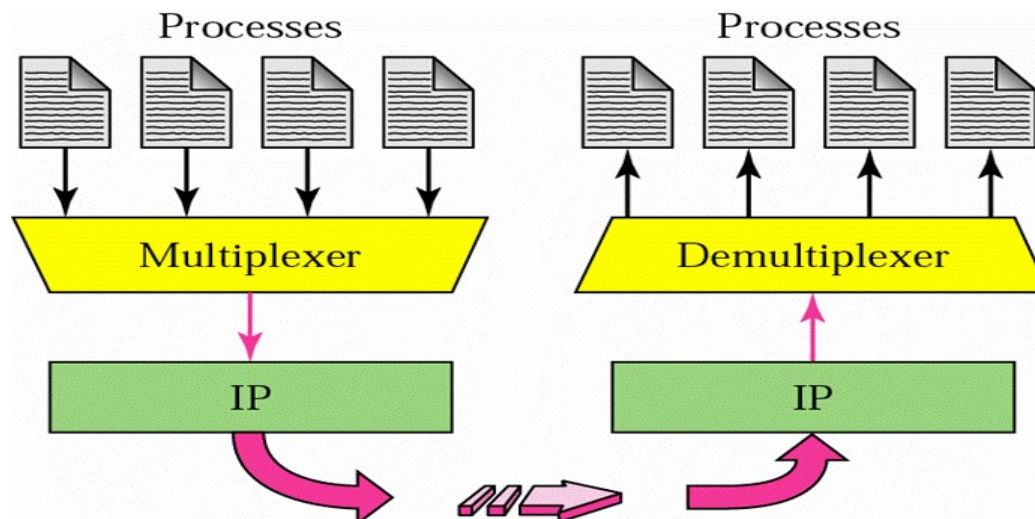
[Forouzan]

IP地址与端口号



[Forouzan]

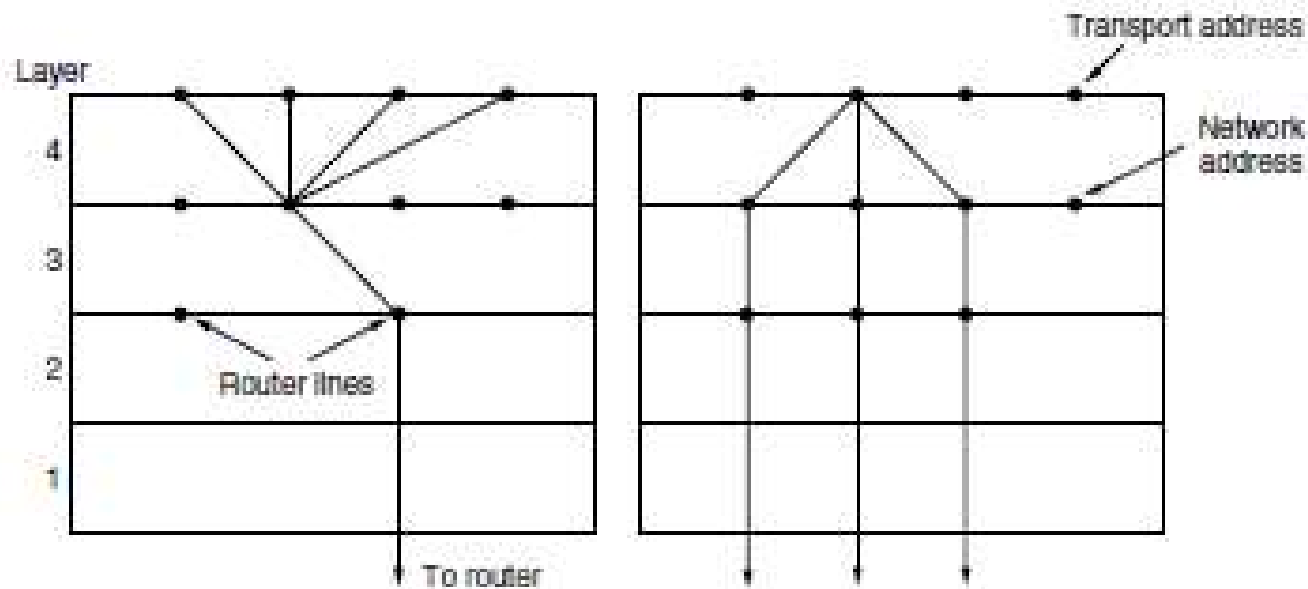
Multiplexing(复用)



- 在发送端，传输层完成复用：从不同的应用进程接收数据报文，以端口号区分
- 在接收端，完成解复用：传输层根据端口号，将报文交给对应的进程

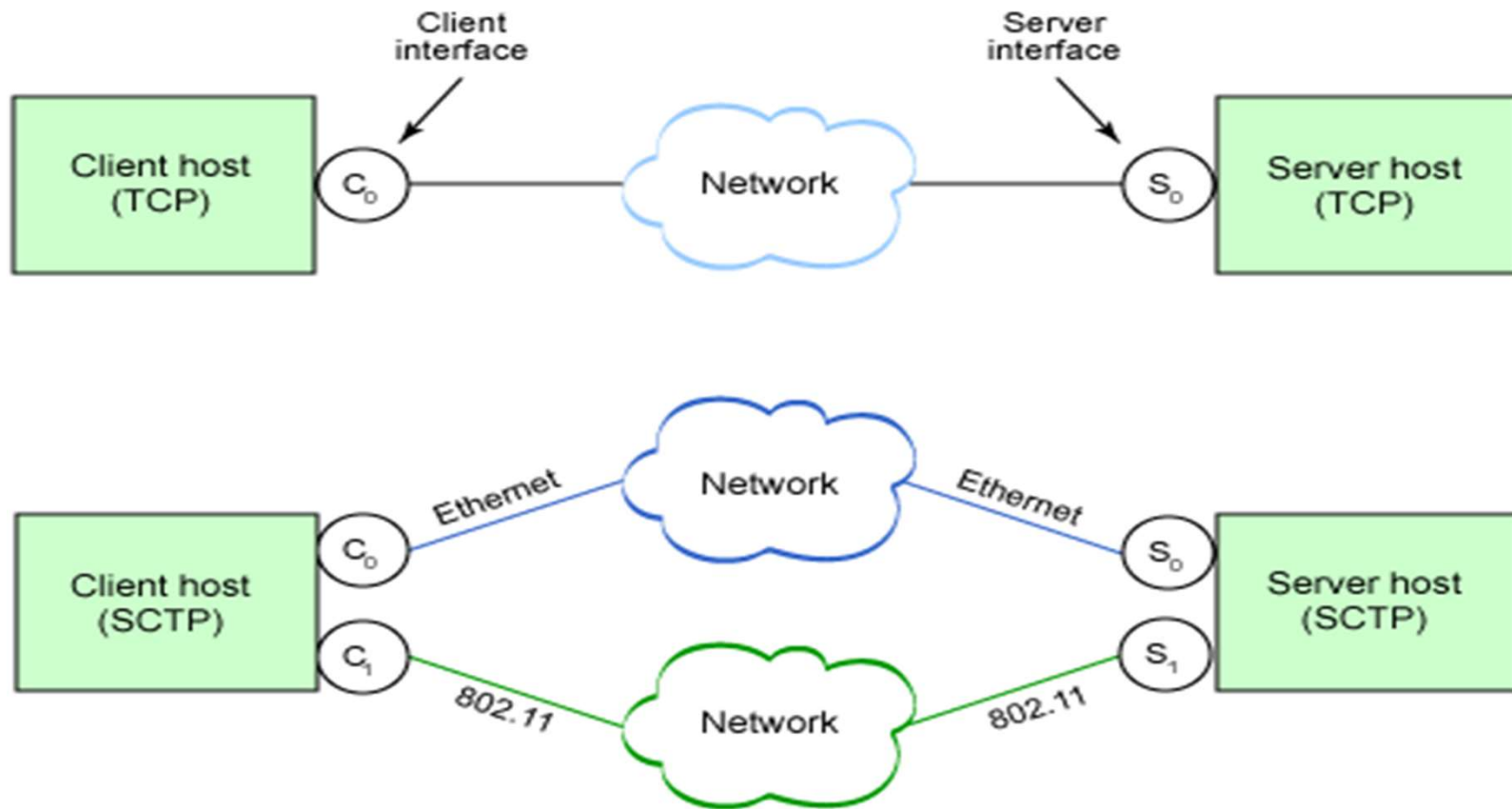
[Forouzan]

逆向复用(Fig.6-17)



- 应用于流控制传输协议SCTP
- RFC 4960
- 一个连接使用多个网络接口

TCP连接 与 SCTP联合



<https://www.ibm.com/developerworks/library/l-sctp/>

连接建立：问题

- 建立一个连接听起来很容易

- ◆ 两次握手

- ◆ 发送方向接收方发送`ConnectionRequest`消息，然后等待`ConnectionAccepted`应答

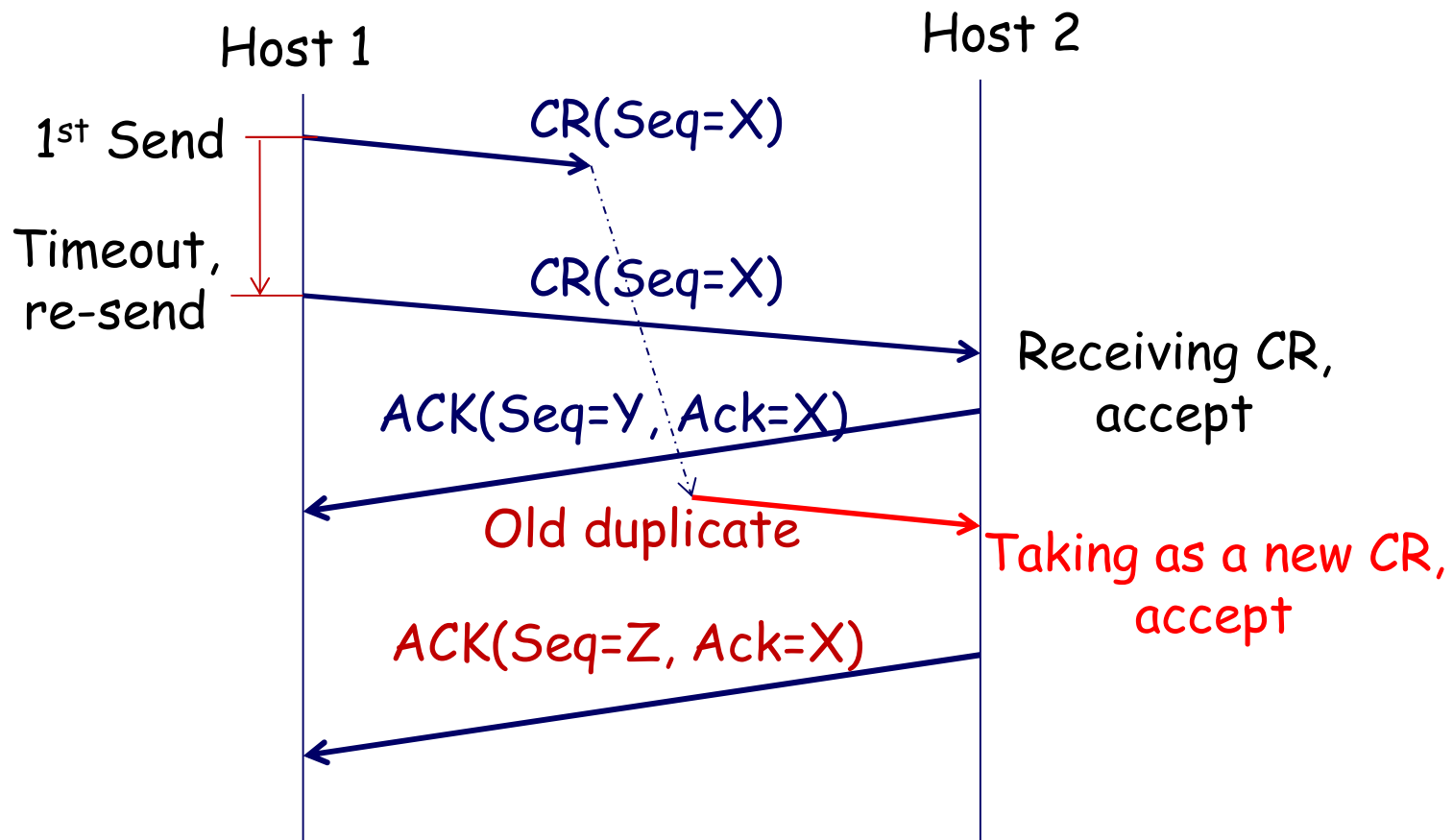
- 存在的问题

- ◆ 网络可能延误数据包，导致重复的TPDU（传输层协议数据单元）

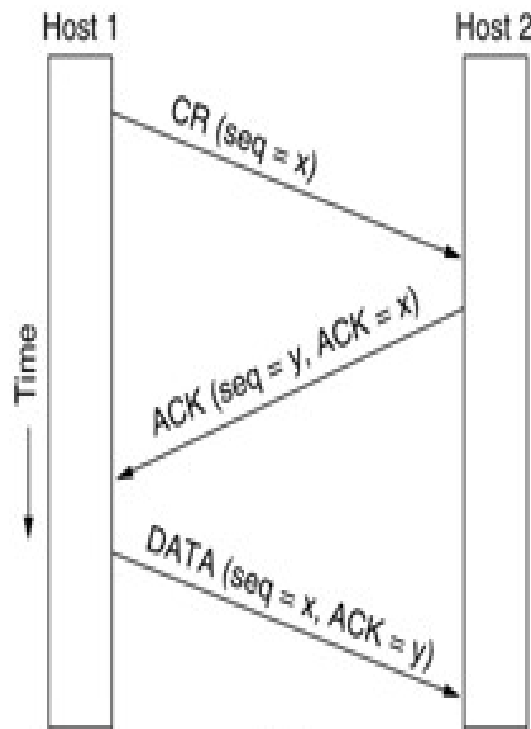
- ◆ 接收方无法区分重复数据，当做新数据来处理

被延迟的重复连接请求(CR)

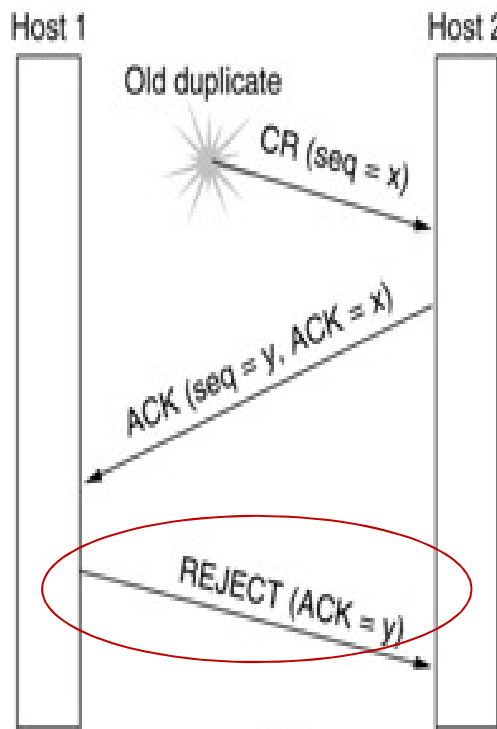
- 连接请求(CR)被网络延迟并重传, 导致接收方主机2将重复的CR当做新的CR进行处理



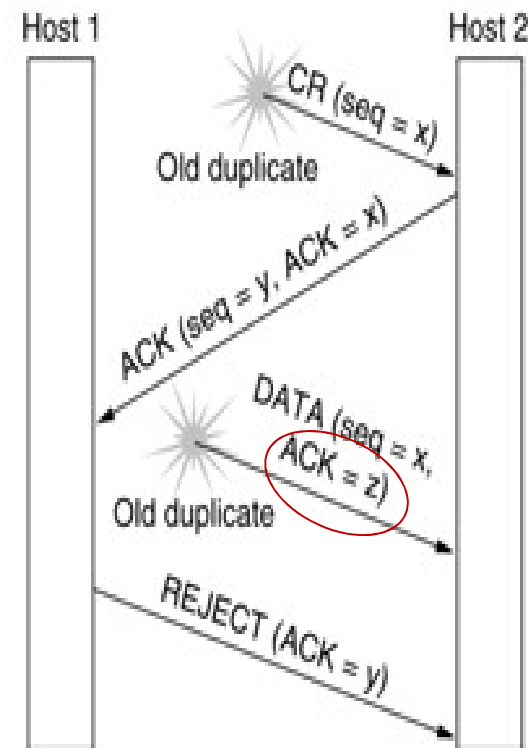
解决方案：三次握手



(a)
正常操作



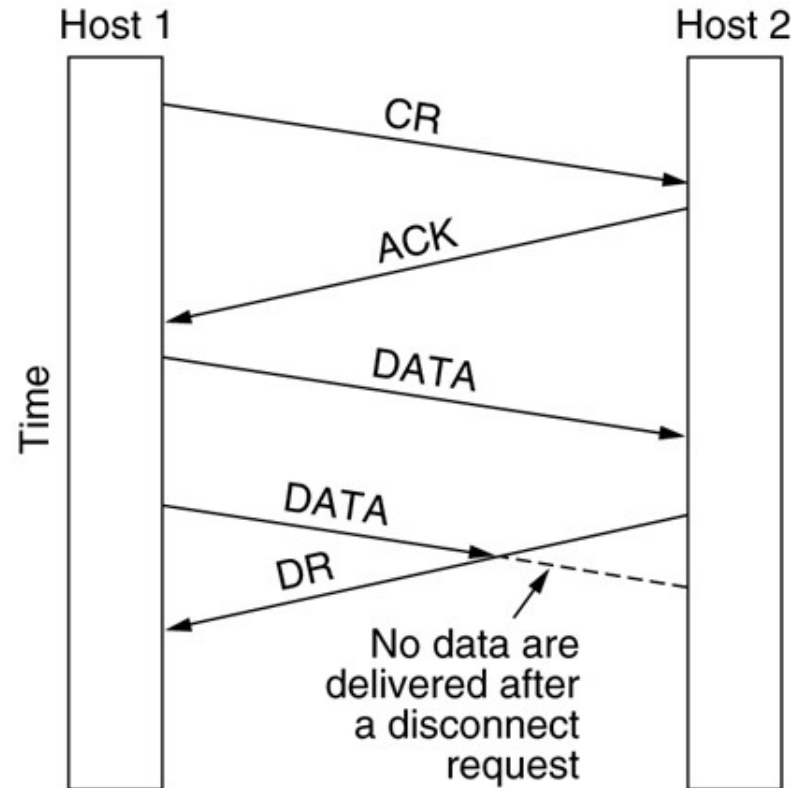
(b)
重复的连接请求



重复的连接请求
和重复的ACK

连接释放：问题

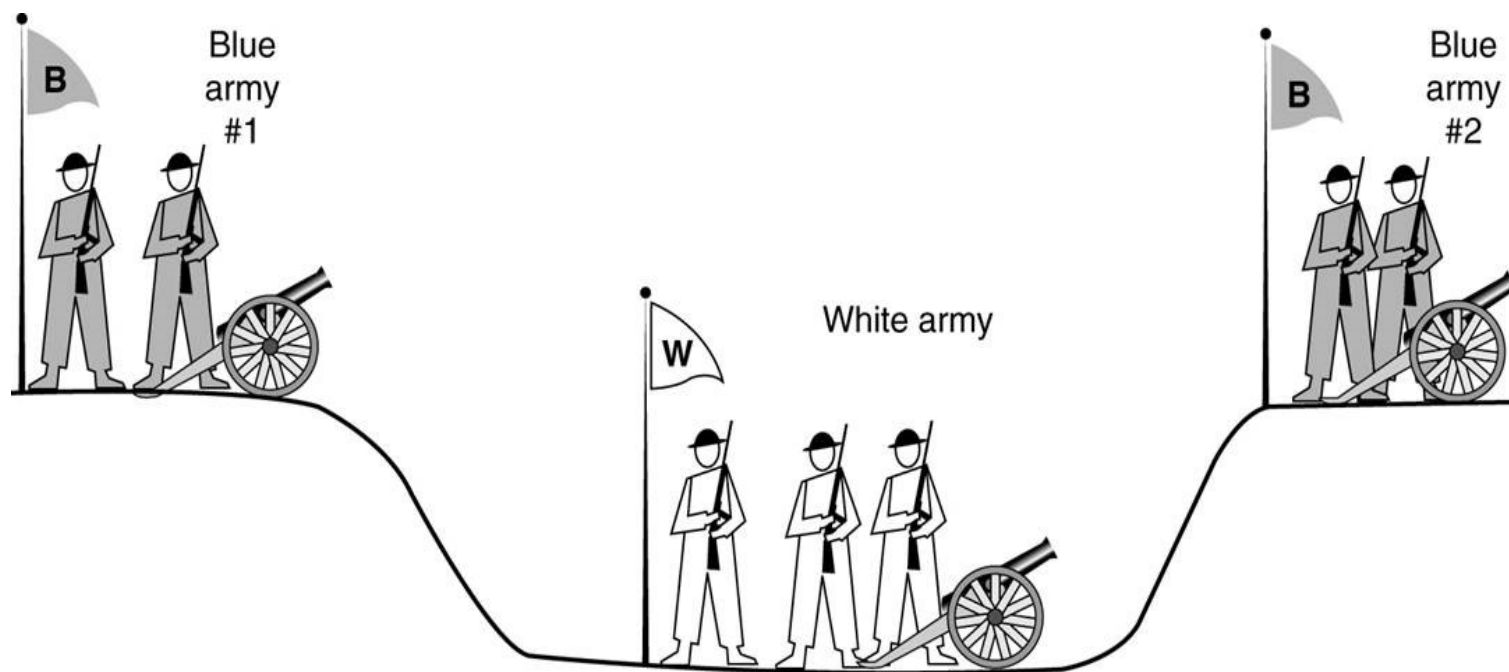
- 非对称释放：类似电话系统，突然释放可能导致数据丢失



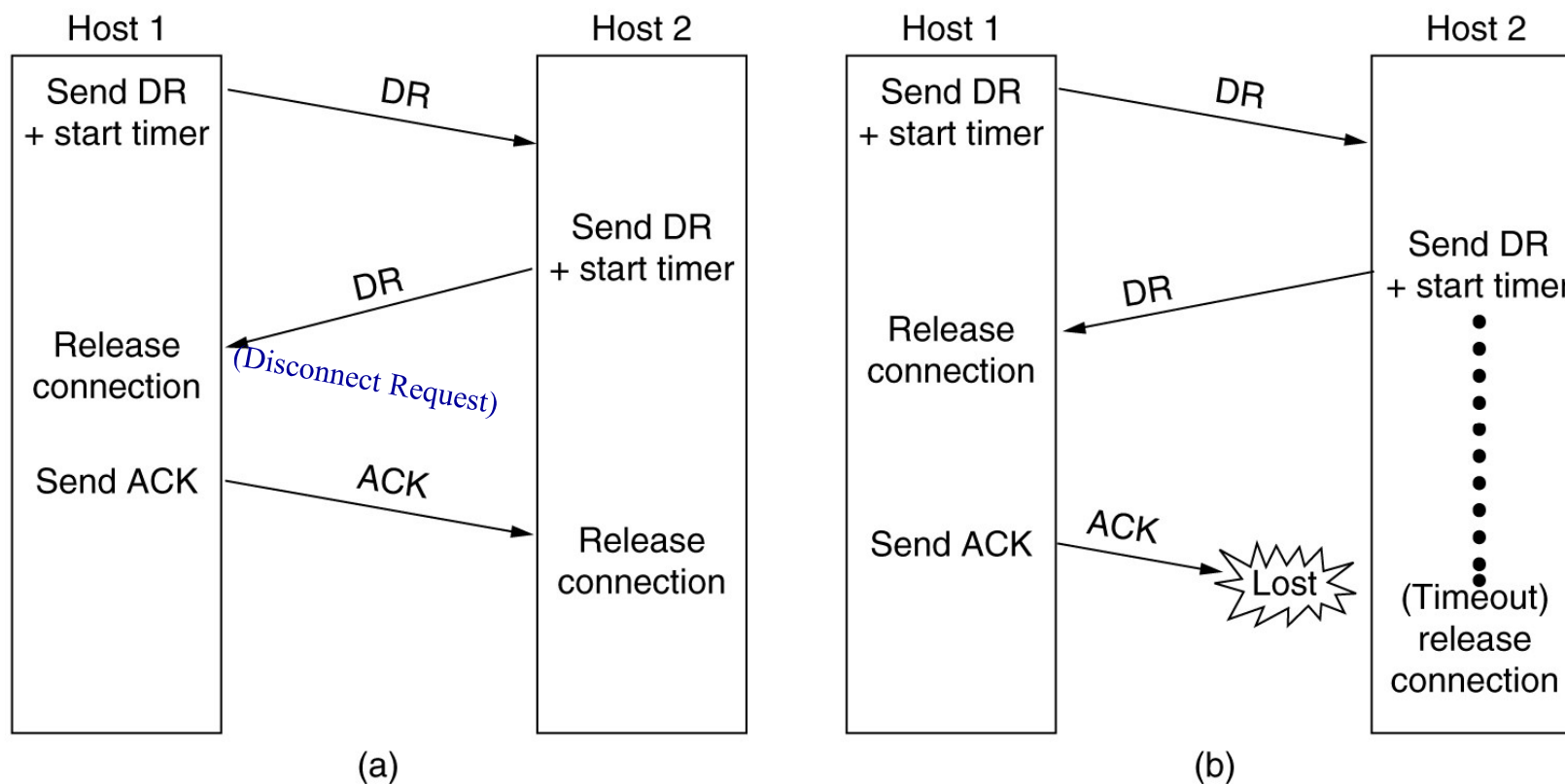
- 对称释放：每个方向分别释放连接，独立于另一个方向

连接释放：两军对垒问题

- 两支蓝军需要同时攻击白军才能获胜;否则他们就会被打败
- 蓝军只能穿过白军控制的区域才能通信，白军可以拦截信使



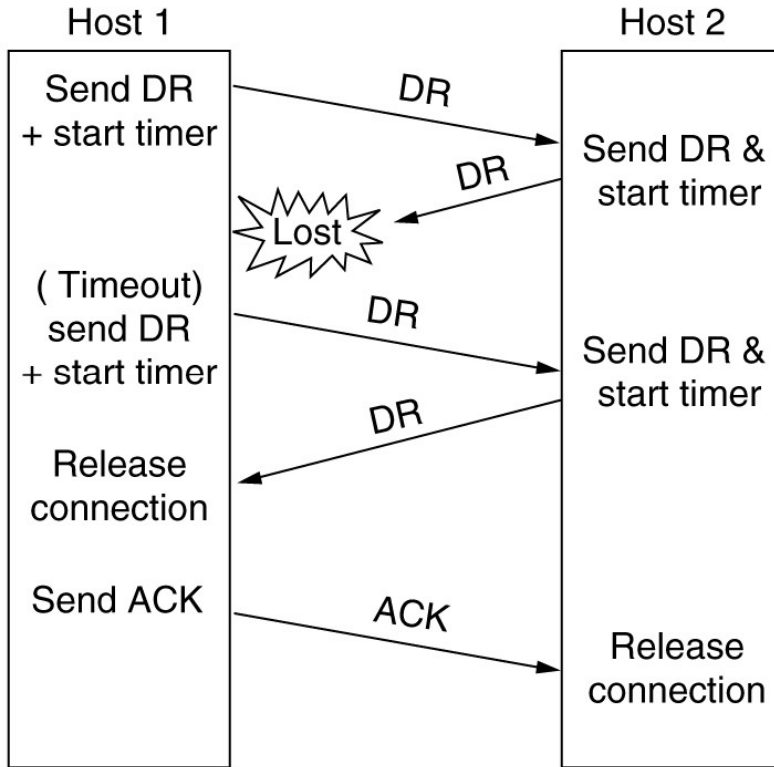
连接释放的解决方案：三次握手



(a) 正常的三次握手

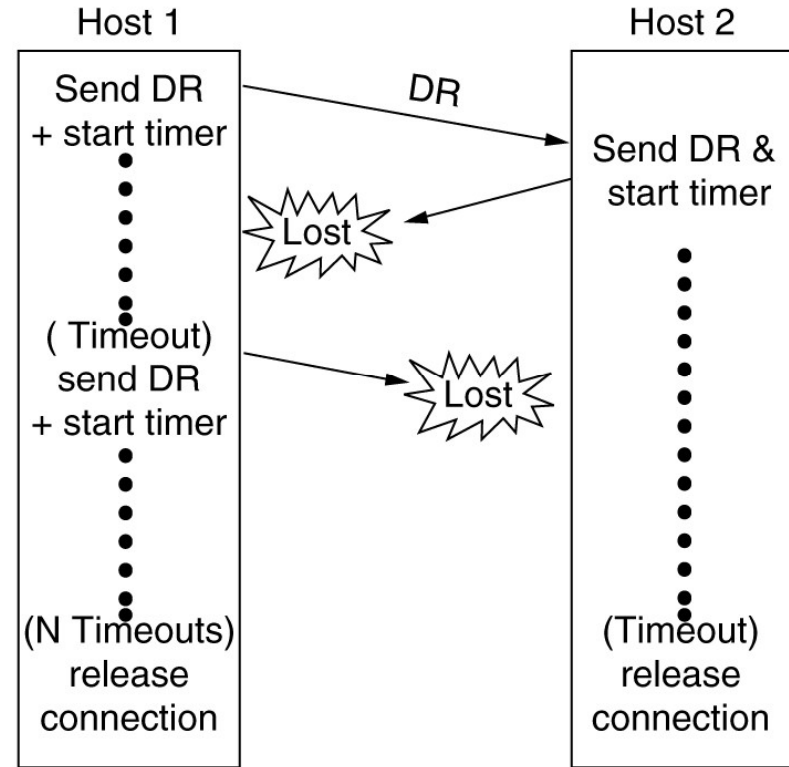
(b) 最后一个ACK丢失

连接释放



(c)

(c) 响应丢失



(d)

(d) 响应和随后的DR消息丢失

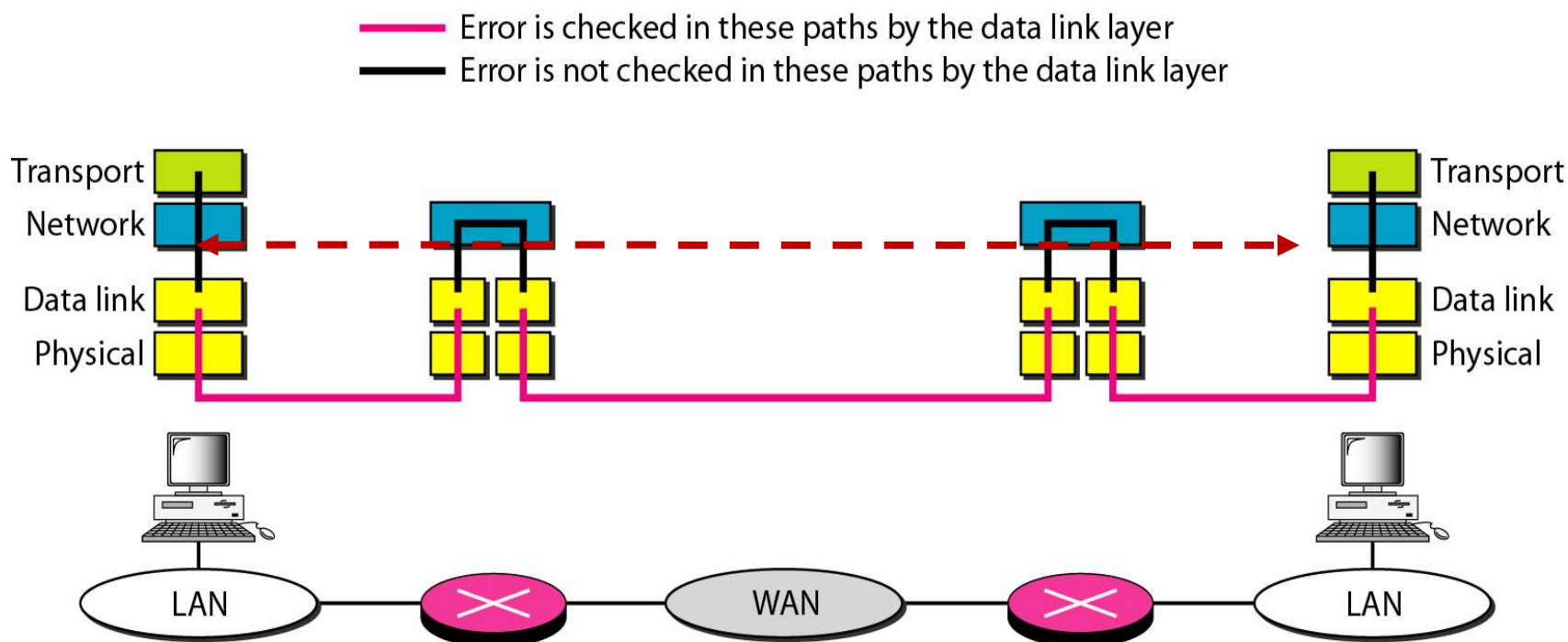
差错控制

■ 自动请求重传(ARQ)

- ◆ 报文段携带**检错码**（如校验和）
- ◆ 报文段携带用于标识本段的**序号**，接收方成功接收后返回**确认**；发送方如果没有收到确认，将重发段
- ◆ **滑动窗口协议**结合了上述特性，并支持双向数据传输。

■ 端到端/进程-进程

差错控制：传输层 与 数据链路层



■ 功能上的差异

◆ 一条链路上的可靠性 **vs.** 跨越网络路径的可靠性

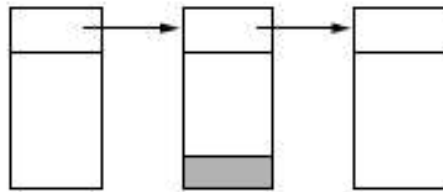
■ 程度上的差异

◆ 小滑动窗口 **vs.** 大滑动窗口（复杂）

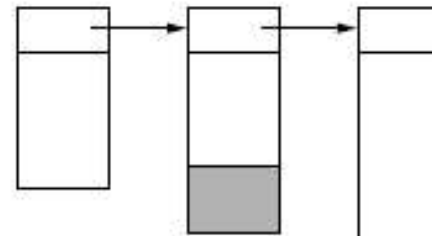
缓存的权衡

- 主要依靠发送方缓存还是接收方缓存？
 - ◆ 取决于连接之上的业务量的类型
- 对于低带宽的突发性业务, 如BBS, 不用预先分配缓存, 动态分配, 由发送方缓存已发出、未确认的数据
- 对于文件传输和其他高带宽业务, 接收方应该提供一个专用的满窗口的缓冲区, 以允许数据以最大速率发送。
 - ◆ TCP使用这种策略。

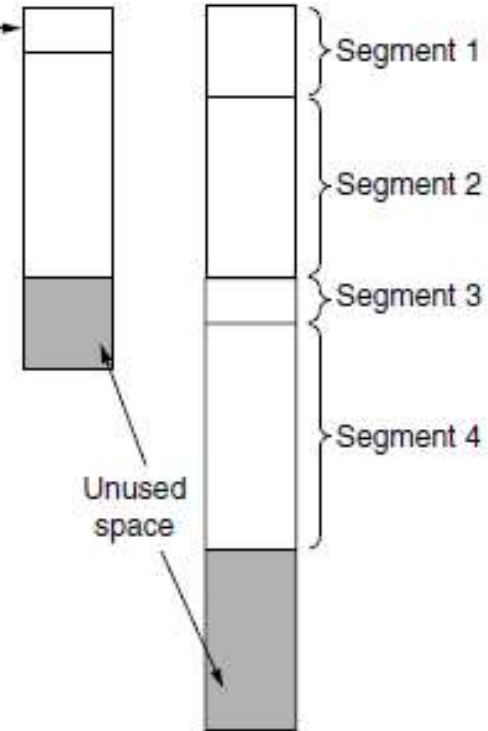
如何组织缓存池



固定大小缓存
的链接



可变大小缓存
的链接



每个连接
一个大的循环缓存

流量控制： 动态缓存分配

	<u>A</u>	<u>Message</u>	<u>B</u>	<u>Comments</u>
1	→	< request 8 buffers>	→	A wants 8 buffers
2	←	<ack = 15, buf = 4>	←	B grants messages 0-3 only
3	→	<seq = 0, data = m0>	→	A has 3 buffers left now
4	→	<seq = 1, data = m1>	→	A has 2 buffers left now
5	→	<seq = 2, data = m2>	...	Message lost but A thinks it has 1 left
6	←	<ack = 1, buf = 3>	←	B acknowledges 0 and 1, permits 2-4
7	→	<seq = 3, data = m3>	→	A has 1 buffer left
8	→	<seq = 4, data = m4>	→	A has 0 buffers left, and must stop
9	→	<seq = 2, data = m2>	→	A times out and retransmits
10	←	<ack = 4, buf = 0>	←	Everything acknowledged, but A still blocked
11	←	<ack = 4, buf = 1>	←	A may now send 5
12	←	<ack = 4, buf = 2>	←	B found a new buffer somewhere
13	→	<seq = 5, data = m5>	→	A has 1 buffer left
14	→	<seq = 6, data = m6>	→	A is now blocked again
15	←	<ack = 6, buf = 0>	←	A is still blocked
16	...	<ack = 6, buf = 4>	←	Potential deadlock

- 箭头表示传输方向
- 省略号(...)表示丢失了报文段

什么是拥塞？

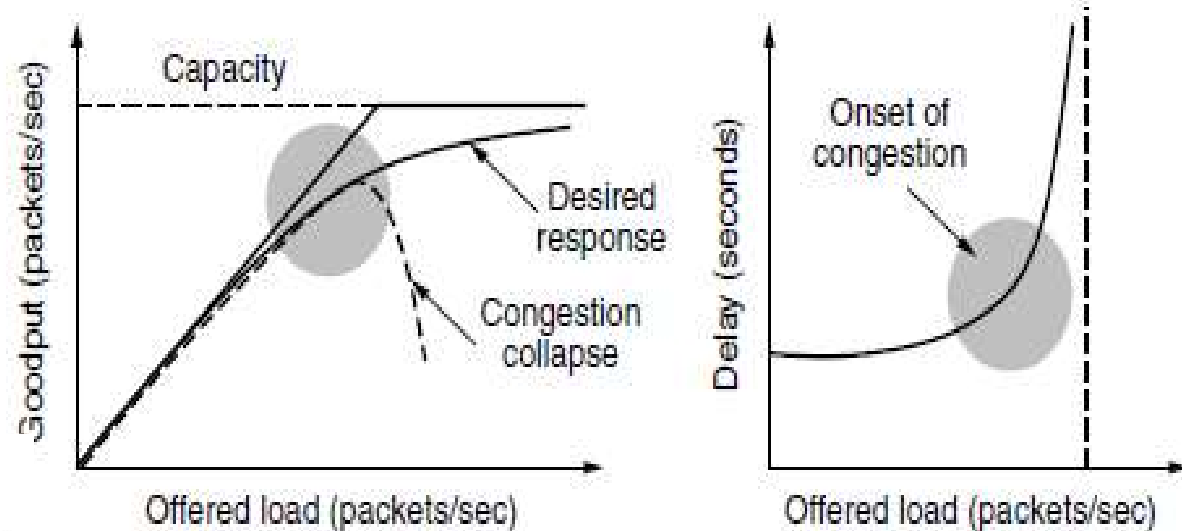
- 如果太多主机的传输实体太快地向网络中发送了太多数据包，网络就会变得拥塞

- 症状：

- ◆ 数据包丢失
- ◆ 时延变长

- 效率与功率

$$power = \frac{load}{delay}$$



衡量拥塞程度的指标

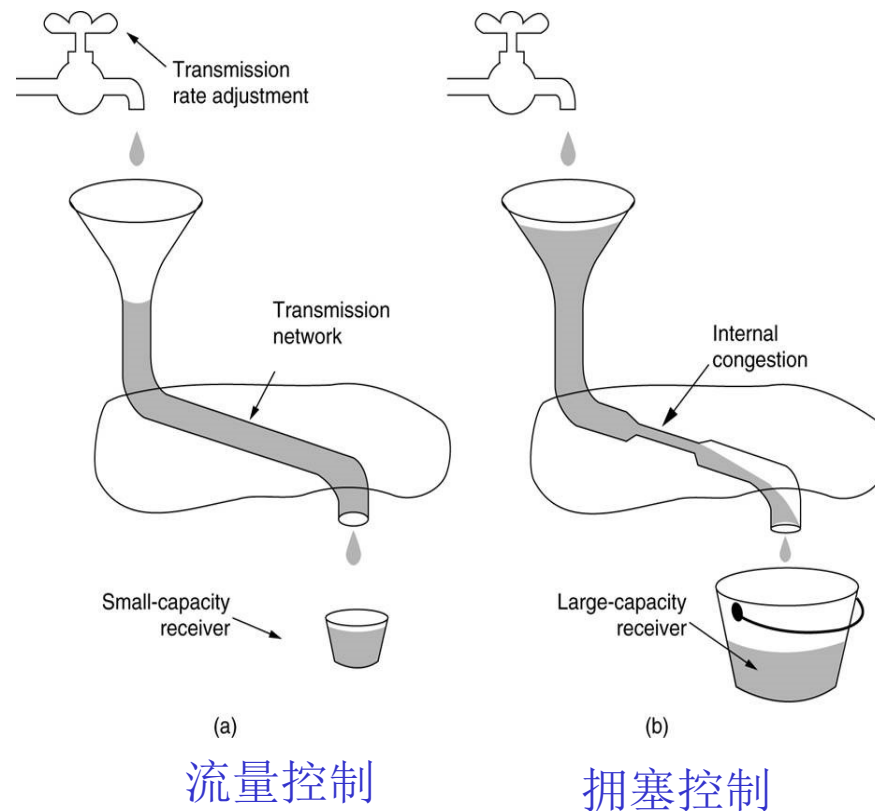
达到最大**功率**时的负载即是传输实体发送到网络的高效负载

拥塞控制

■ 控制拥塞、将负载限制在容量之下的机制和技术

➤ 或者在拥塞发生之前预防拥塞，或者在拥塞发生后去除拥塞

■ 与流量控制不同



共同的解决方法：调整发送速率

拥塞控制（续）

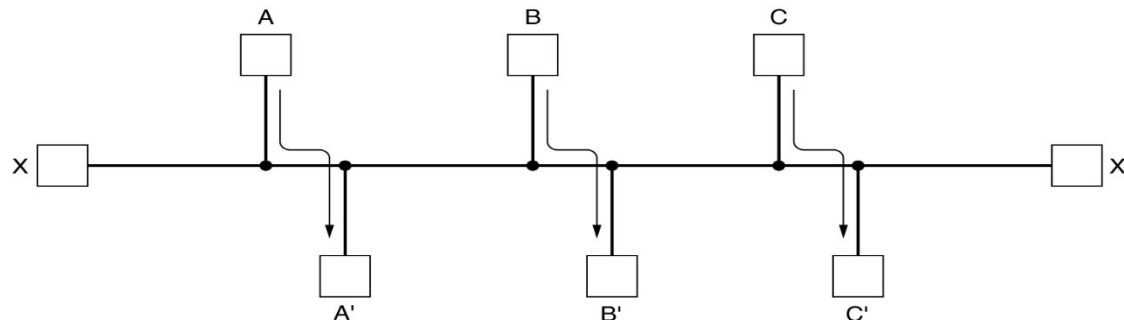
■ 网络层和传输层在拥塞控制中的作用

- ◆ 拥塞在路由器上发生, 因此网络层负责检测拥塞 (长时延、分组丢失)
- ◆ 对传输层而言, 唯一有效的控制拥塞的手段是降低向网络中发送数据包的速度

■ 因特网的拥塞控制很大程度上依赖传输层协议TCP实现

■ 拥塞控制算法的目标

- ◆ 避免拥塞
- ◆ 找到一种好的带宽分配方案, 尽量利用全部可用带宽 (高效性)
- ◆ 公平对待竞争的传输实体 (公平性)
- ◆ 能快速跟踪流量需求的变化 (收敛性)

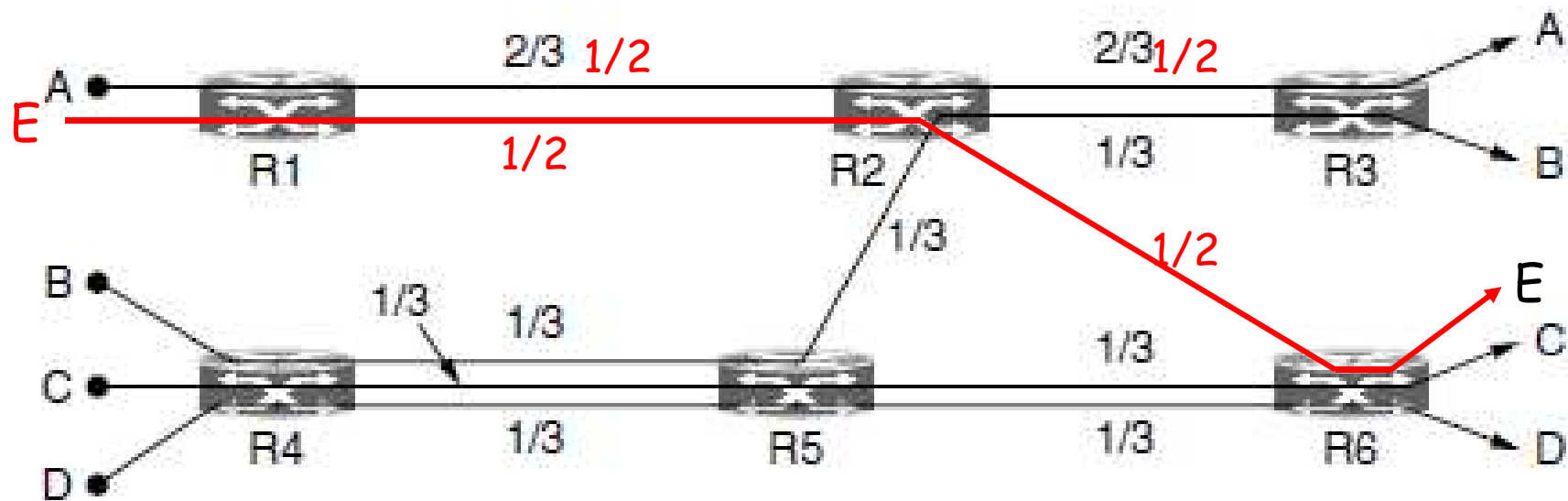


如何在各传输层发送方之间公平分配带宽？

■ 最大最小公平性

- ◆ 分配是可行的，当且仅当试图增加任何参与者的分配必然会导致分配相等或更小的其他参与者的分配减少
- ◆ 基本含义：使得资源分配向量的最小分量的值最大，防止任何网络流被‘饿死’，同时在一定程度上尽可能增加每个流的速率。
- ◆ 原则：一个信道上，在满足最小需求的前提下，各流尽量均分带宽；如果增加任何一个参与者（分量）的带宽，将导致其它的具有相同或者更少带宽的参与者（分量）的带宽下降，此时即满足了最大最小公平性。

最大最小公平性举例



A-D: 4个流

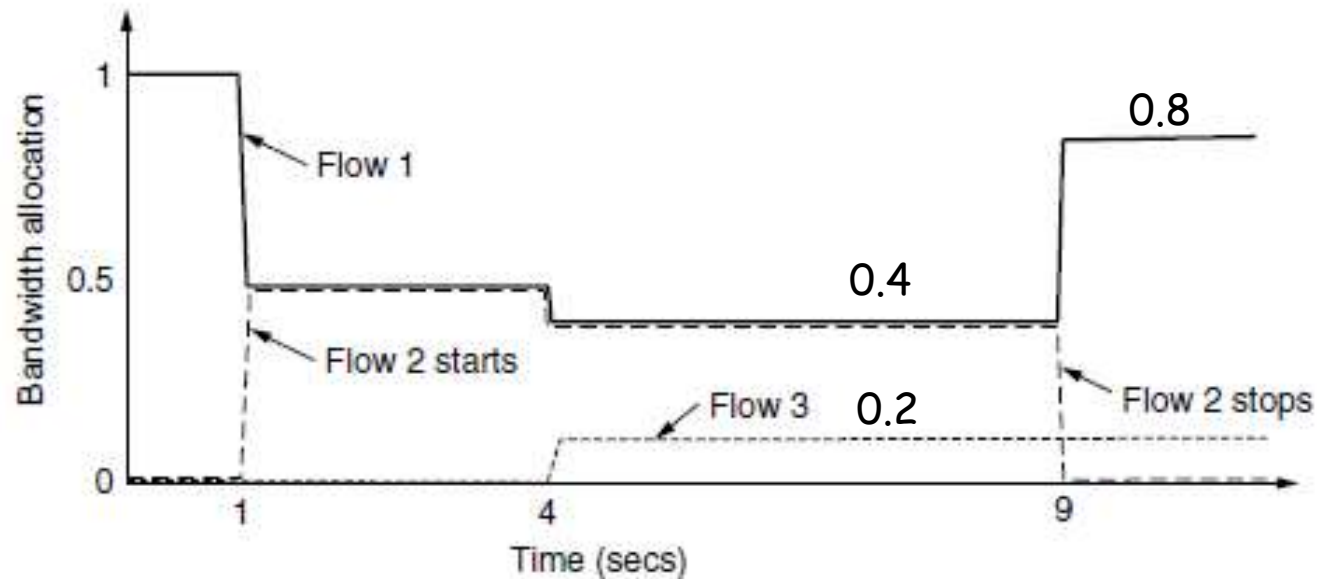
每个链路具有相同的容量, 单位为1

如果增加B的带宽, 将导致与B具有相同带宽的C和D的带宽减少, 因此当前分配已满足最大最小公平性

问题: 假如加入一个新的流E, 路径E→R1→R2→R6, 请问5个流的带宽如何变化?

收敛性

- 好的拥塞控制算法应该快速收敛到理想的操作点(公平且高效的带宽分配)，并且应该随着时间的变化能够跟踪该点。（尽快达到理想分配）



在任何时候，总分配带宽大约是100%，因此网络被充分利用，竞争流得到平等的对待

Internet的拥塞控制

■ 三步走

1. 发现拥塞：路由器根据队列长度或者线路利用率判断
2. 将拥塞情况通知源主机

显式拥塞通知：TCP with ECN

或 隐式拥塞通知：源主机(TCP)自己根据丢包情况判断

3. 源主机采取措施 (TCP)

在无拥塞时，增加发送速率；

在出现拥塞时，降低发送速率

拥塞控制协议的信号

Protocol	Signal	Explicit?	Precise?
XCP	Rate to use	Yes	Yes
TCP with ECN	Congestion warning	Yes	No
FAST TCP	End-to-end delay	No	Yes
Compound TCP	Packet loss & end-to-end delay	No	Yes
CUBIC TCP	Packet loss	No	No
TCP	Packet loss	No	No

■ 如何调整发送速率

- ◆ 如果无拥塞信号，发送端应该增加发送速率
- ◆ 给出拥塞信号时，发送端应该降低发送速率

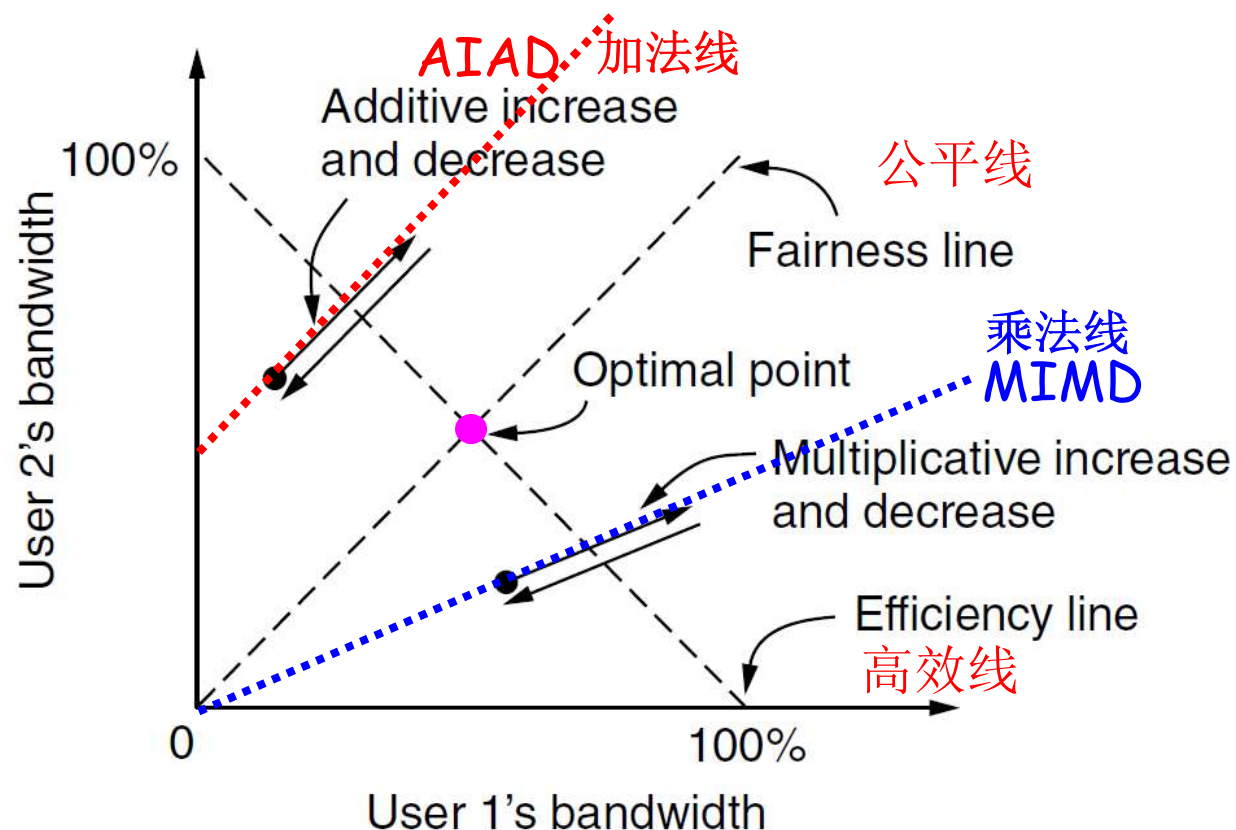


具体原则参见TCP

控制法则：AIMD(加法增乘法减)

■ 动机：达到高效性和公平性的操作点

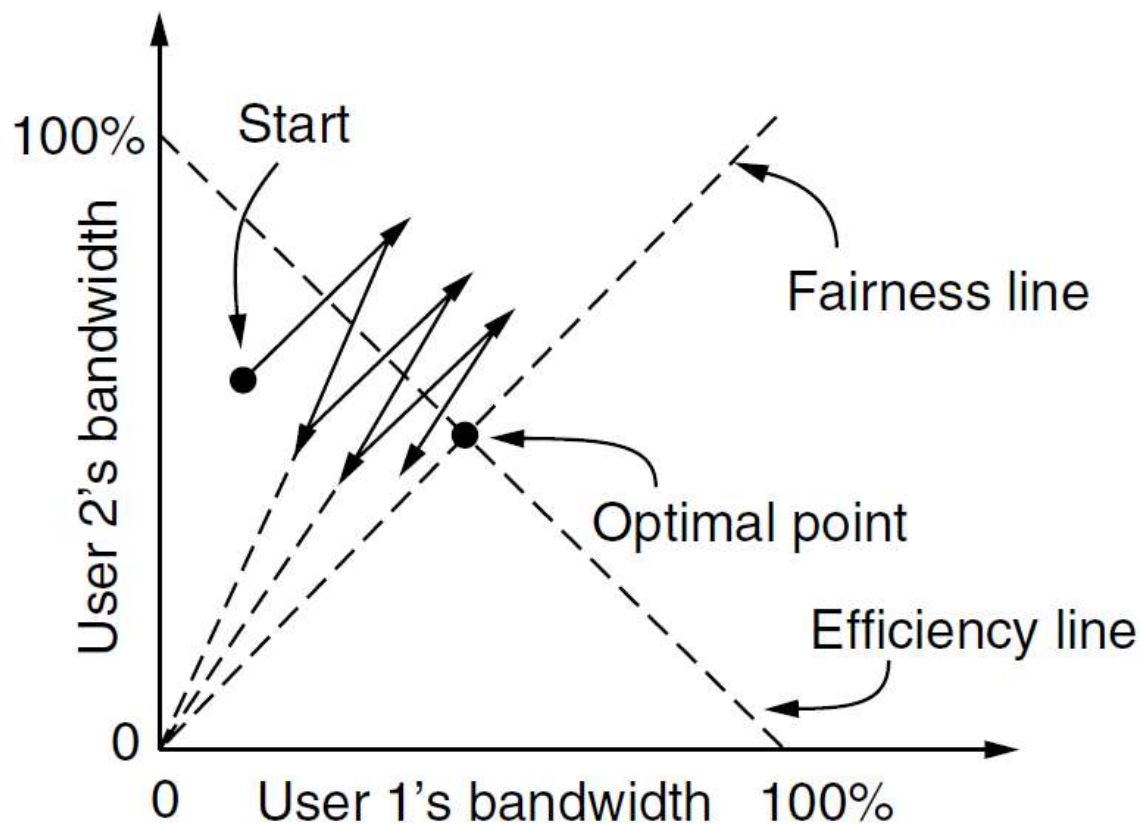
假设两个传输层
连接竞争一个
链路的带宽



AIAD、MIMD很难收敛到最优的操作点(既公平又高效)！

控制法则：加法增乘法减(AIMD)

- 加法增乘法减(AIMD): AIMD可以收敛于最优点, 兼具高效性、公平性和收敛性



TCP的拥塞控制法则

■ AIMD

◆ 稳定性观点：驱使网络拥塞很容易但是从中恢复却很难，因此带宽递增策略应该温和，带宽递减策略应该激进。

■ 实际中通常使用的策略是调整滑动窗口的大小，而不是直接调整速率

■ TCP友好的拥塞控制

◆ 新协议必须能与TCP进行公平的竞争

◆ TCP和非TCP传输协议可以自由地混合，而没有任何不良影响

思考(学习科学技术研究的方法)

■ 从网络发送速率调整方法的研究中我们能学到什么样的研究方法？

◆ 提出问题（功能需求）： 无拥塞时，增加发送速率；出现拥塞时，降低发送速率。

◆ 明确评价指标：效率+公平性

效率=竞争流的带宽占用率和为100%；公平性=1/竞争流数目

◆ 问题的数学建模（最关键，最重要，需要将网络技术问题做适当简化和假设）

◆ 数学问题求解

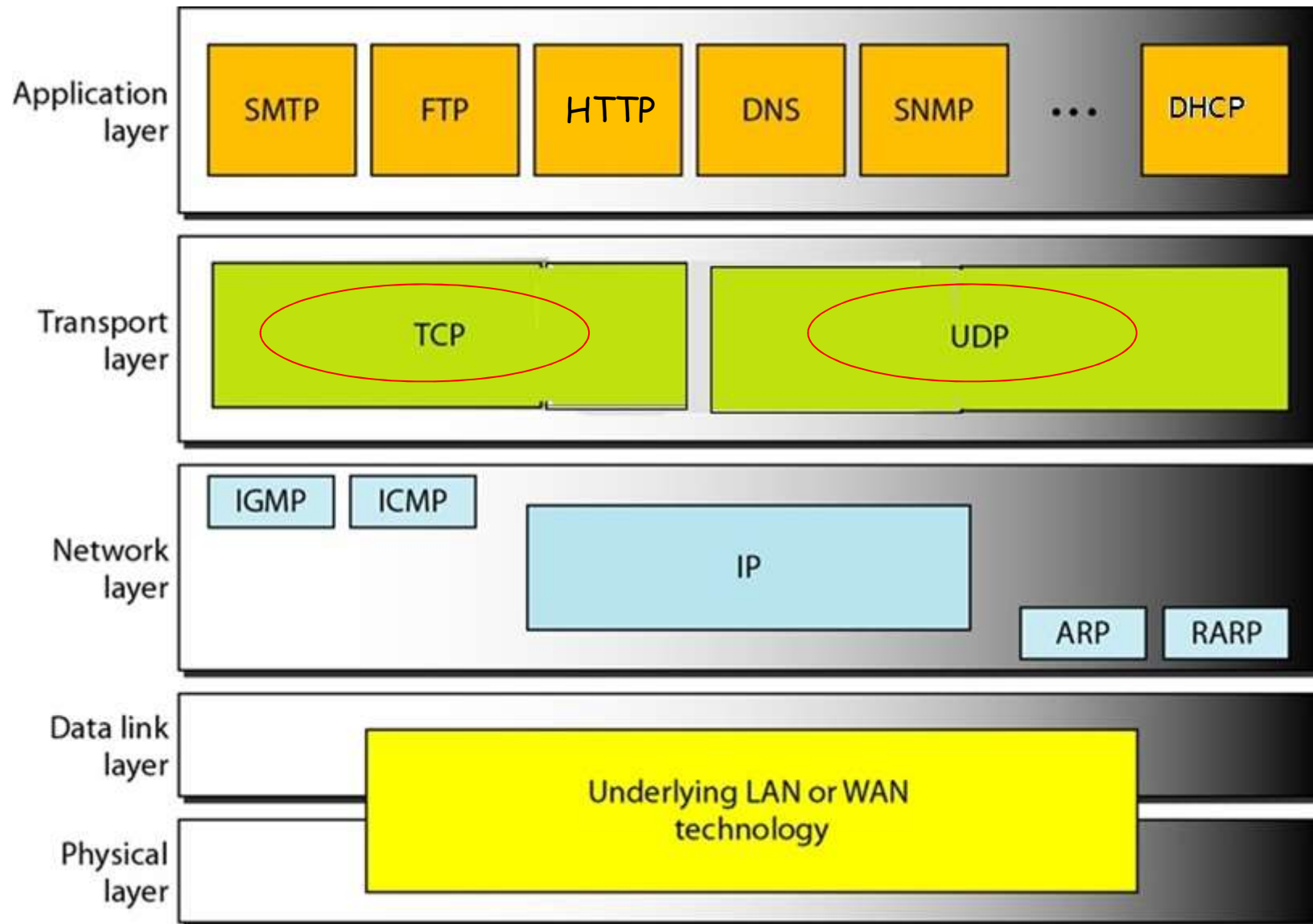
◆ 将数学问题求解过程映射回网络技术和协议的设计

◆ 实验验证或者原型验证

目录

- 传输层的功能与服务
- 传输协议要素
- 因特网的传输层协议
 - ◆ 用户数据报协议 (UDP)
 - ◆ 传输控制协议 (TCP)

TCP/IP模型中的传输层协议



UDP: 用户数据报协议

- 无连接
- 不可靠传输
 - ◆ 没有确认来表明数据已交付
 - ◆ 没有检测消息丢失或乱序的机制
 - ◆ 没有自动重传机制
 - ◆ 没有流量控制机制
 - ◆ 与IP一样提供尽力而为服务
- 提供IP之上的多路复用与解复用
- 仅需要最少的开销、计算和通信

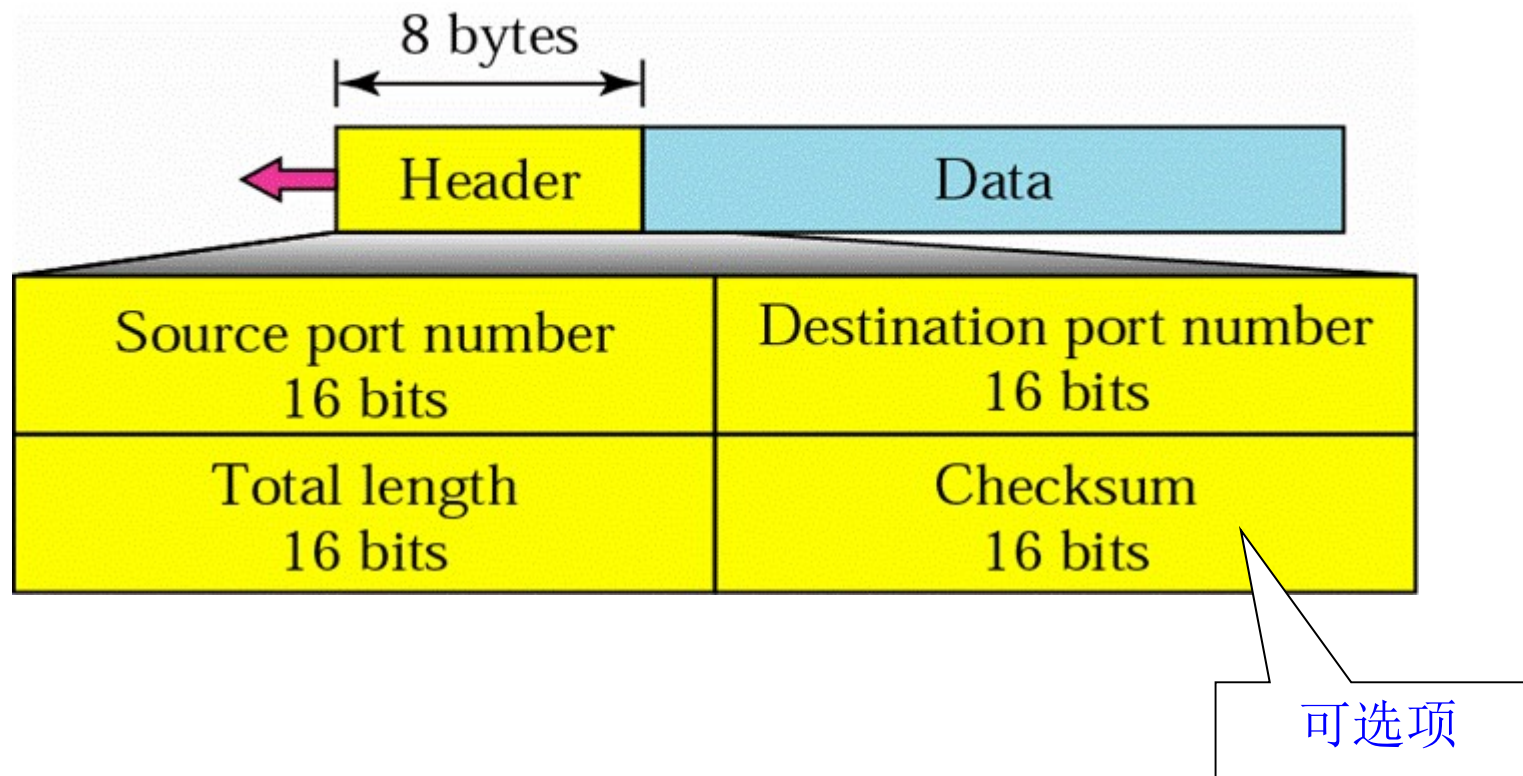
基于UDP的应用

- DNS域名解析(Domain Name System, 53)
- DHCP 动态主机配置(Dynamic Host Configuration Protocol, 67/68)
- RIP路由信息协议 (Routing Information Protocol)
(要求知道上述三种协议的传输层采用UDP协议)
- RTP (Real-time Transport Protocol)
- P2P (Peer-to-Peer, DHT, 'Kademlia')

UDP的常用端口

<i>Port</i>	<i>Protocol</i>	<i>Description</i>
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
53	Nameserver	Domain Name Service
67	Bootps	Server port to download bootstrap information
68	Bootpc	Client port to download bootstrap information
69	TFTP	Trivial File Transfer Protocol
111	RPC	Remote Procedure Call
123	NTP	Network Time Protocol
161	SNMP	Simple Network Management Protocol
162	SNMP	Simple Network Management Protocol (trap)

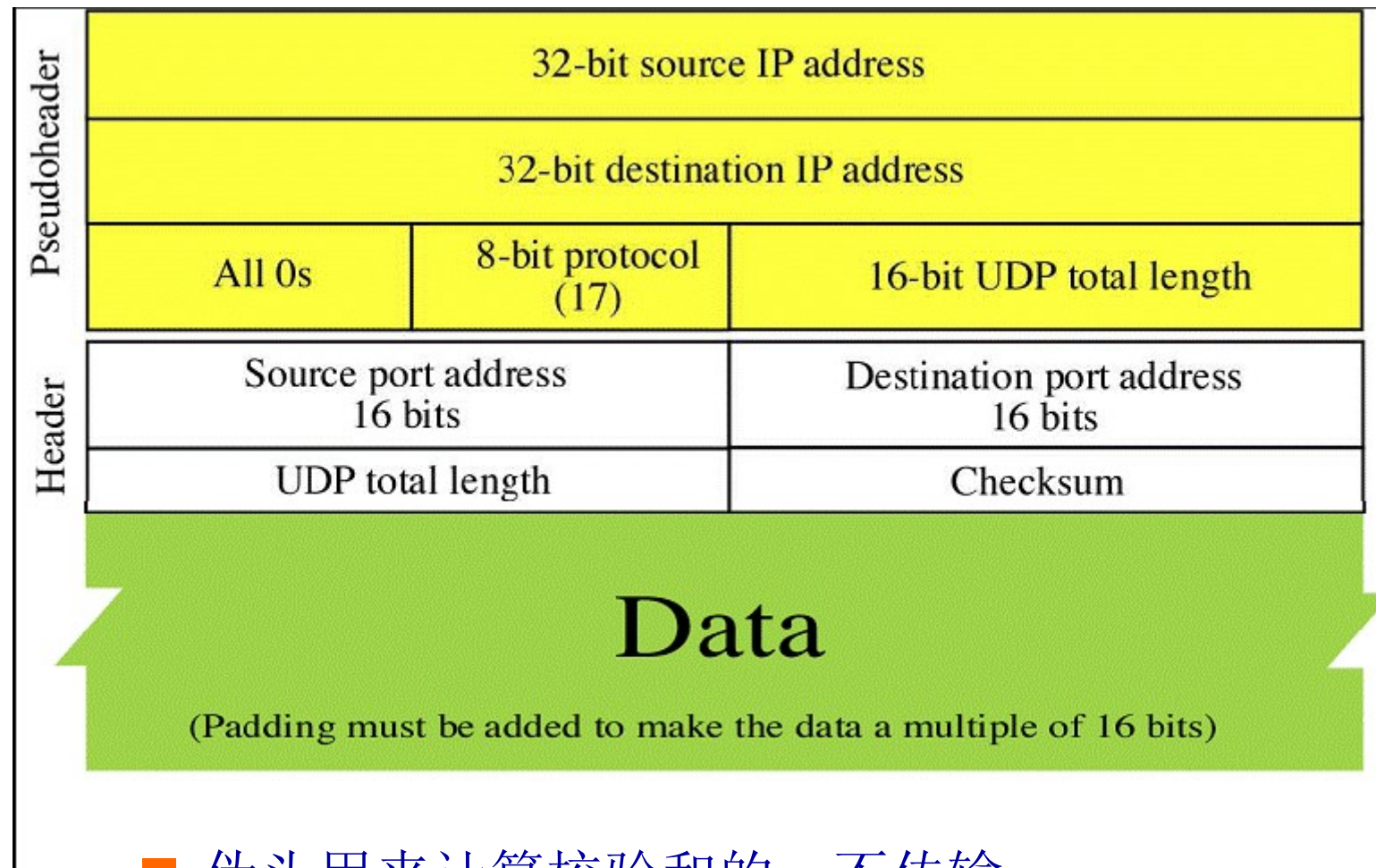
UDP数据报格式



UDP在发送方与接收方之间保留消息边界

[Forouzan]

伪报头



- 伪头用来计算校验和的，不传输

校验和的计算

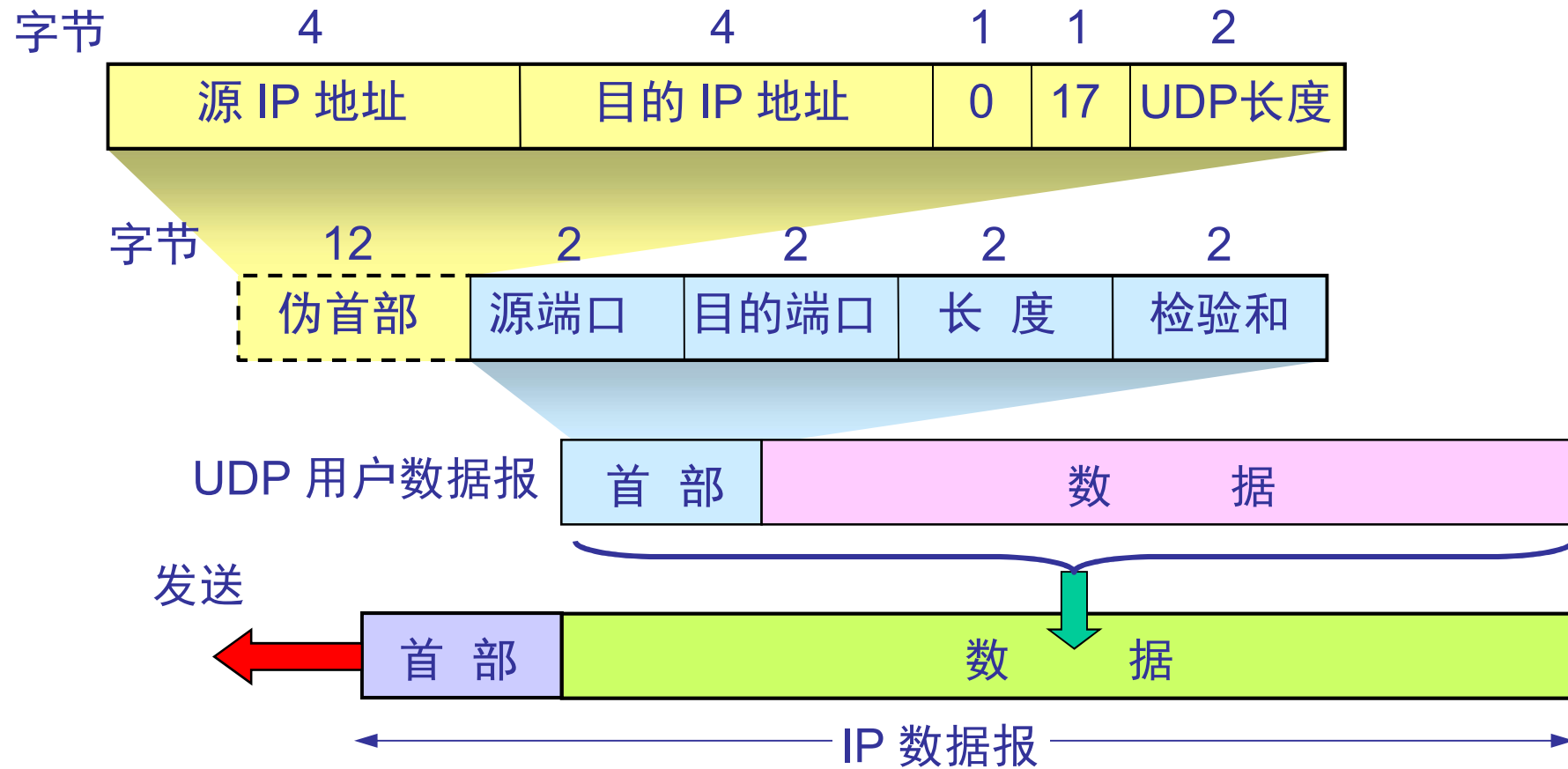
Same algorithm as used in IP

153.18.8.105			
171.2.14.10			
All 0s	17	15	
1087		13	
15		All 0s	
T	E	S	T
I	N	G	All 0s

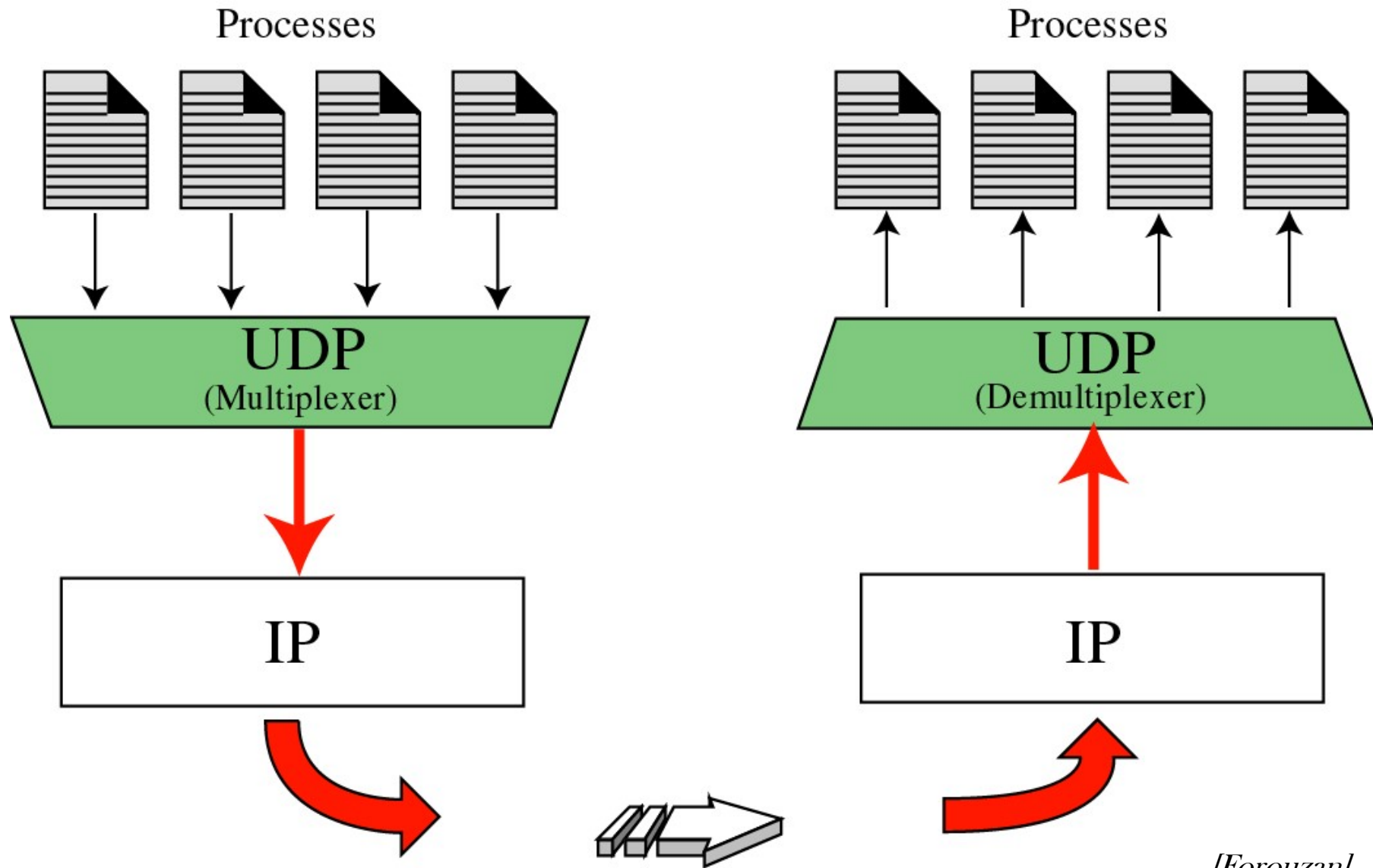
10011001 00010010	→	153.18
00001000 01101001	→	8.105
10101011 00000010	→	171.2
00001110 00001010	→	14.10
00000000 00010001	→	0 and 17
00000000 00001111	→	15
00000100 00111111	→	1087
00000000 00001101	→	13
00000000 00001111	→	15
00000000 00000000	→	0 (checksum)
01010100 01000101	→	T and E
01010011 01010100	→	S and T
01001001 01001110	→	I and N
01000111 00000000	→	G and 0 (padding)
10010110 11101011	→	Sum
01101001 00010100	→	Checksum

当用户数据报传递给IP时，伪报头和填充被删除

UDP 校验和



多路复用和解复用

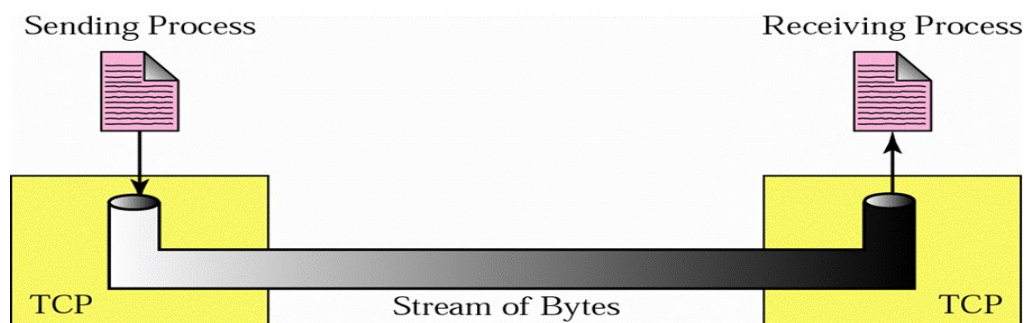


TCP: 传输控制协议

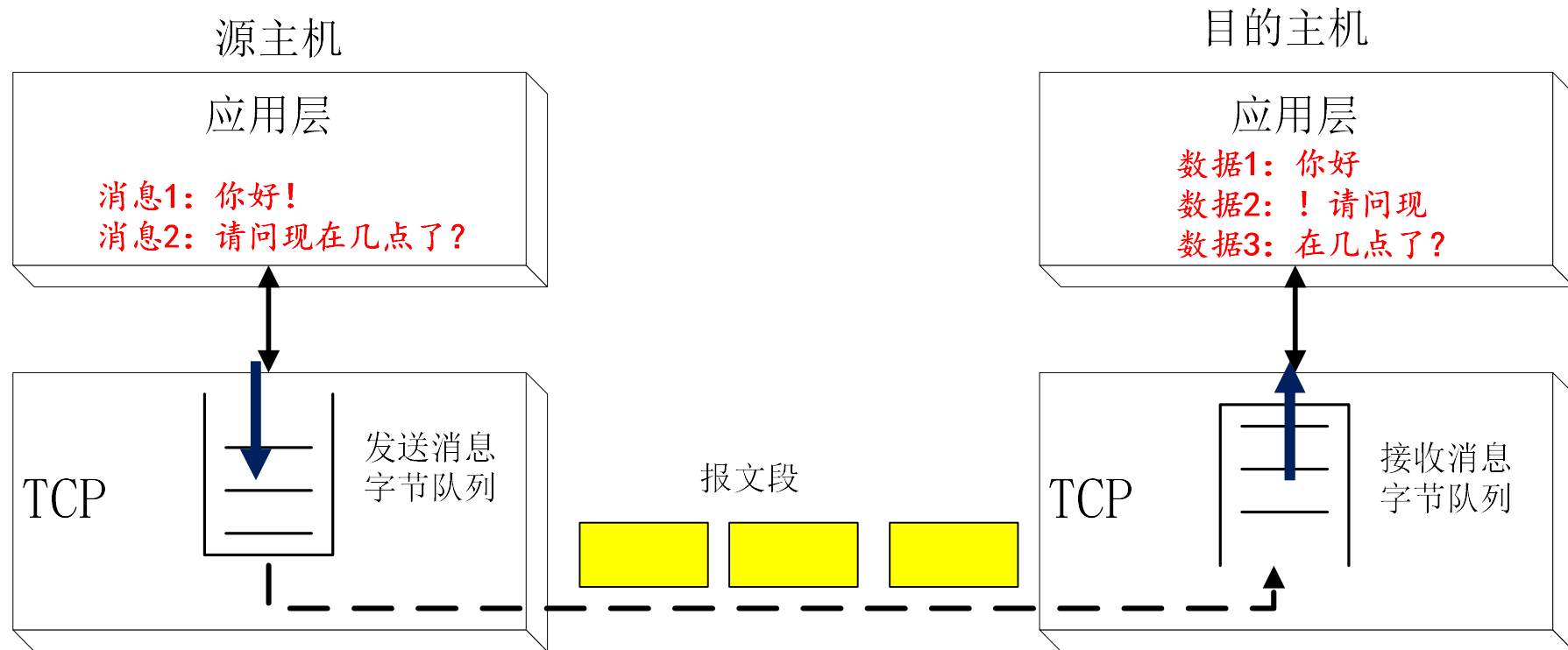
- TCP特性
- TCP段格式
- 连接管理
- 流量控制
- 差错控制
- TCP定时器管理
- 拥塞控制

TCP的特征

- 传输控制协议TCP是为了在不可靠的互联网络上提供面向连接的、可靠的、端到端服务而设计的协议
- 可靠交付 (ARQ)
 - ◆ 以确认来表示数据已交付
 - ◆ 使用检查和来发现数据被破坏
 - ◆ 超时后重传出错的数据
 - ◆ 用序号来检测数据的丢失或乱序
 - ◆ 基于窗口的流量控制防止淹没接收方
- 使用拥塞控制来共享网络容量
- 面向字节流：报文长度任意，没有边界
- 不能保留消息边界

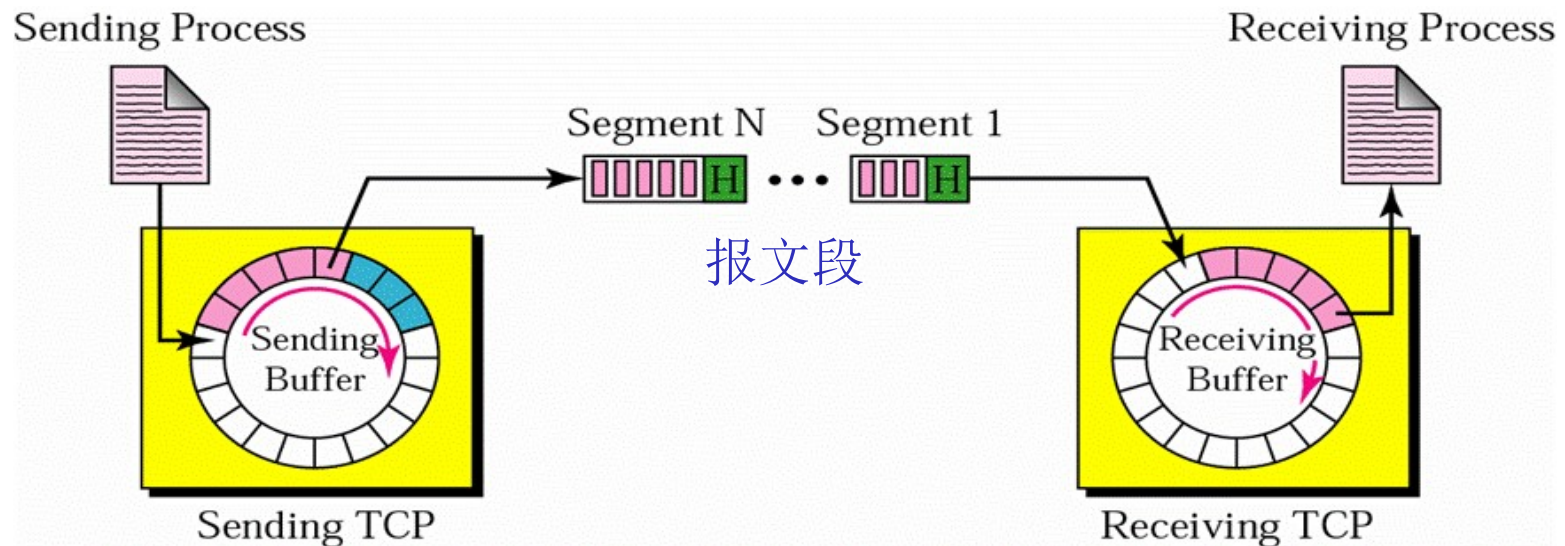
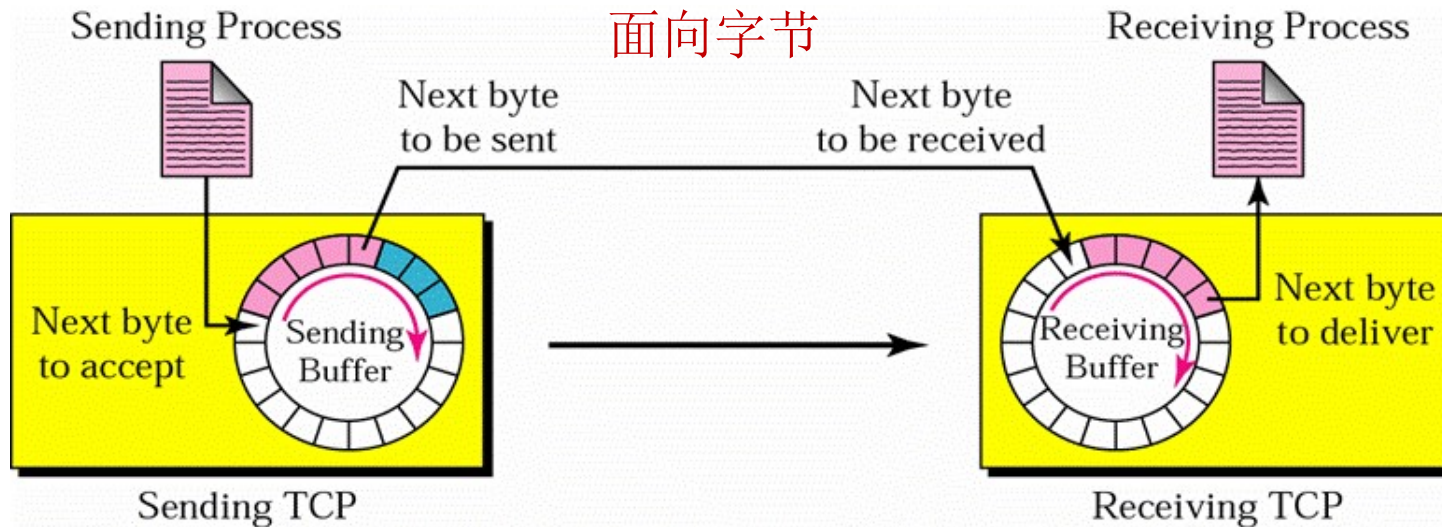


TCP不能保证上层应用消息的边界



- ◆ TCP报文段内携带多少字节数据由TCP协议决定，与上层消息长度无关
- ◆ TCP仅保证交给上层的数据正确、按序

TCP: 缓存和段



TCP报文段

- **TCP 段**=(20或更多字节)**TCP头**+(n字节)**数据**
- 没有任何数据的段是合法的，通常用于**ACK**和控制消息
- 最大段长度**MSS (Maximum Segment Size)**: 段中的最大数据长度
 - ◆ 受**IP**数据报长度的限制（**65515**字节）
 - ◆ 受各数据链路的最大传输单元**MTU**的限制（**MTU**通常是**1500**字节，以太网的负载长度），因此路由器可以**TCP**段进行分片
- 段**序号**和**确认号**均**面向字节**
 - ◆ **TCP**段中的序号是该段第一个字节的序号
 - ◆ **ACK**段中的确认序号是下一个期待的字节的序号
- 滑动窗口大小动态可变

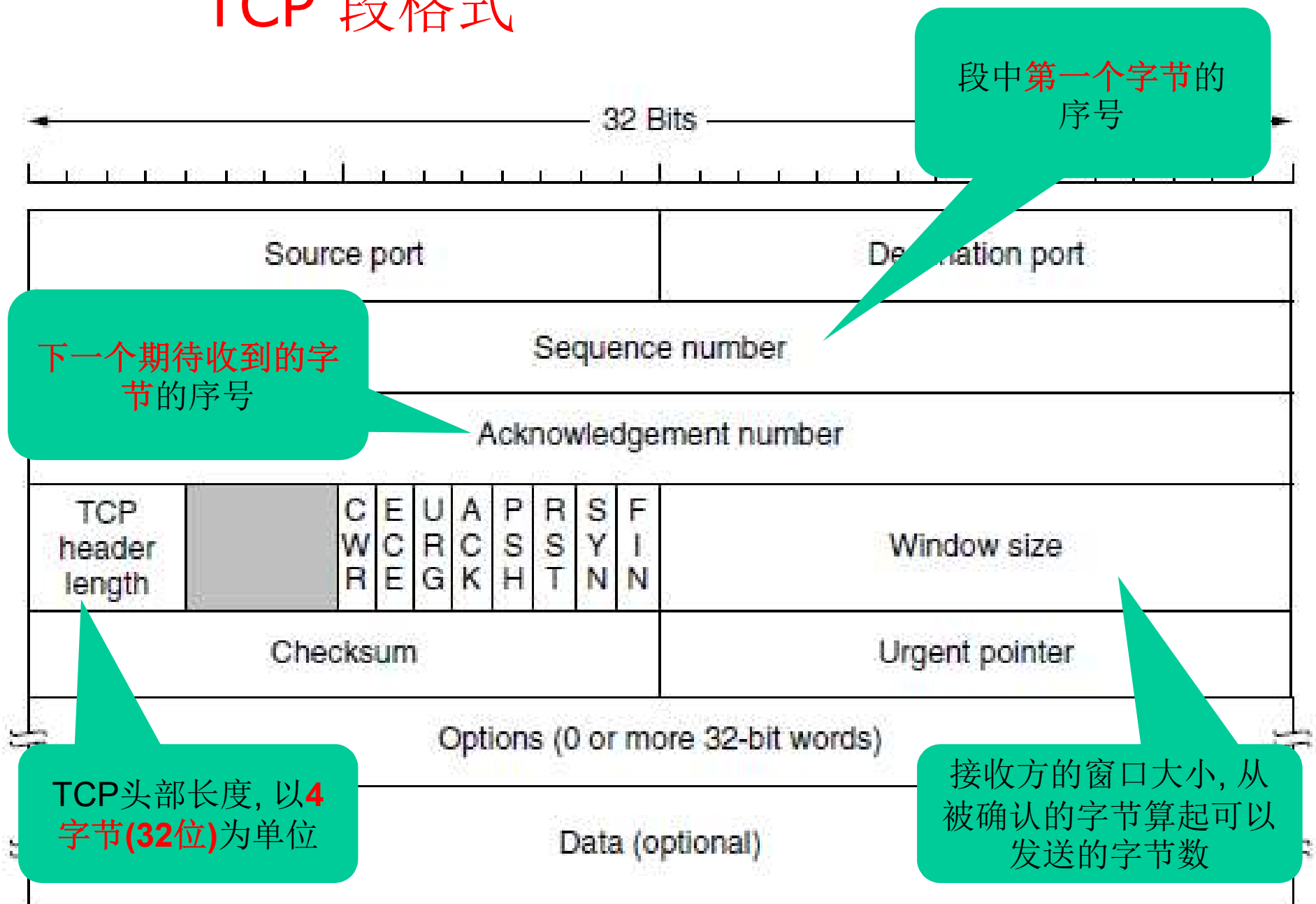
TCP支持的常用端口

Port	Protocol	Use
20, 21	FTP	File transfer
22	SSH	Remote login, replacement for Telnet
25	SMTP	Email
80	HTTP	World Wide Web
110	POP-3	Remote email access
143	IMAP	Remote email access
443	HTTPS	Secure Web (HTTP over SSL/TLS)
543	RTSP	Media player control
631	IPP	Printer sharing
23	Telnet	Remote login

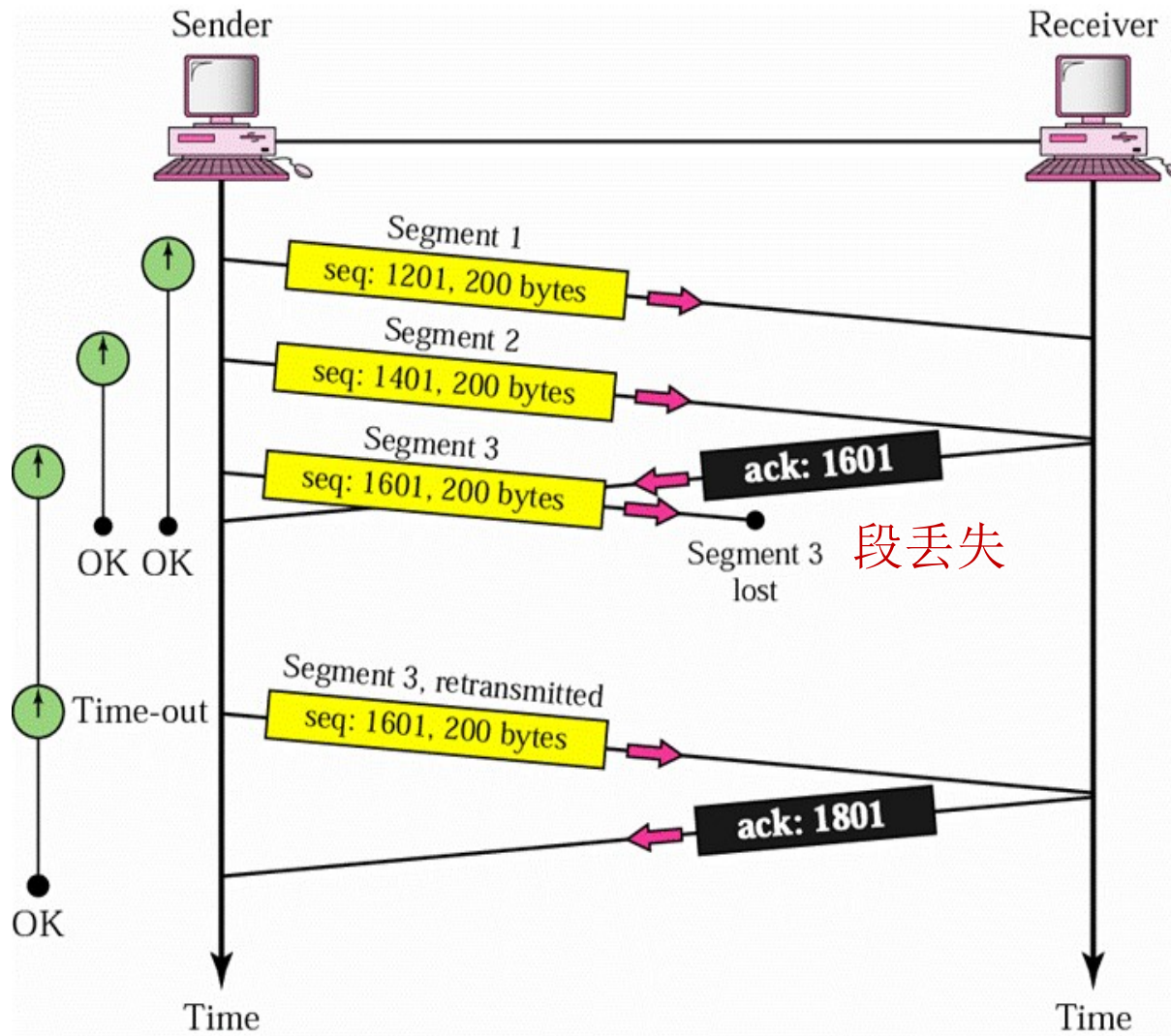
TCP: 传输控制协议

- TCP特性
- TCP段格式
- 连接管理
- 流量控制
- 差错控制
- TCP定时器管理
- 拥塞控制

TCP 段格式



举例：序号、确认号和窗口大小



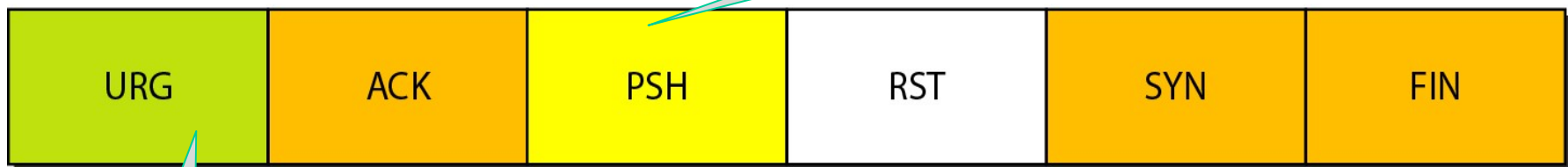
[Forouzan]

标志域

URG: Urgent pointer is valid
ACK: Acknowledgment is valid
PSH: Request for push

RST: Reset the connection
SYN: Synchronize sequence numbers
FIN: Terminate the connection

不得延误传输或交付,
eg. 交互式游戏

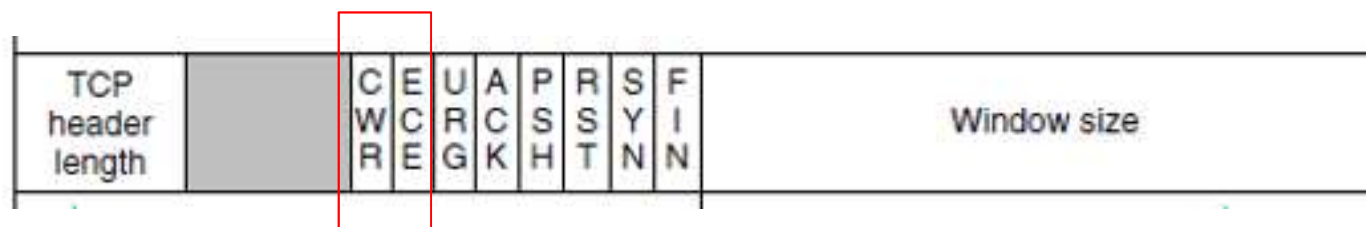
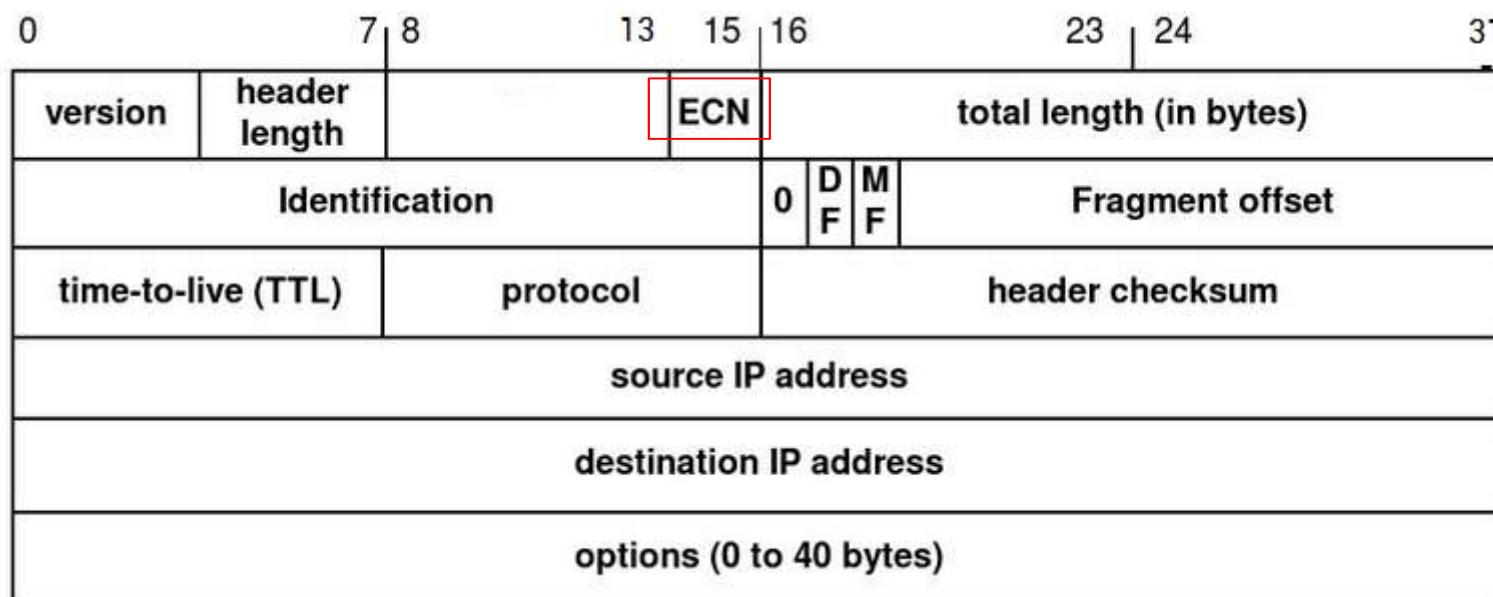


尽快处理
并交付应
用层，如
中断信号

<i>Flag</i>	<i>Description</i>
URG	The value of the urgent pointer field is valid.
ACK	The value of the acknowledgment field is valid.
PSH	Push the data.
RST	Reset the connection.
SYN	Synchronize sequence numbers during connection.
FIN	Terminate the connection.

注：仅当**ACK=1**时，“窗口大小”才有效

ECN 和ECE/CWR



举例：ECE和CWR

■ RFC 3168

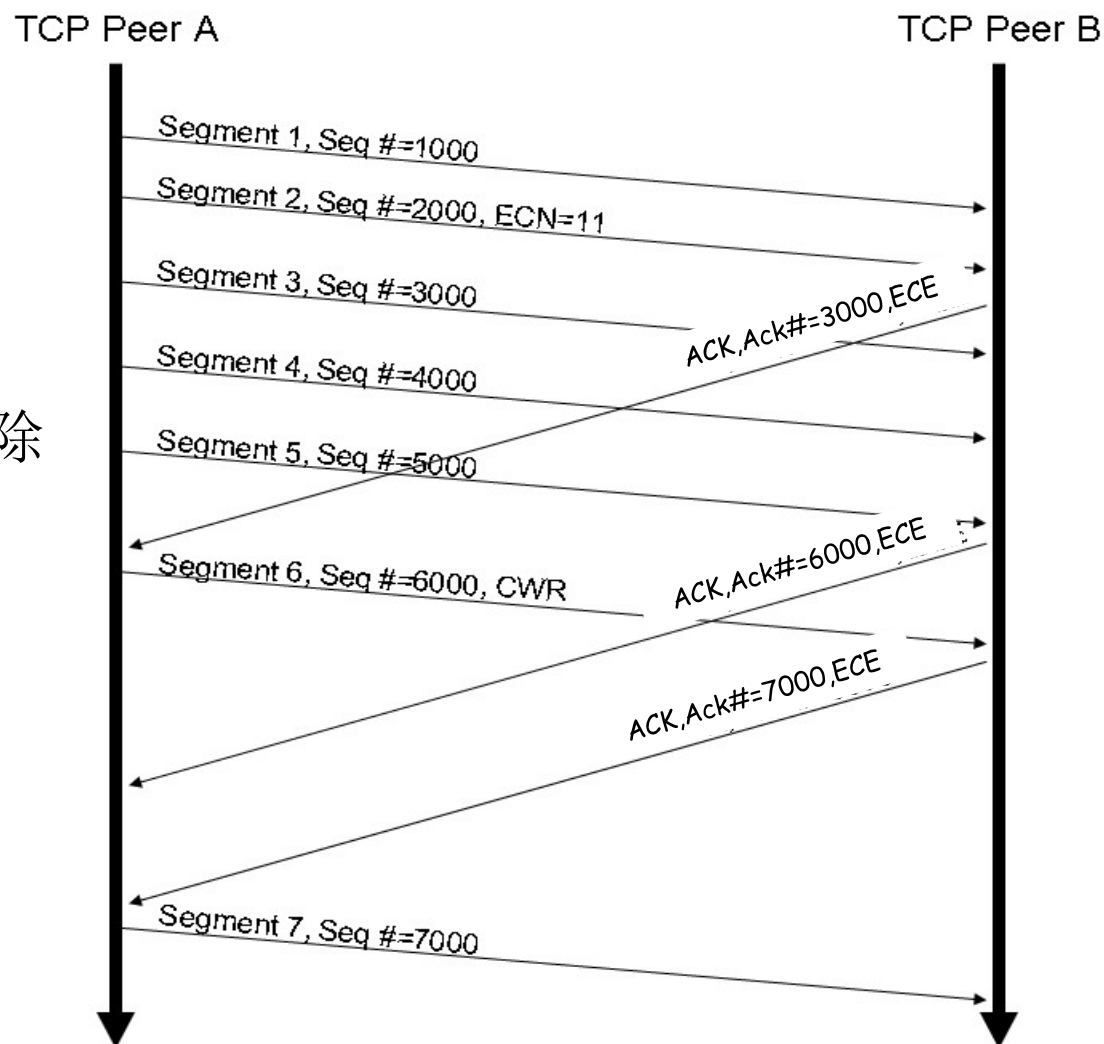
ECN由路由器设置；

ECE由接收方TCP实体设置

CWR由发送方TCP设置

收到CWR之后，接收主机清除ECE

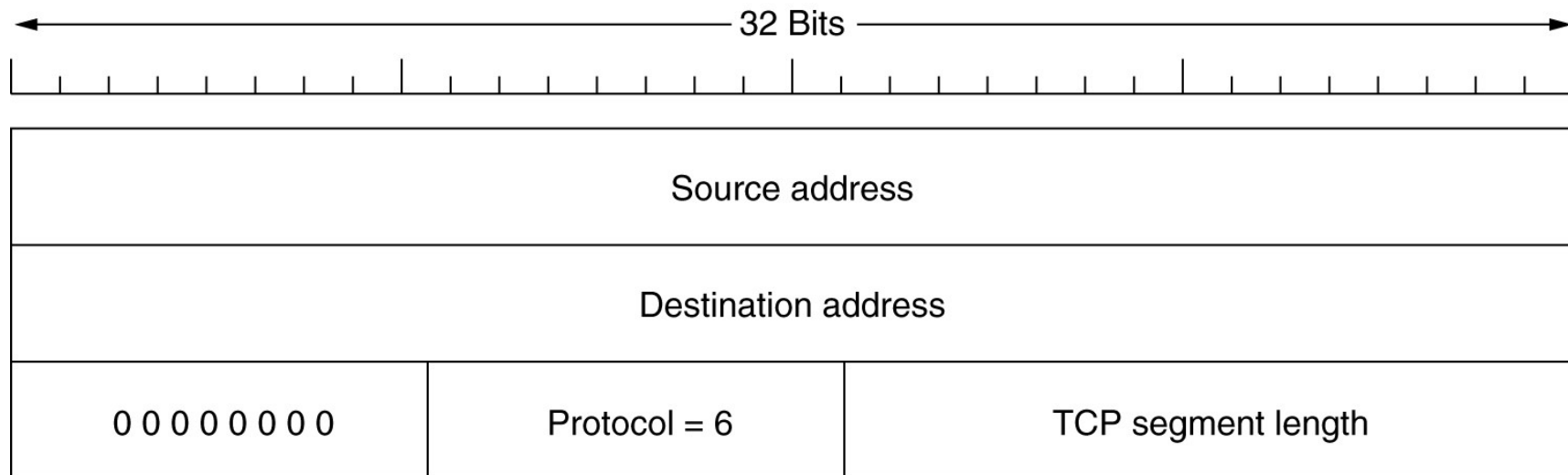
显式拥塞通知



TCP校验和

■ TCP校验和

- ◆ 提供额外的可靠性
- ◆ 校验范围：TCP段头、数据和伪报头三部分
- ◆ 伪报头:帮助检测错误的数据包
- ◆ TCP中的校验和是必须支持的



包含在TCP校验和中的伪报头

TCP头标中的可选项

■ MSS(最大段大小)

- ◆ 允许主机指定其愿意接受的最大段长(即有效载荷长度)
- ◆ 在连接建立阶段进行协商
- ◆ 如果未使用该选项，则主机默认可接受的最大段长是536字节
- ◆ 两个方向上的最大段长MSS可以不同

■ Window scal(窗口尺度)

- ◆ 允许发送方和接收方在连接建立阶段协商窗口尺度因子

■ Timestamp(时间戳)

- ◆ 用于计算往返时间样值

■ SACK (选择确认)

- ◆ 允许接收方告诉发送方已经接收到的段的序号范围。

TCP选项: SACK

■ SACK (选择确认)

- ◆ 是对确认序号的补充，可用在一个数据包已经丢失但后续（或重复的）数据到达的情况
- ◆ 使用**SACK**，发送方能够明确地知道接收端收到了哪些数据，并据此确定应该重传什么数据
- ◆ **SACK**内容由**RFC2108**和**RFC 2883**定义

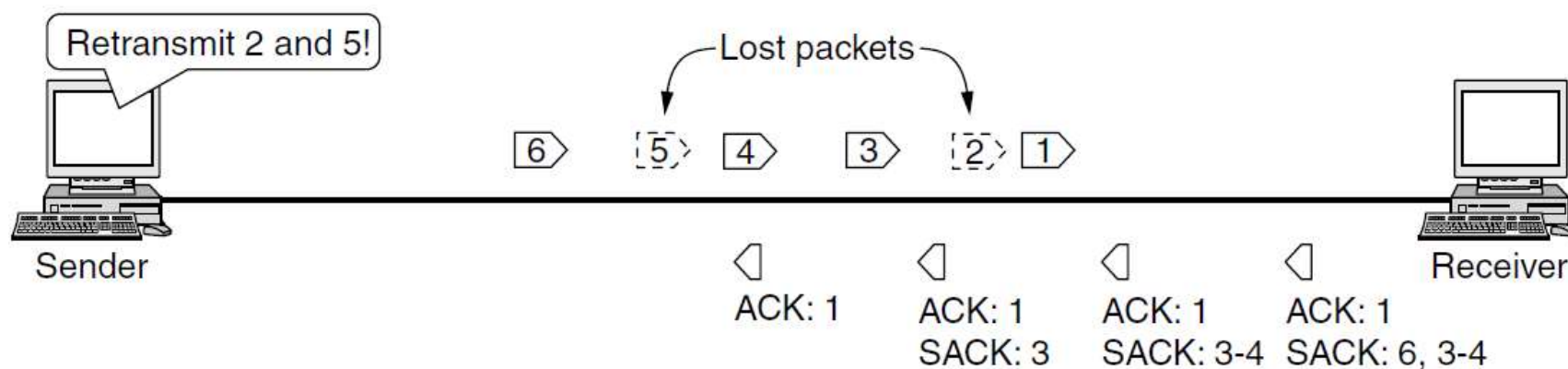


Figure 6-48. Selective acknowledgements

Review

■ UDP

- ◆ 无连接、快捷、不可靠
- ◆ 通过端口号寻址到进程和复用
- ◆ 可以保留上层消息的边界

■ TCP

- ◆ 面向连接、可靠的字节流传输
- ◆ 序号采用字节编号
- ◆ 不能保留上层消息的边界
- ◆ 显式拥塞通知：和路由器配合
- ◆ **MSS**：最大数据长度；**SACK**选项：支持选择重传

■ 校验和

- ◆ IP、UDP和TCP使用相同的校验算法
- ◆ IP：只对包头校验
- ◆ UDP：可选，对**伪报头**+UDP数据报（报头+数据）进行校验
- ◆ TCP：必选，对**伪报头**+整个报文段（TCP头+数据）进行校验

TCP: 传输控制协议

- TCP特性
- TCP段格式
- 连接管理
- 流量控制
- 差错控制
- TCP定时器管理
- 拥塞控制

TCP连接建立

■ 客户端

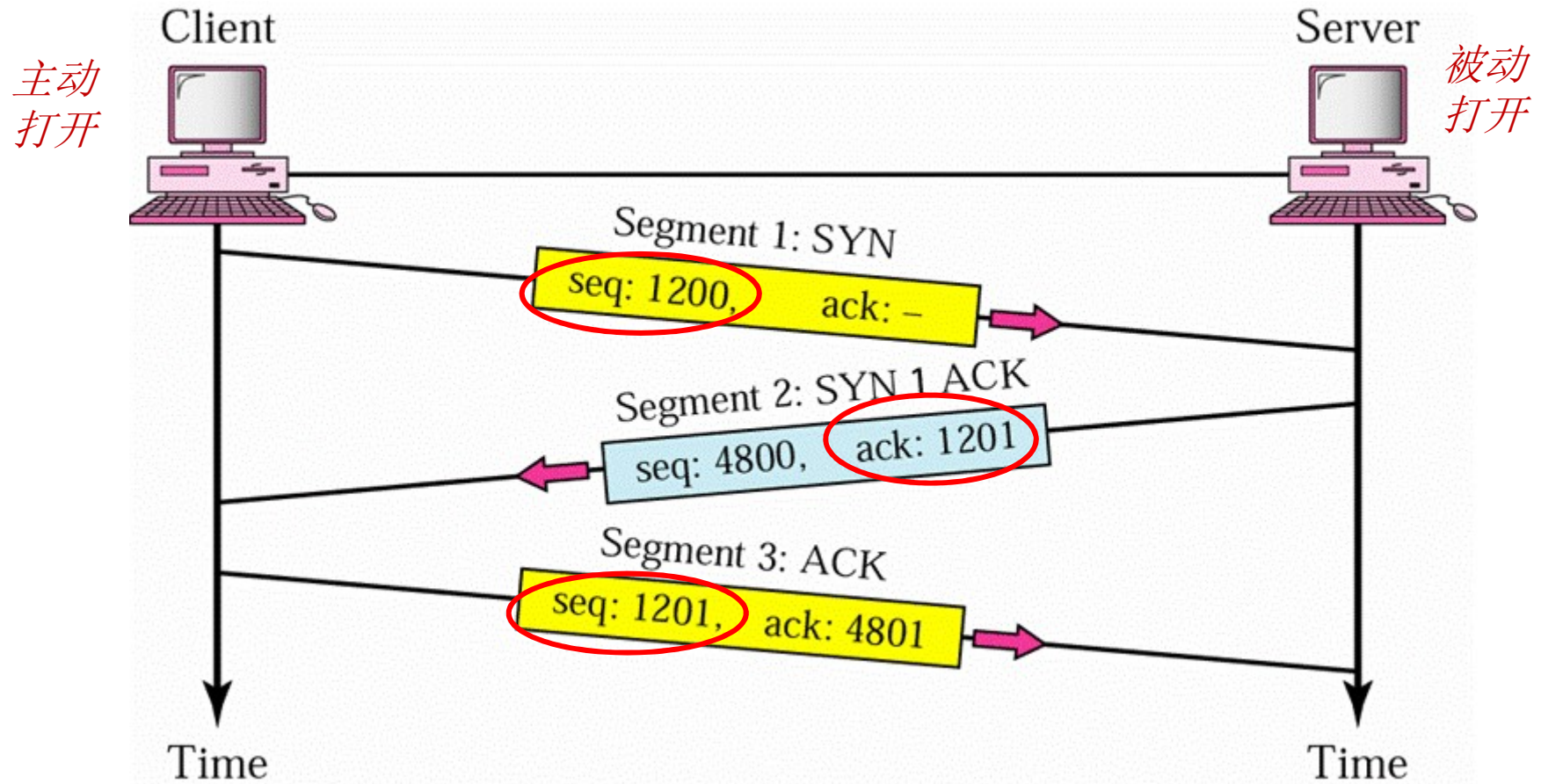
- ◆ 执行CONNECT原语, 说明其希望连接的IP地址和端口、愿意接收的最大TCP段长以及其他一些可选参数,
- ◆ TCP实体发送一个SYN=1和ACK=0的TCP段, 等待服务器端的响应

■ 服务器端

- ◆ 当客户端的SYN段到达后, TCP实体检查是否有一个进程已经在目标端口字段指定的端口上执行了LISTEN
 - 如果没有, 则发送一个设置了RST (RST=1) 的应答报文, 拒绝该连接
 - 如果某个进程正在该端口监听, 则将入境的TCP段交给该进程处理,

连接建立—三次握手

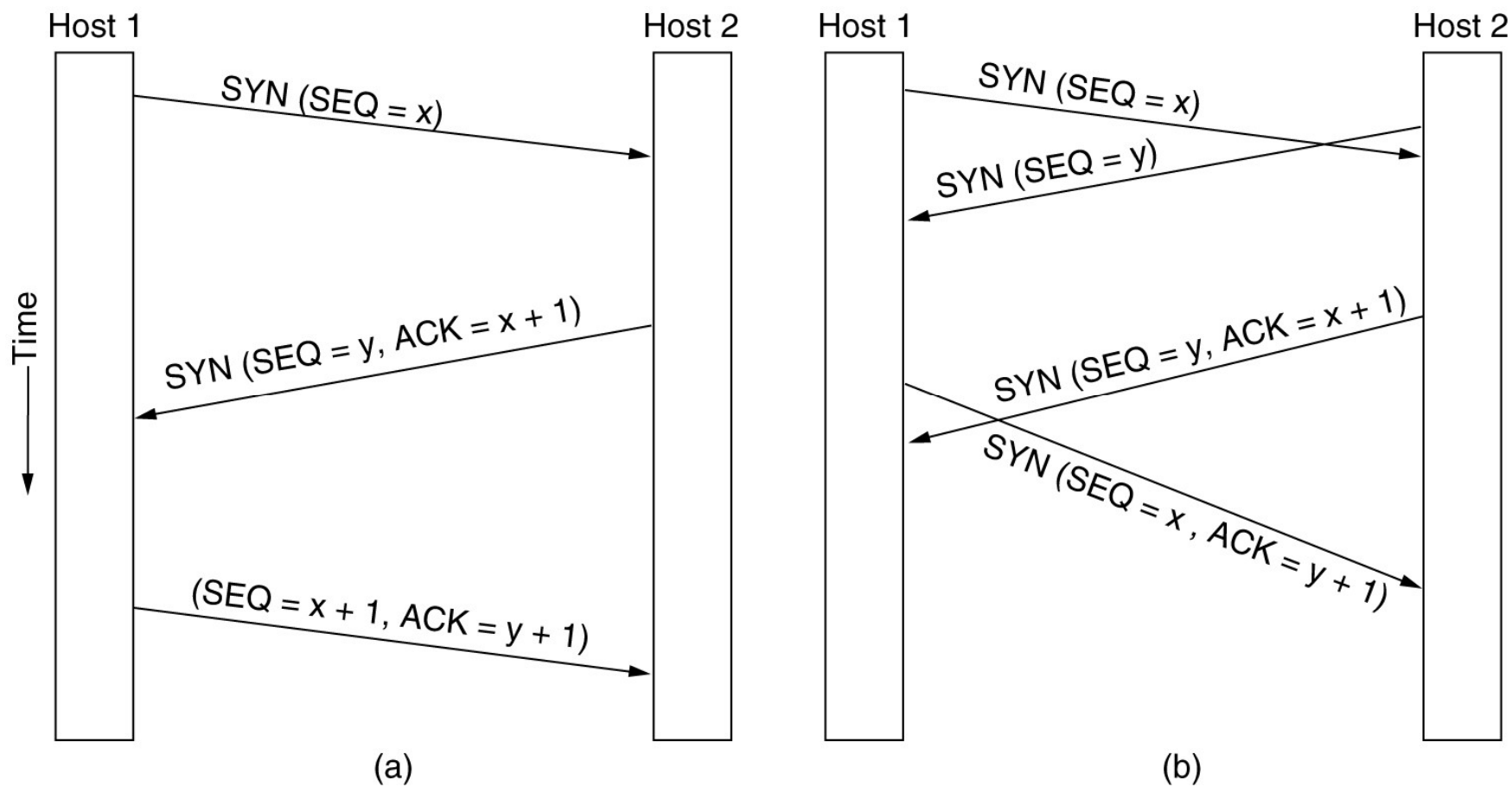
双方同意每一方的初始序号，解决了延迟的重复段的问题



一个**SYN**段不携带数据，但是消耗**1**个序号

一个 **SYN+ACK** 段不携带数据,但是消耗**1**个序号

TCP连接建立



(a) 正常情况下的TCP连接建立

(b) 两端同时建立连接的情况

初始序号

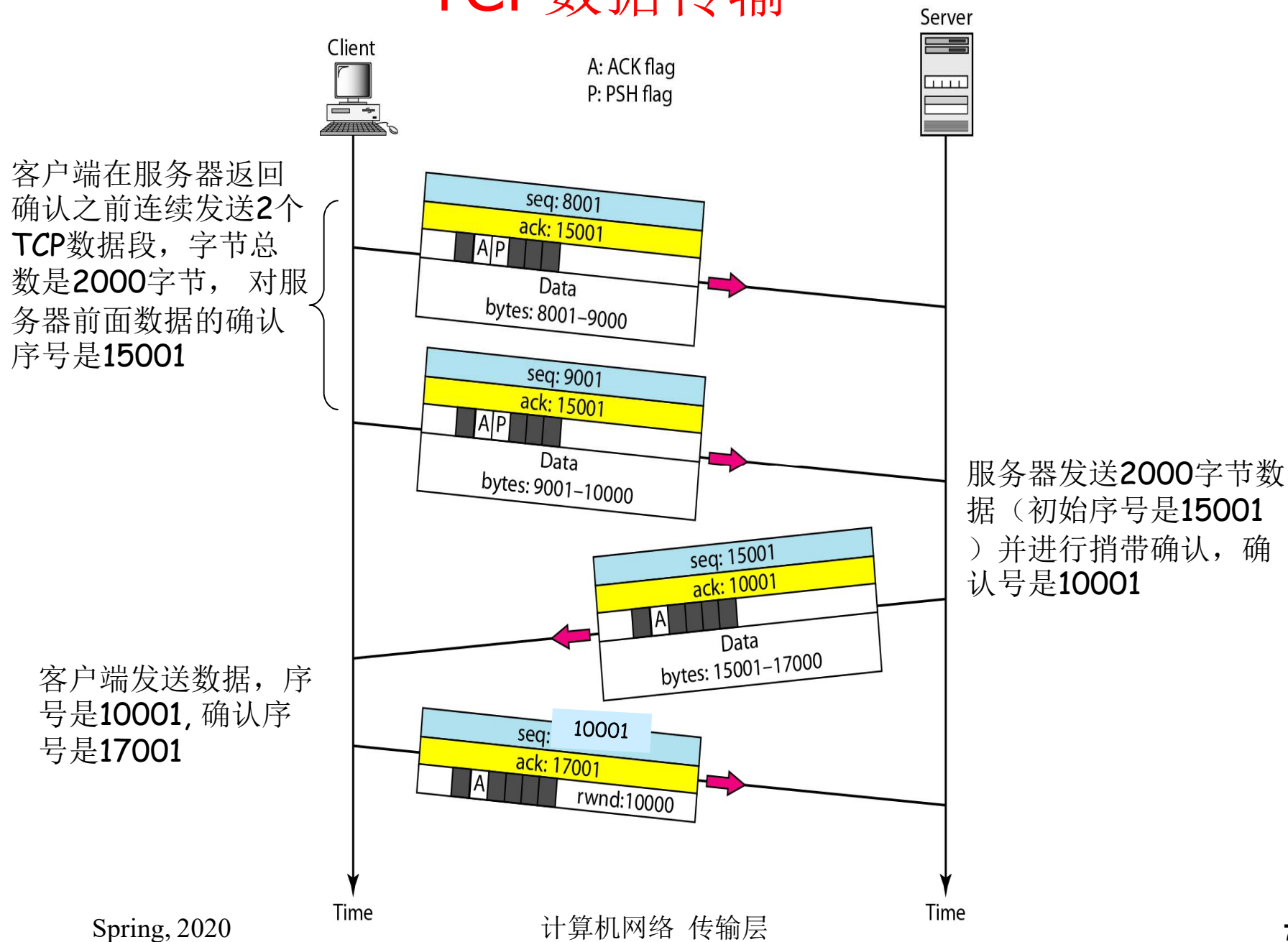
■ 初始序号

- ◆ 采用了一个基于时钟的方案, 时钟每4 μsec 滴答一次

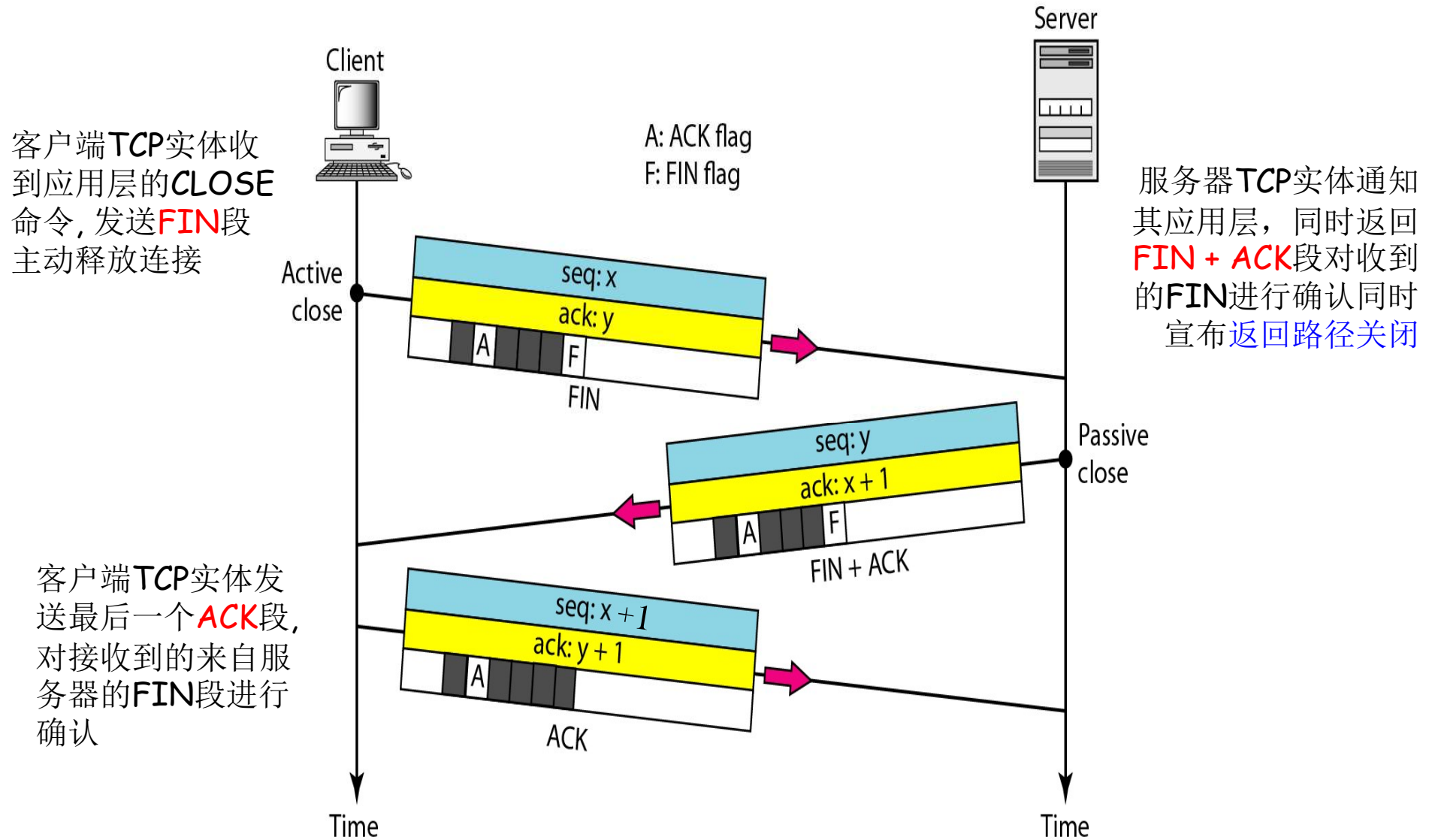
■ 安全性

- ◆ 三次握手方式存在漏洞, 当监听进程接受一个连接请求, 并以**SYN**段作为响应时, 它必须记住该**SYN**段的序号--- **SYN洪泛攻击**
- ◆ **SYN Cookie**—一种抗**SYN**洪泛攻击的方法

TCP数据传输



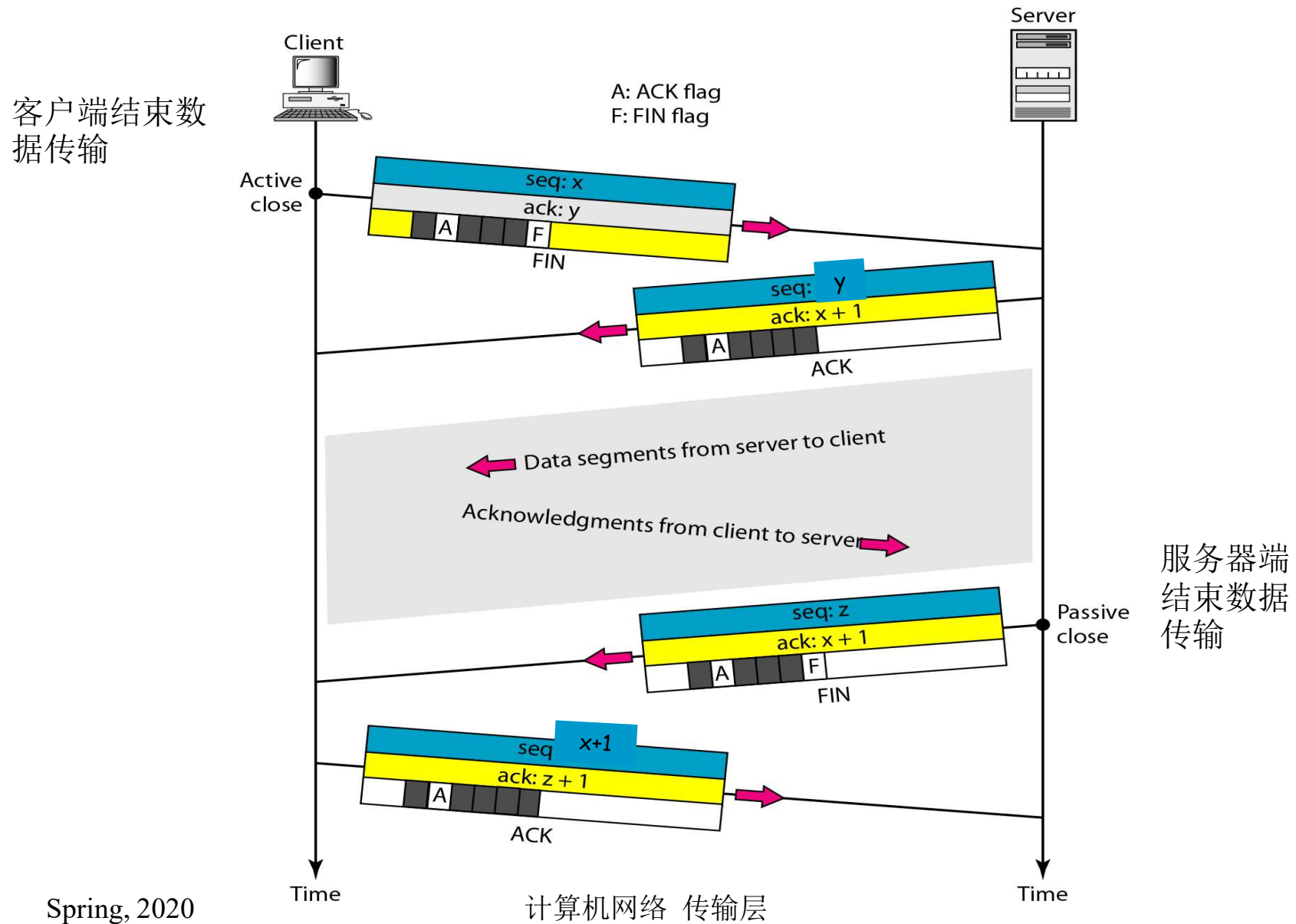
连接释放:三次握手



FIN段不携带数据, 但是消耗1个序号

FIN+ACK 段不携带数据, 但是消耗一个序号.

4步连接释放:半关闭连接



TCP连接建立和连接释放示例

■ 连接建立：三次握手

Source	Destination	Protocol	Length	Info
192.168.0.104	203.119.206.248	TCP	66	50531 → 80 [SYN] Seq=0 Win=17520 Len=0 MSS=1460 WS=256 SACK_PERM=1
203.119.206.248	192.168.0.104	TCP	66	80 → 50531 [SYN, ACK] Seq=0 Ack=1 Win=7300 Len=0 MSS=1412 SACK_PERM=1 WS=512
192.168.0.104	203.119.206.248	TCP	54	50531 → 80 [ACK] Seq=1 Ack=1 Win=17408 Len=0
192.168.0.104	203.119.206.248	TCP	181	50531 → 80 [PSH, ACK] Seq=1 Ack=1 Win=17408 Len=127 [TCP segment of a reassemb

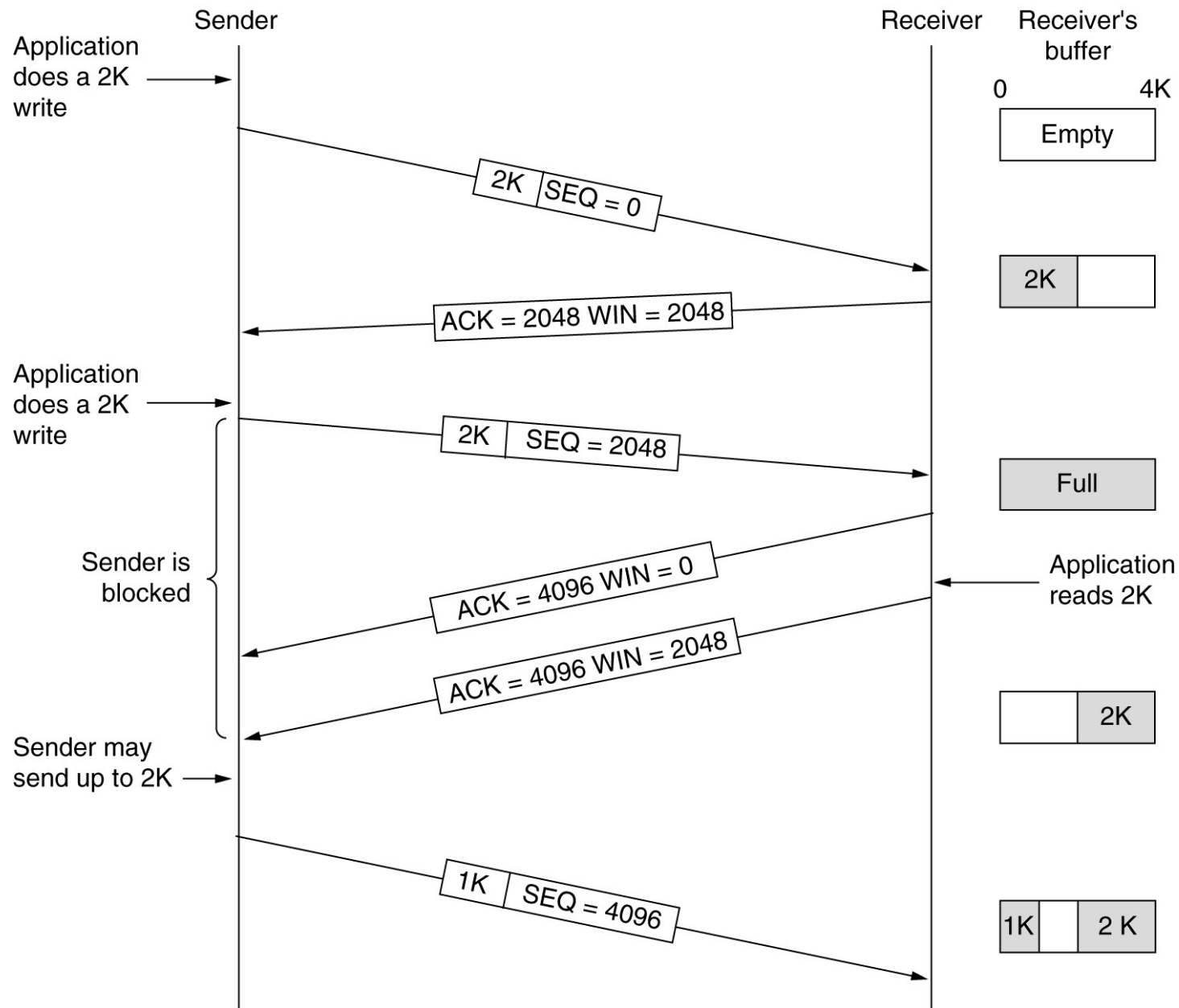
■ 连接释放：四步释放

203.119.206.248	192.168.0.104	TCP	54	80 → 50531 [FIN, ACK] Seq=643 Ack=4211 Win=15360 Len=0
192.168.0.104	203.119.206.248	TCP	54	50531 → 80 [ACK] Seq=4211 Ack=644 Win=16640 Len=0
192.168.0.104	203.119.206.248	TCP	54	50531 → 80 [FIN, ACK] Seq=4211 Ack=644 Win=16640 Len=0
203.119.206.248	192.168.0.104	TCP	54	80 → 50531 [ACK] Seq=644 Ack=4212 Win=15360 Len=0

TCP: 传输控制协议

- TCP特性
- TCP段格式
- 连接管理
- 流量控制
- 差错控制
- TCP定时器管理
- 拥塞控制

TCP中的窗口管理



流量控制：滑动窗口

- 将正确接收段的**确认**和**接收端缓冲区的分配**分离开来
- 与数据链路层流量控制的区别：
 - **TCP**的滑动窗口是面向字节的, 而数据链路层的滑动窗口是面向帧的
 - **TCP**的滑动窗口大小是变化的（确认号+ 窗口尺寸）, 但是数据链路层的窗口大小是固定的
- 当接收方宣布窗口窗口尺寸为**0**时
 - 发送方不能再正常发送段
 - 紧急数据仍可发送
 - 窗口探针：发送方可以发送**1**字节的段以便刺激接收方发送**ACK**和更新窗口尺寸防止死锁

TCP滑动窗口的特征

- 发送端不一定接收到应用进程传递过来的数据就马上将数据传送出去.
- 窗口大小动态变化
 - ◆ 发送方窗口大小完全由接收方窗口值控制. 但由于网络可能拥塞, 因此实际值可能更小
- 接收方可以在其愿意的时候发送确认消息

Nagle算法

■ 问题：小数据包问题 (发送小包网络效率低)

◆ 开销: TCP head + IP head=20+20 bytes

◆ 在TELNET使用时，1字节的数据需要40字节的开销

■ 改进思路：尽量发比较大的段(避免发小包)

◆ 发送方：Nagle算法

➤ 发送第一块数据并缓冲剩余的数据

➤ 不再发送数据除非满足下面条件之一：

▣ 发送出去的数据段被确认

▣ 缓冲数据填满半个窗口或达到最大段(MSS)长度

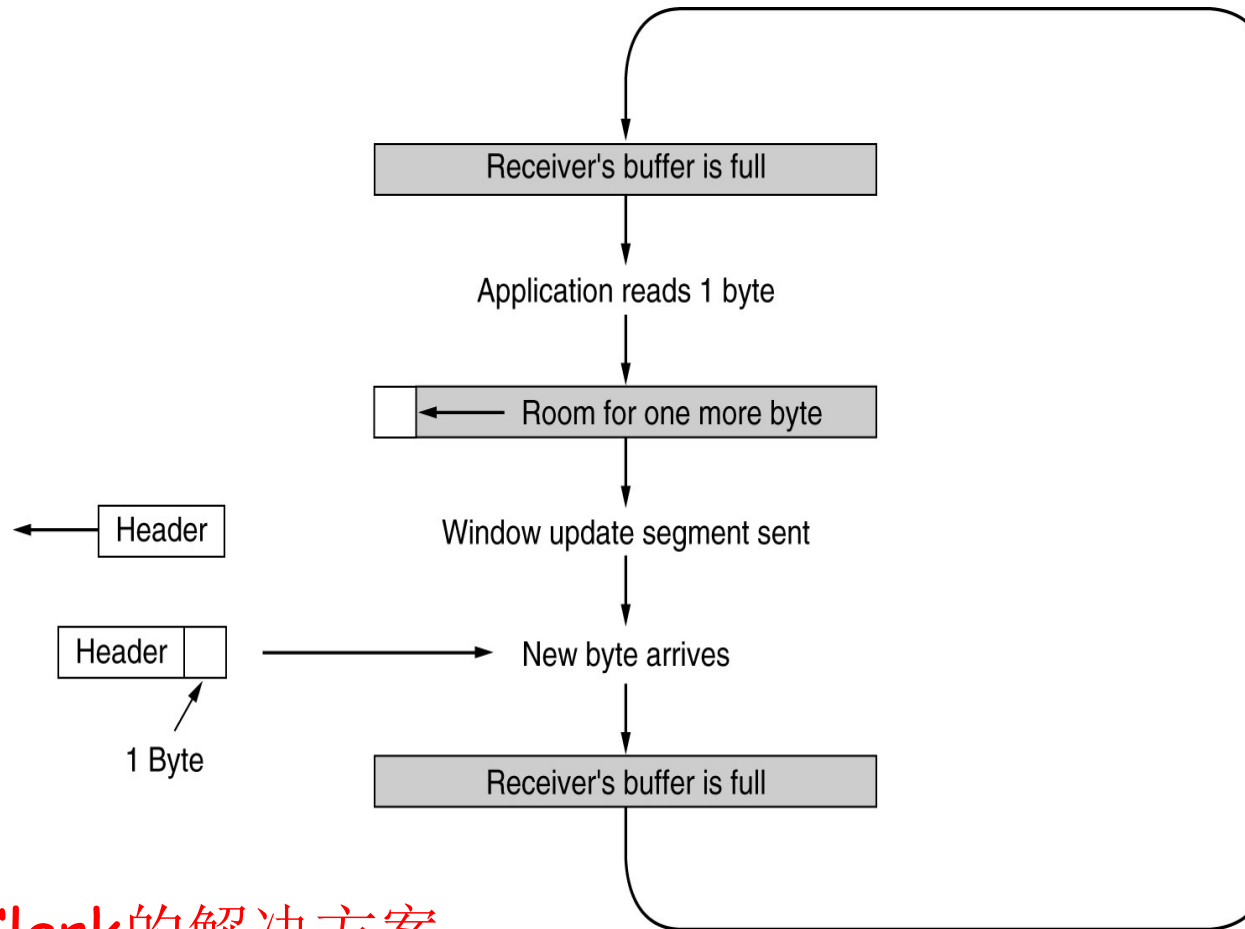
◆ 接收方：延迟确认和窗口更新，降低接收方注入网络的负载

■ TCP选项

◆ TCP_NODELAY: 禁用Nagle算法&使用PUSH位

◆ Linux TCP_CORK: 禁用Nagle算法，等待200ms来积累数据以便发送

问题2：低能(愚钝)窗口综合症



■ Clark的解决方案

只有当接收方能够处理其宣告的最大数据段或者其缓冲区一半为空（两者之间取较小的值），才能发送窗口更新段

TCP的传输效率

- TCP发送小包会降低网络的传输效率
- 场景1：发送应用进程1次发送一个字节数据
- 解决方案：Nagle's Algorithm发送端避免发送小包
 - ◆ 方法：TCP连接上最多只有一个未被确认的小包，TCP发送方缓存后续的数据直到收到TCP确认或者积累了足够的数据(如：MSS段)
- 场景2：接收应用进程1次接收处理一个字节数据(低能/糊涂窗口综合症)
- 解决方案：Clark's Algorithm禁止接收端TCP发送只有一个字节的窗口更新段
 - ◆ 方法：接收端TCP有数据到达就可以发送ACK，但如果缓存不够大，一直宣布window size=0
- 两种方法配合使用

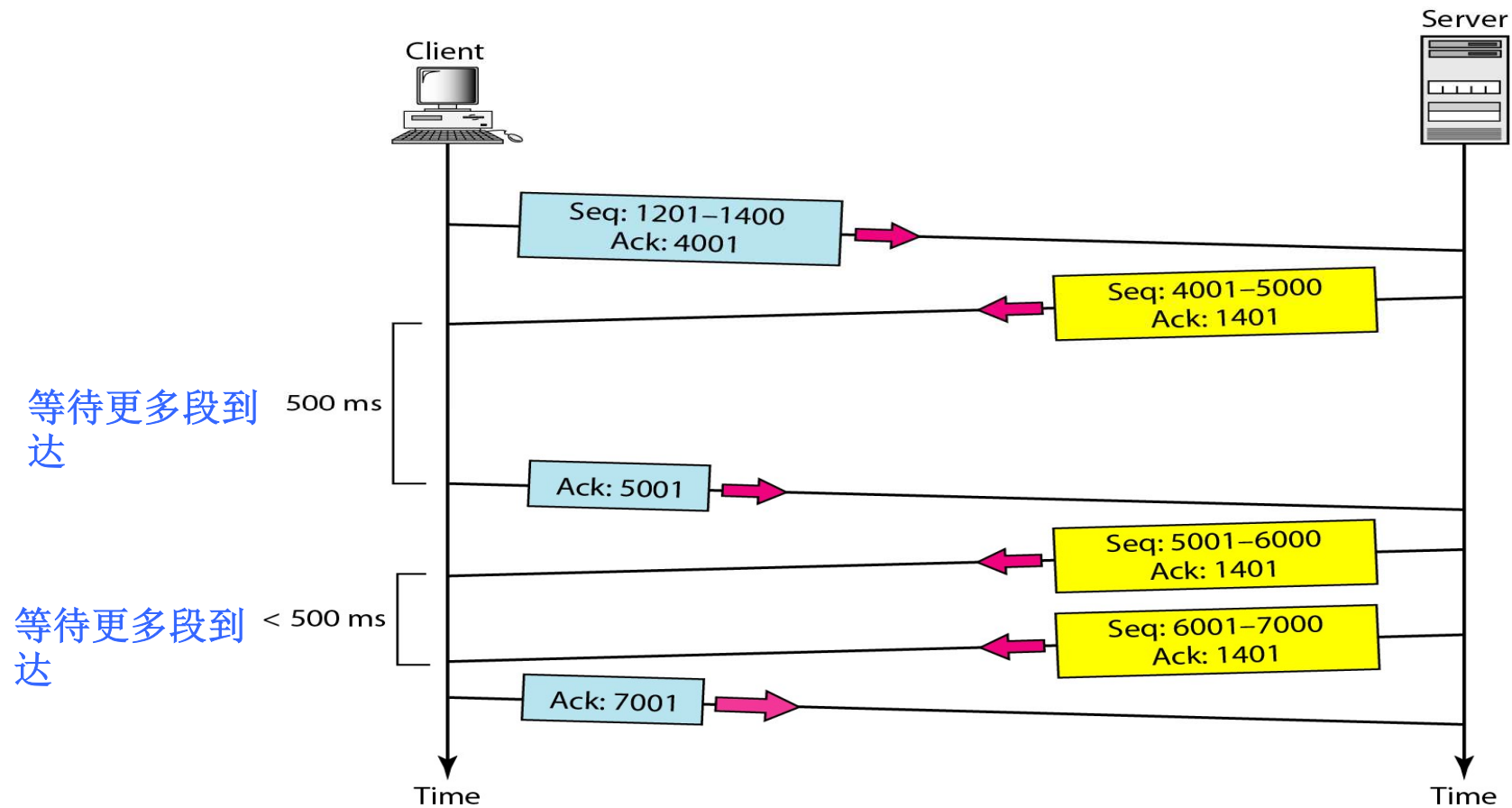
TCP: 传输控制协议

- TCP特性
- TCP段格式
- 连接管理
- 流量控制
- 差错控制
- TCP定时器管理
- 拥塞控制

TCP差错控制

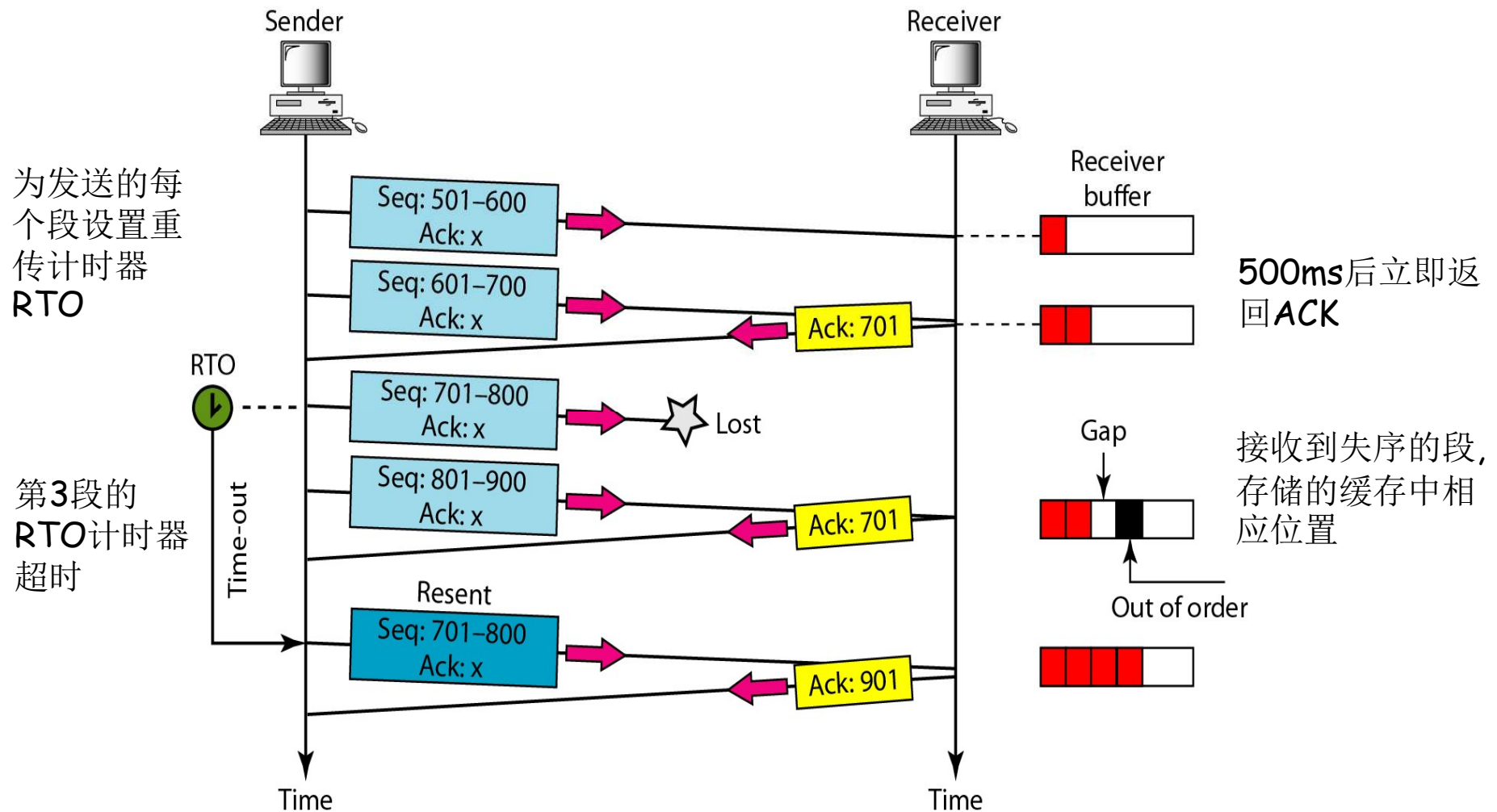
- TCP 提供可靠的传输
- 差错控制包括检测差错段、丢失段、失序段和重复段的机制.
- 使用ARQ进行错误检测和纠正
 - ◆ 校验和: 检测差错段并丢弃.
 - ◆ 确认 (ACK): 对正确接收到的段进行确认.
 - ◆ 超时: 为段的重传设置计时器, 称为RTO(重传计时器)
- 差错控制机制的核心是重传.
- 确认消息(ACK段)无相关计时器也不需要重传

正常操作

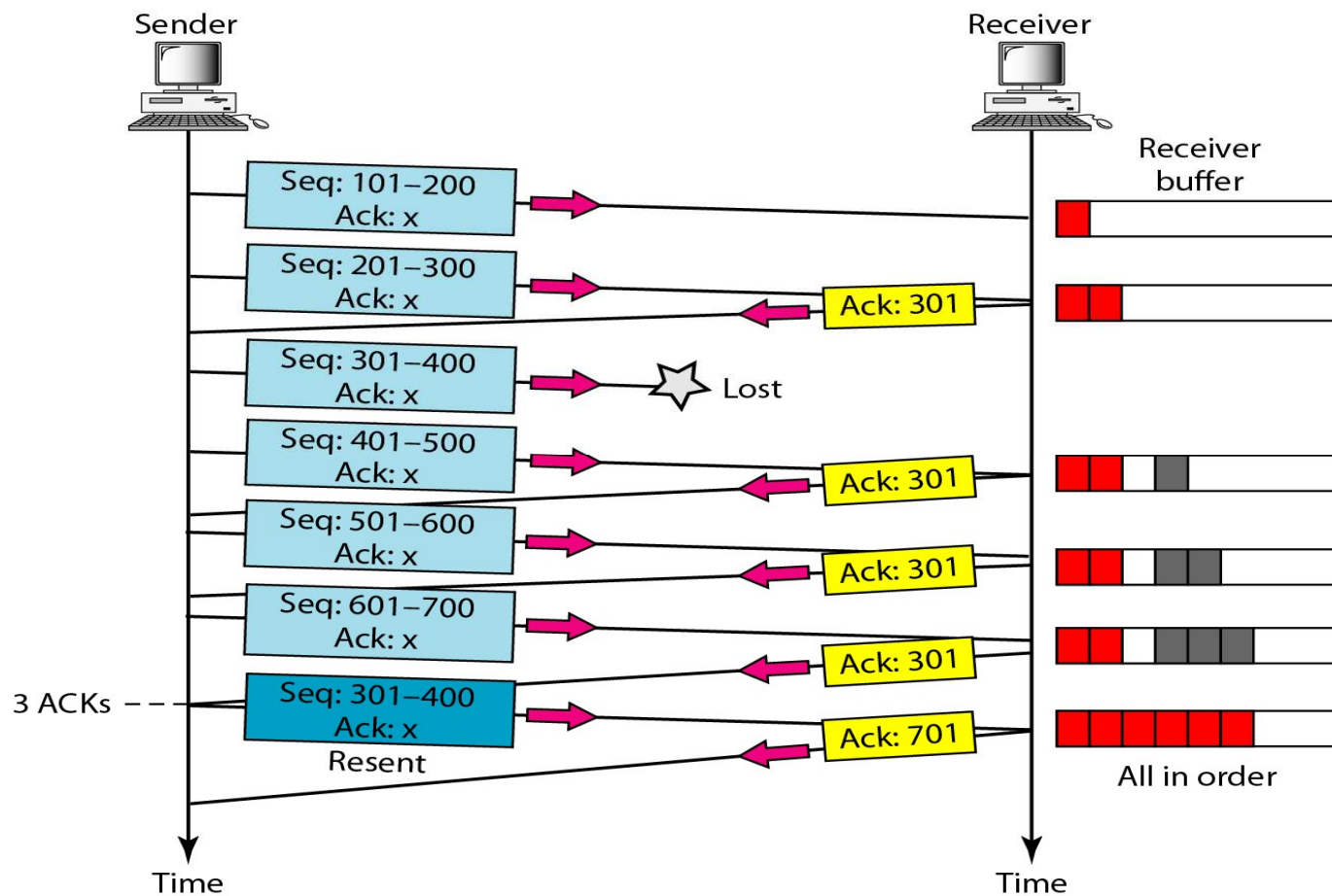


接收方在发送确认之前等待**500 ms**, 以便能够等待更多段的到达, 进行累积确认(**累积ACK**). (例如 跳过了**ACK: 6001**).

报文段丢失

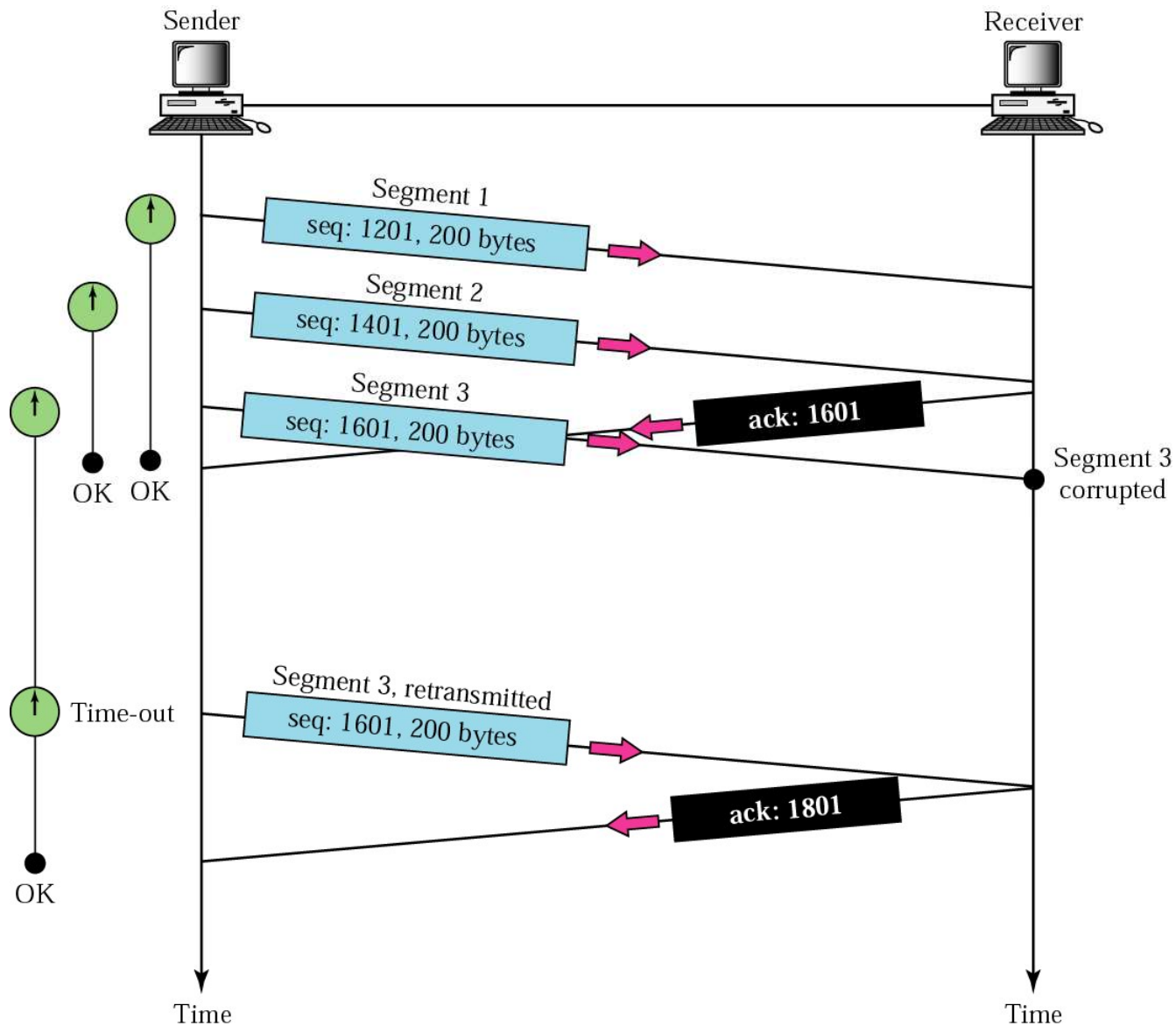


快速重传

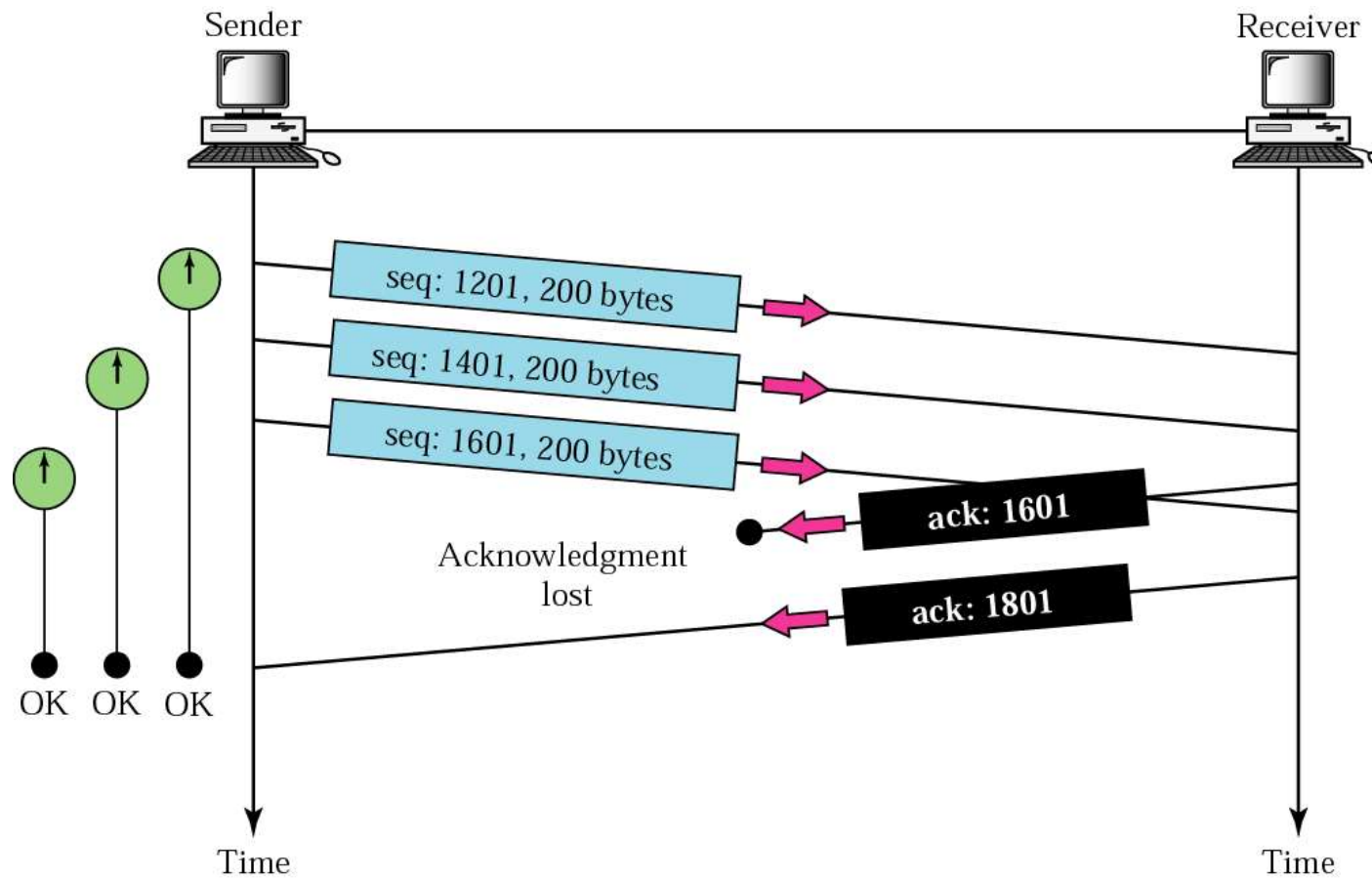


快速重传: 当收到三个重复ACK(相同确认号)时, 发送方立即重传丢失的段而不需要等待重传计时器RTO超时.

差错段



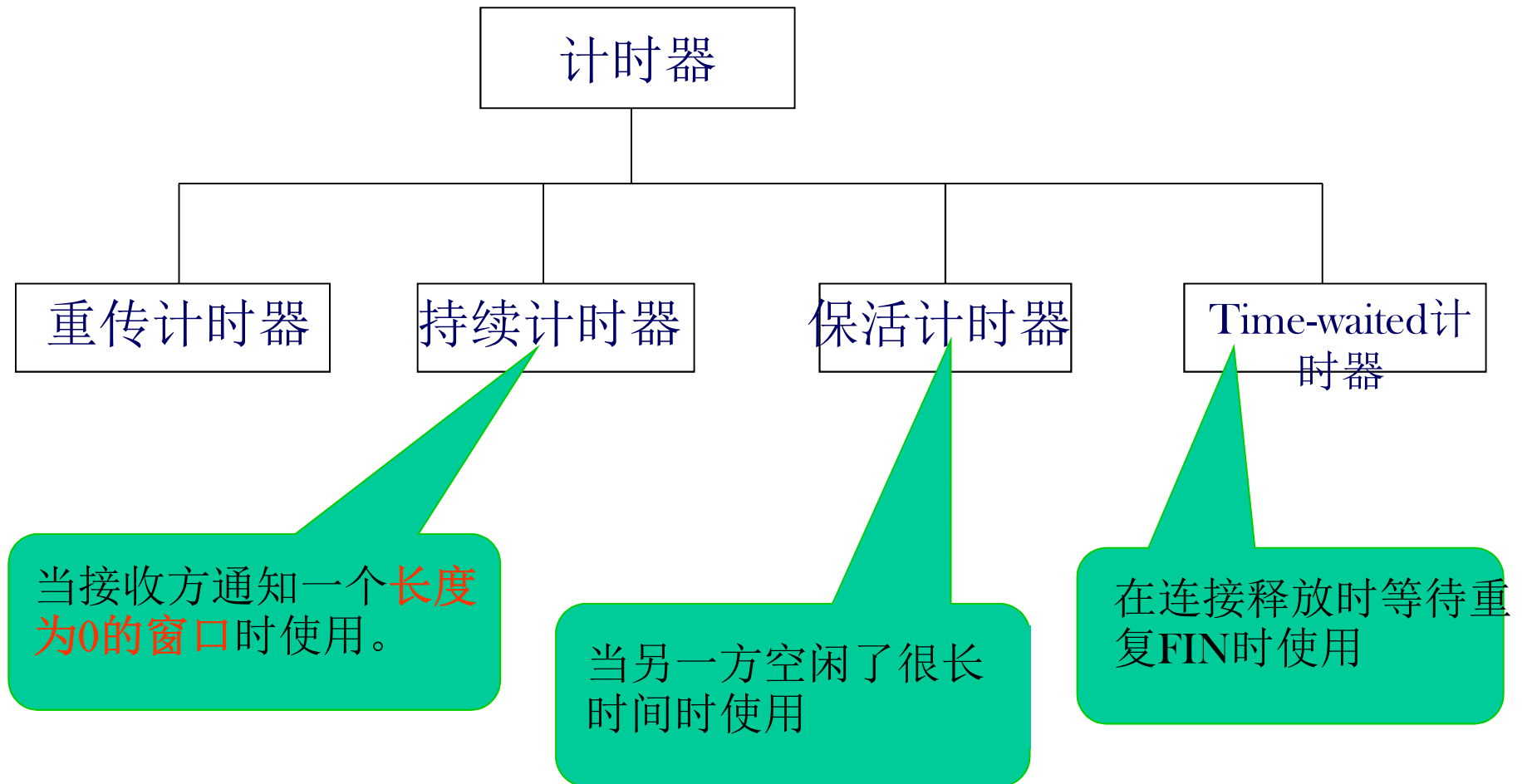
确认丢失



TCP: 传输控制协议

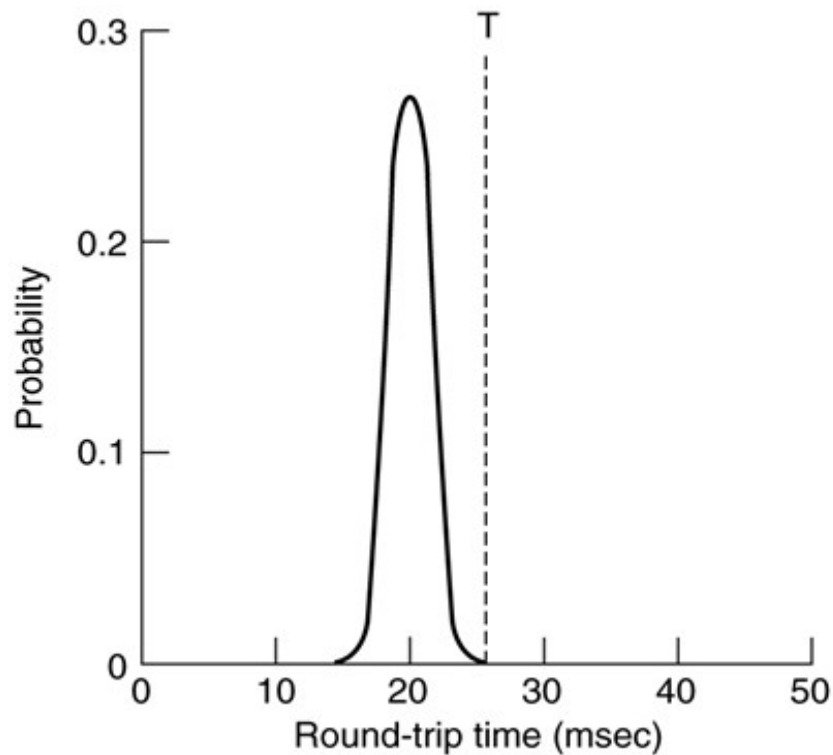
- TCP特性
- TCP段格式
- 连接管理
- 流量控制
- 差错控制
- TCP定时器管理
- 拥塞控制

TCP计时器管理



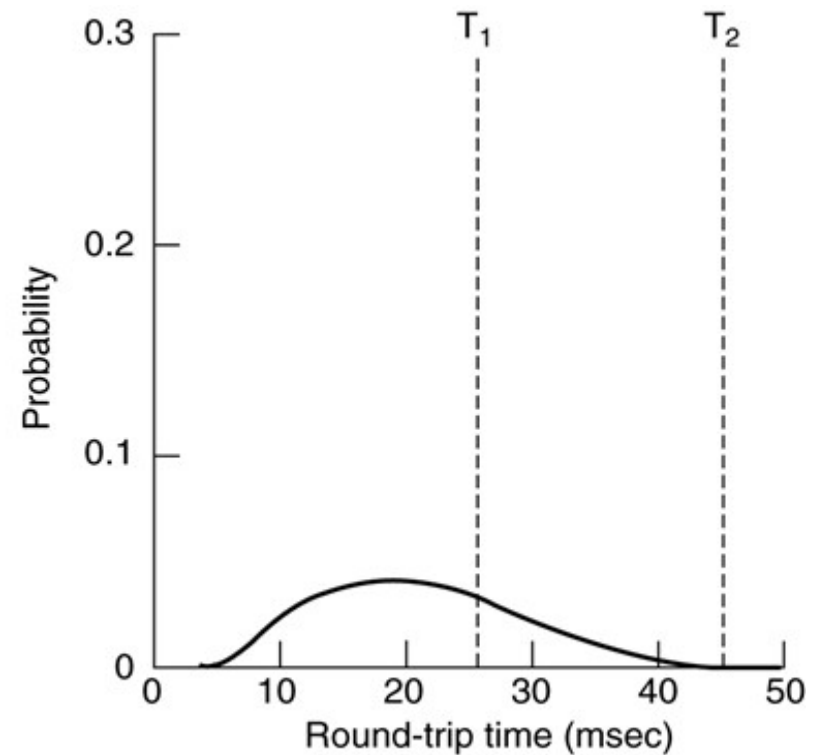
RTT 估算：数据链路层与TCP

■ 往返时间的概率密度(RTT)



(a)

数据链路层



(b)

TCP

重传计时器 (Textbook 6.5.10)

■ 如何设置超时?

- ◆ 太长: 低吞吐量, 低效率
- ◆ 太短: 数据包重复, 拥塞加重

■ 解决方案: making timeout proportional to RTT

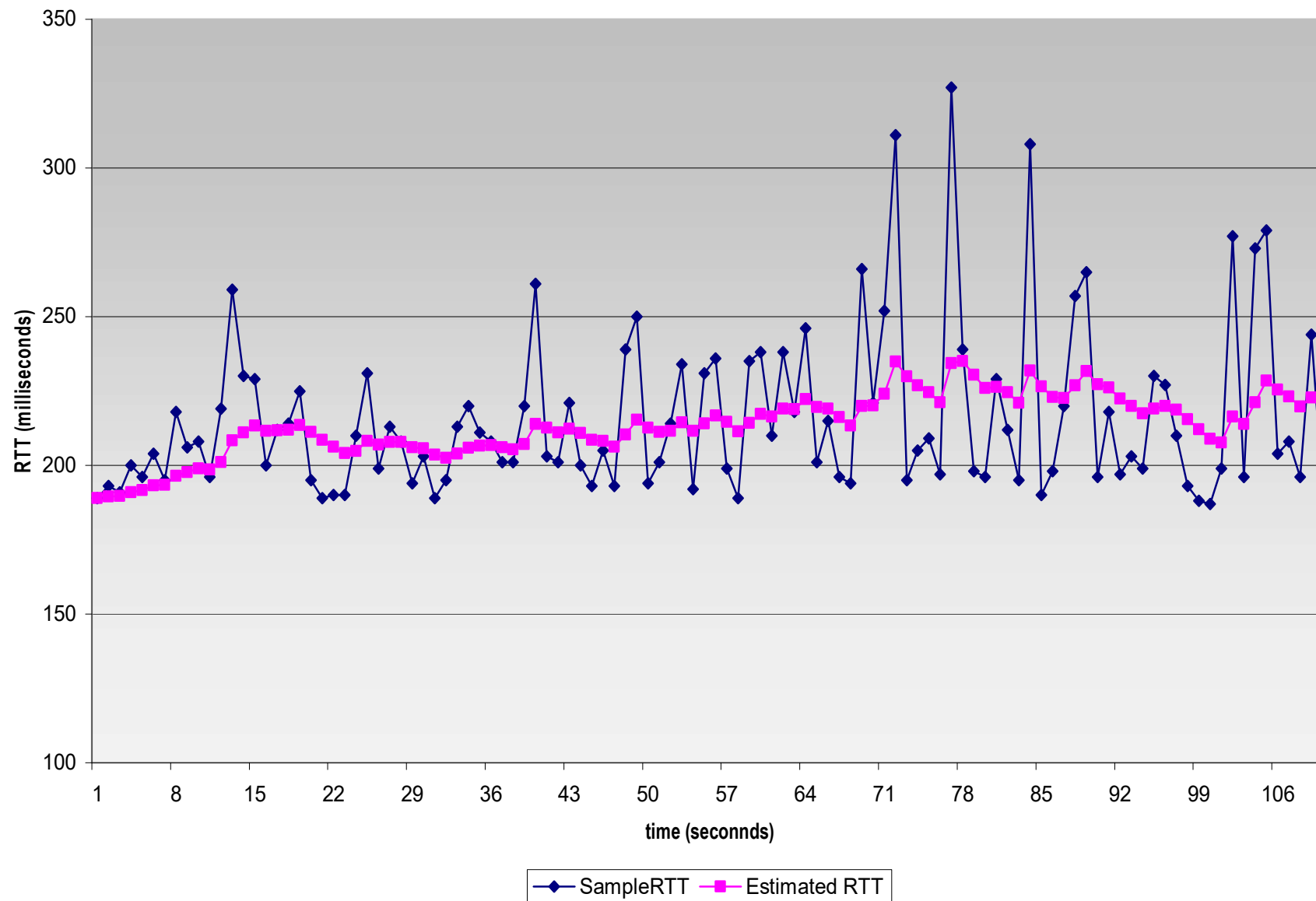
- ◆ 新RTT = $\alpha \times (\text{旧RTT}) + (1 - \alpha) \times (\text{新样值})$

样值: $AckRcvdTime - SendPacketTime$

$0 \leq \alpha < 1$, usually $7/8$

- ◆ SRTT: 平滑的RTT

举例：RTT估算



重传计时器: RTO

- Jacobson: 使用平均偏差
- RTO对RTT-variance (RTTVAR)和SRTT敏感
 - ◆ $RTTVAR = \beta \times RTTVAR + (1-\beta) \times |SRTT-Sample|$
 - ◆ $RTO = SRTT + 4 \times RTTVAR$
- 问题:对于重传, 如何计算采样和RTO?
 - ◆ Karn算法: 不更新任何重传段的估算值.
 - ◆ RTO在每次连续重传时加倍

TCP 计时器

■ 持续计时器

- ◆ 设计目的是防止0窗口死锁
- ◆ 当定时器超时，发送方向接收方发送一个探测消息，接收方对探测消息的响应中会给出窗口大小。

■ 保活计时器

- ◆ 设计目的是为了防止半开的连接
- ◆ 当连接长时间处于空闲状态时，**keepalive**计时器可能会超时，从而促使一端检查另一端是否仍然还在。

■ **TIMED WAIT**状态时使用的计时器

- ◆ 该定时器的超时值为两倍于数据包的最大生存期，主要用来确保当连接关闭后，由它创建的所有数据包已经完全消失

TCP: 传输控制协议

- TCP特性
- TCP段格式
- 连接管理
- 流量控制
- 差错控制
- TCP定时器管理
- 拥塞控制

因特网中的拥塞控制

- 当队列长度增大到很大时，路由器(工作在网络层)检测到拥塞，并试图通过丢弃数据包来管理拥塞
- 源主机(传输层, **TCP**)接收到网络层反馈来的拥塞信息，通过减慢其发送到网络的速率来缓解拥塞。

拥塞窗口

- 拥塞窗口(**cwnd**)是TCP发送方为每个连接保存的一个变量.

- ◆表示网络能够接收多少字节

- Van Jacobson解决方案

- 近似一个**AIMD**拥塞窗口

- ◆TCP维护一个拥塞窗口(**cwnd**)和一个流量控制窗口(**rwnd**),
两者都是动态变化的, 发送方窗口是其中较小的一个

- ◆分组丢失是网络拥塞的信号

- 仅适用于传输误码率相对很低的情况

- 这也说明了为什么**802.11**有自己的重传机制

- ◆需要一个良好的重传定时器来准确地检测丢包信号

- 计时器含**RTT**的均值和方差的估算方法

拥塞控制中需要解决的问题

■ 如何检测拥塞

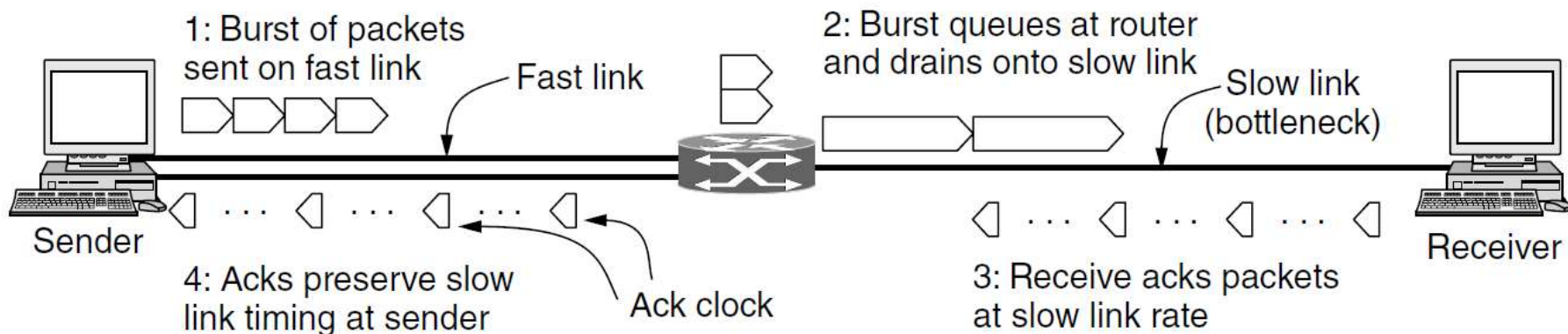
- ◆ 丢包是拥塞的最好标志
- ◆ 如何检测分组丢失? **ACKs**
 - TCP使用**ACK**来告知数据的接收
 - **ACK**指示接收到的最后一个连续字节
- ◆ 超时后没有**ACK**意味着丢包

■ 如何调整发送速率

- ◆ 收到**ACK**后，增加速率
- ◆ 当检测到丢包时，降低速率

ACK时钟

- 数据包确认的时间反映了数据包穿越慢速链路后到达接收方的时间
- As the ACKs travel over the net and back to the sender, routers **preserve** this timing
- 如果TCP以ACK返回的速度向网络中注入包，则包将以慢速链路允许的速度发送，不会排队并阻塞路径上的任何路由器
- 通过使用ACK时钟，TCP平滑了流量，避免了路由器上不必要的队列

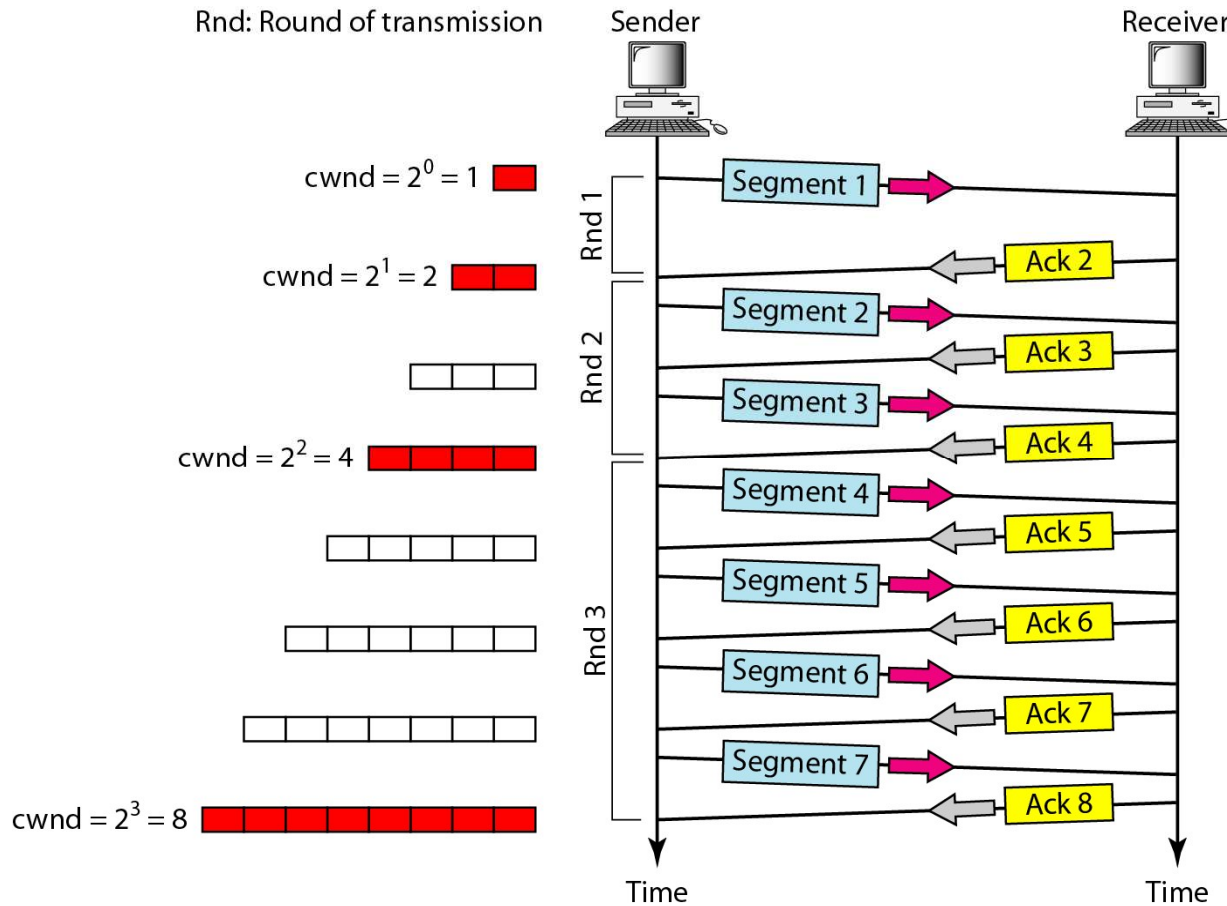


TCP拥塞控制算法

- 为了简单起见, 使用了段数而不是字节数
- 慢启动: initial rate is slow ($cwnd=1$)
 - ◆ 接收到ACK, $cwnd+1$
 - ◆ 指数增长很快就填满了网络“管道”
- 加法增模式: 一个更温和的调整算法——AIMD
 - ◆ 加法递增乘法递减
 - ◆ 缓慢递增, 剧烈递减
 - 收到ACK, $cwnd += 1/cwnd$
 - 检测到分组丢失 (超时), $cwnd = 1$
- 什么时候从慢启动阶段进入AIMD阶段?
 - ◆ 有必要设置一个阈值: *ssthresh*

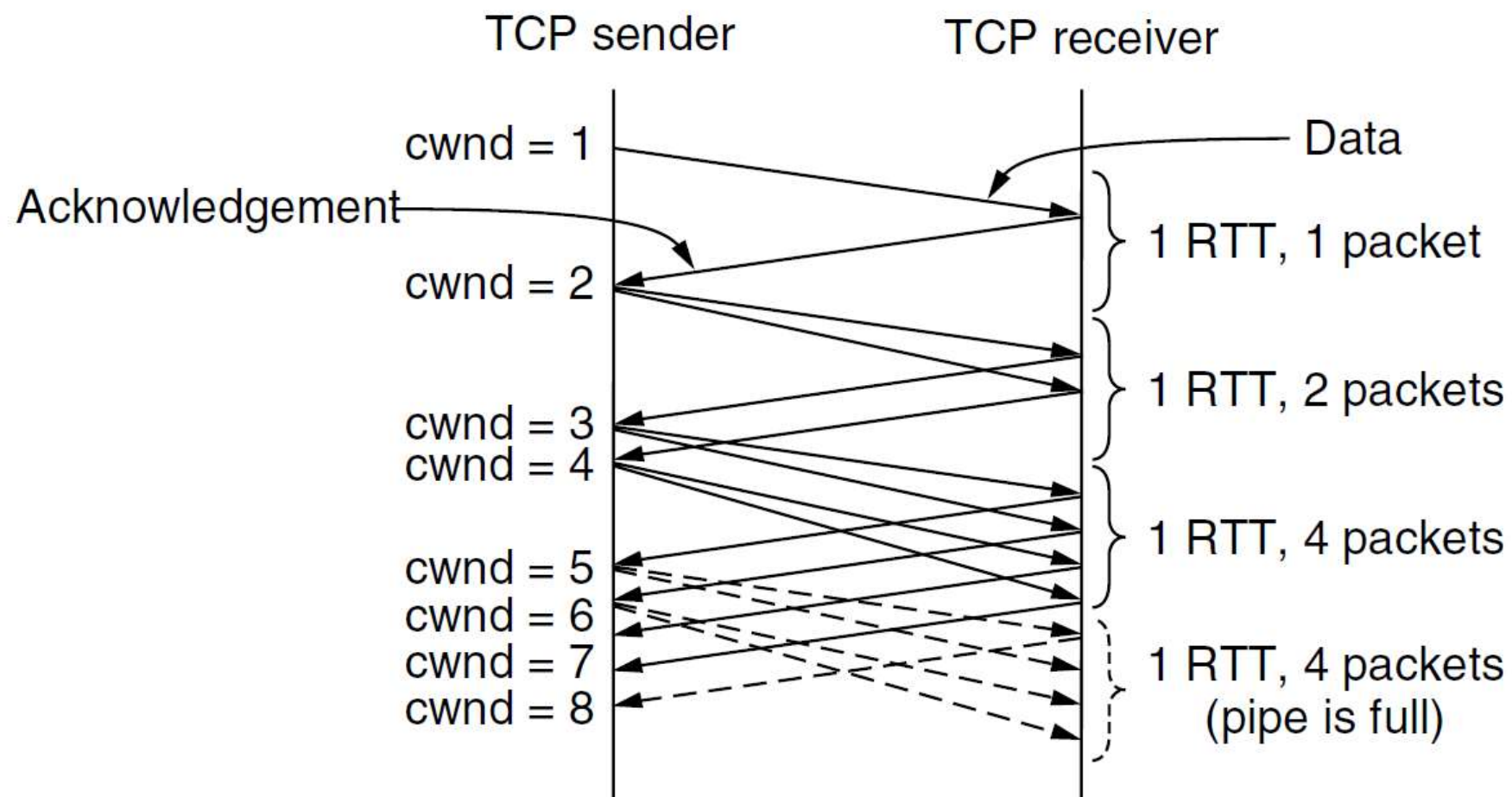


慢启动：指数递增

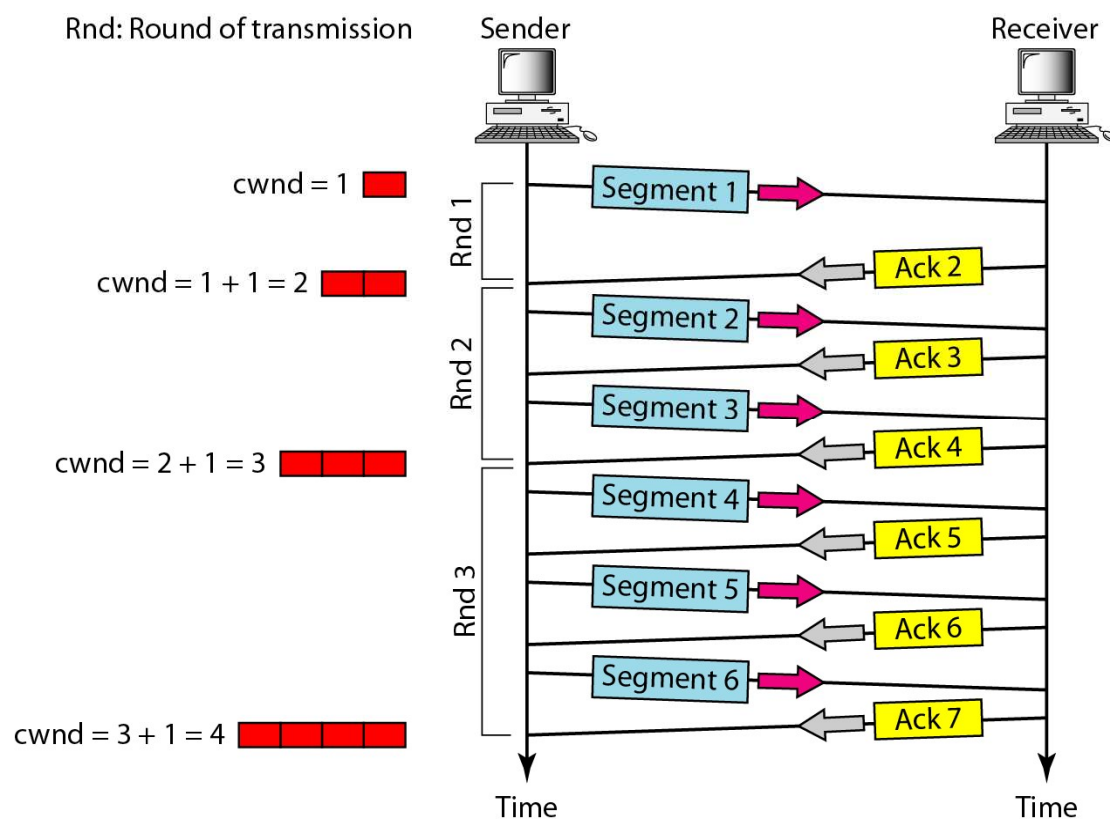


在慢启动算法中，拥塞窗口的大小呈**指数递增**，直到达到阈值

慢启动：初始拥塞窗口从1 MSS开始

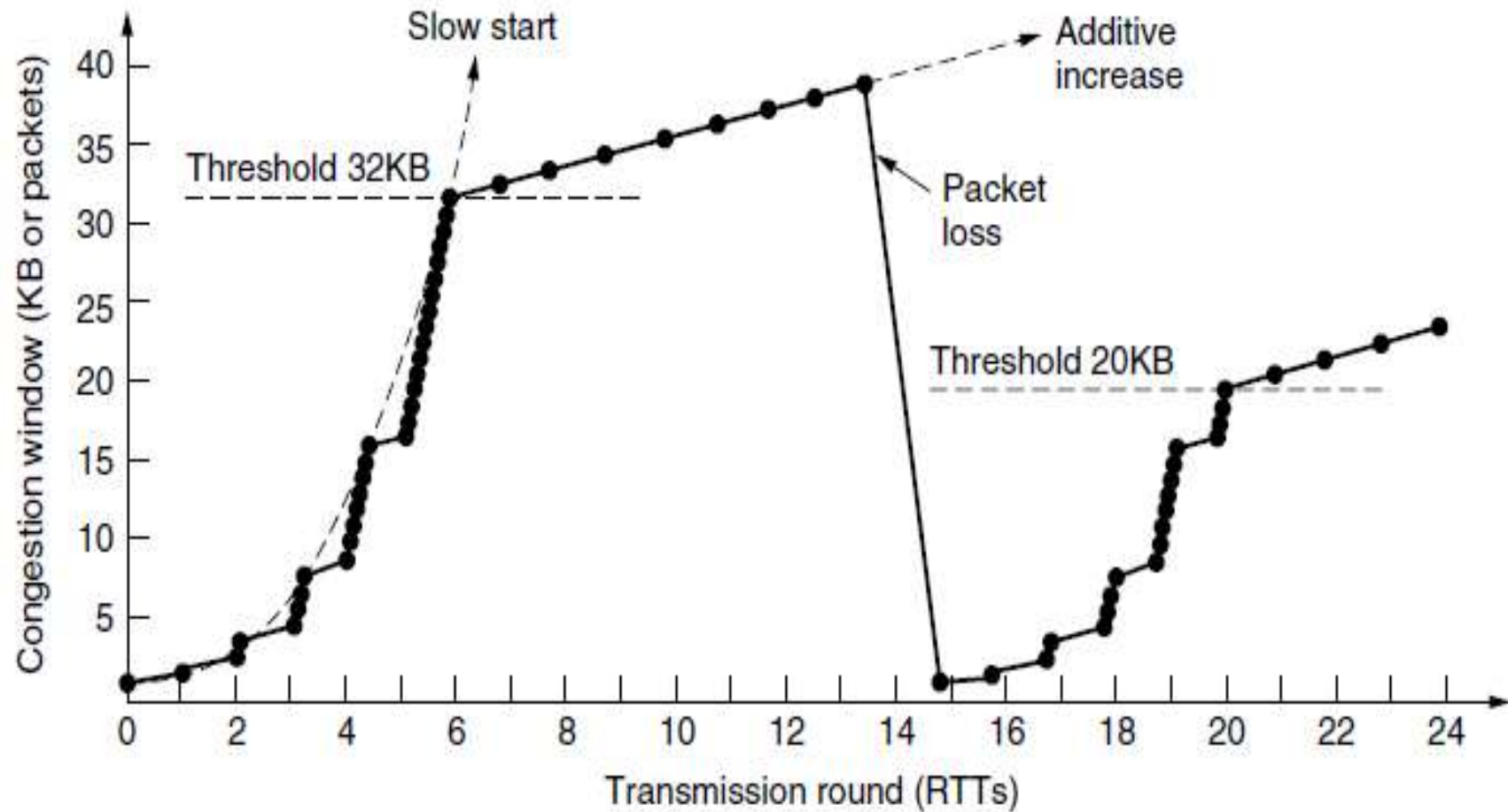


拥塞避免：加法递增



在拥塞避免算法中，拥塞窗口的大小**加法递增**，直到检测到拥塞为止。

慢启动和AIMD举例



TCP的拥塞控制策略

- 拥塞检测：隐式拥塞通知，TCP发送端根据丢包自行判断
- 慢启动：
 - ◆ 启动速率很低，TCP连接建立时， 拥塞窗口 = 1 MSS
 - ◆ 按指数级增加发送速率（每一轮发送结束，拥塞窗口加倍）
 - ◆ 直至拥塞窗口达到阈值或发现数据丢失
- AIMD（拥塞避免）
 - ◆ 拥塞窗口达到阈值时，降低发送速度的增长，按线性增长（每一轮发送结束，拥塞窗口增加1个MSS）
 - ◆ 出现定时器超时，拥塞窗口降为1个MSS，开始新的慢启动（阈值降为超时时的一半）
 - ◆ 收到3个重复的ACK，拥塞窗口降为当前值的一半，开始新的拥塞避免（AI）

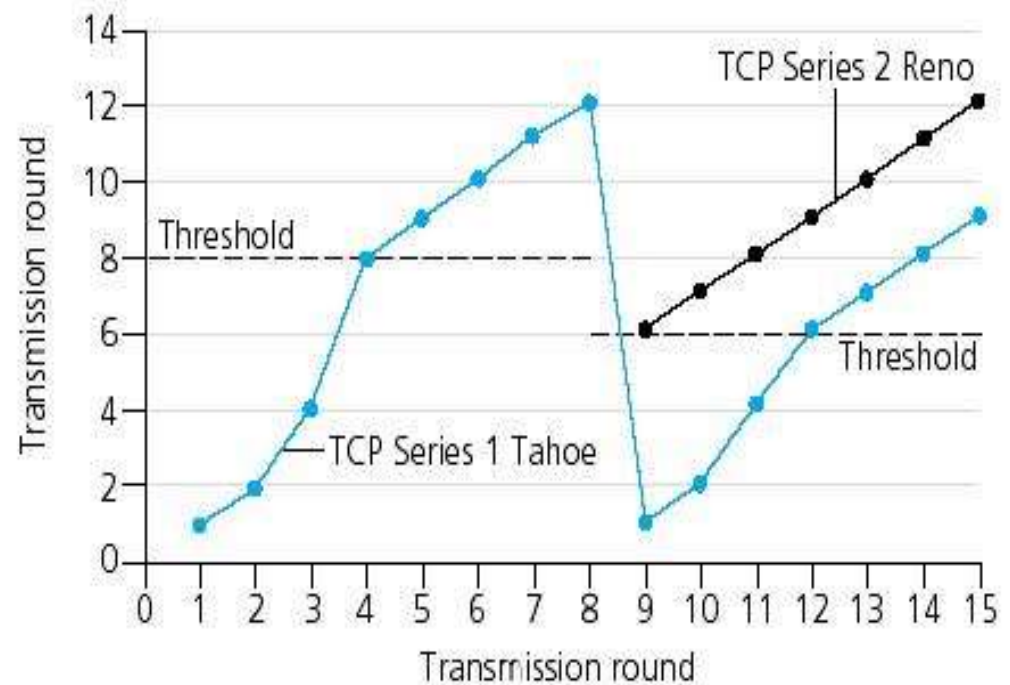
快速重传和快速恢复

- **快速重传**: TCP Tahoe将3个重复的ACK视为超时，然后开始一个新的慢启动阶段。
- 对于3个重复的ACK, TCP Reno将拥塞窗口减半，执行“快速重传”，并进入一个新的拥塞避免阶段：

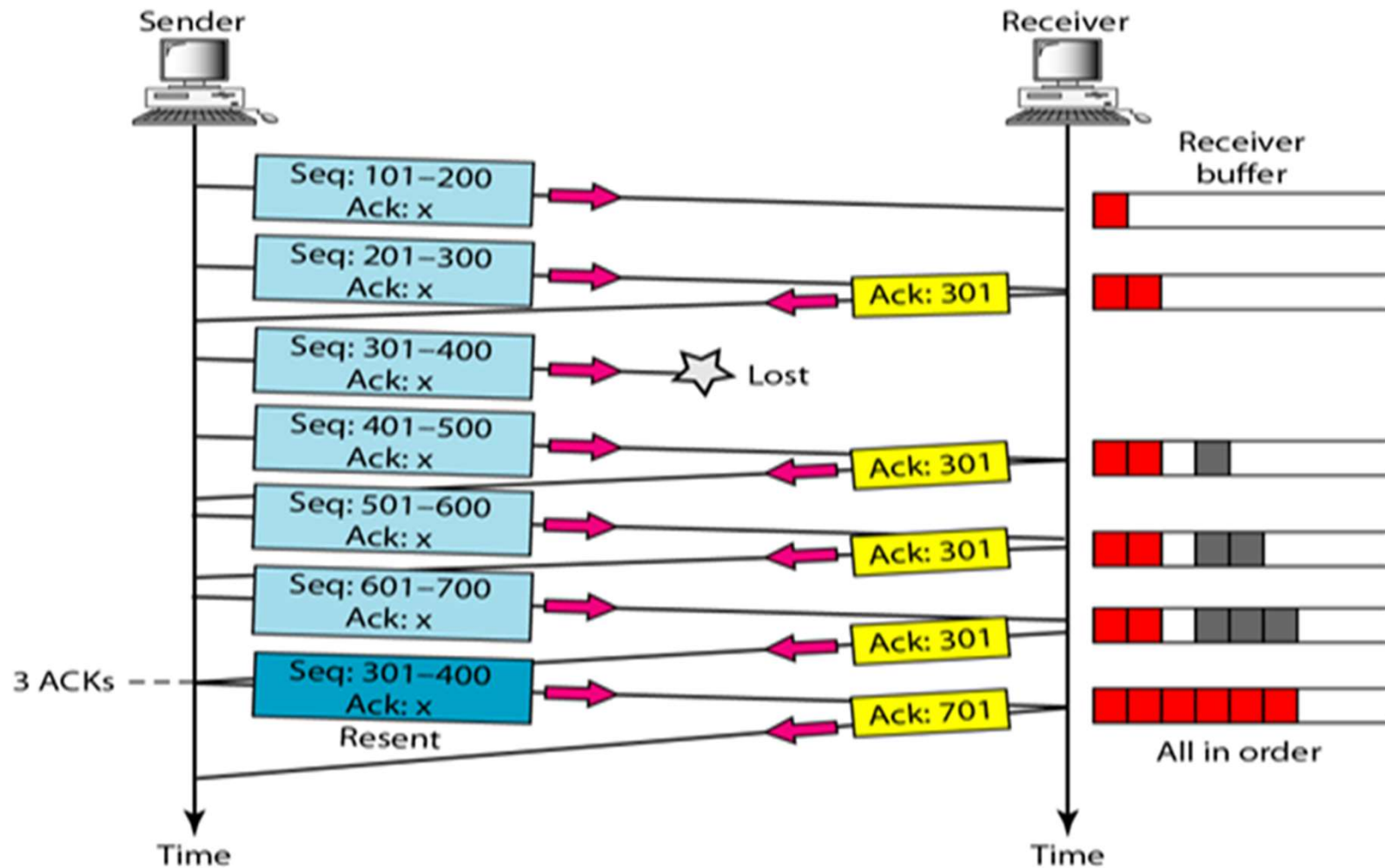
快速恢复

◆ $\text{threshold} = \frac{1}{2} \text{ window}$

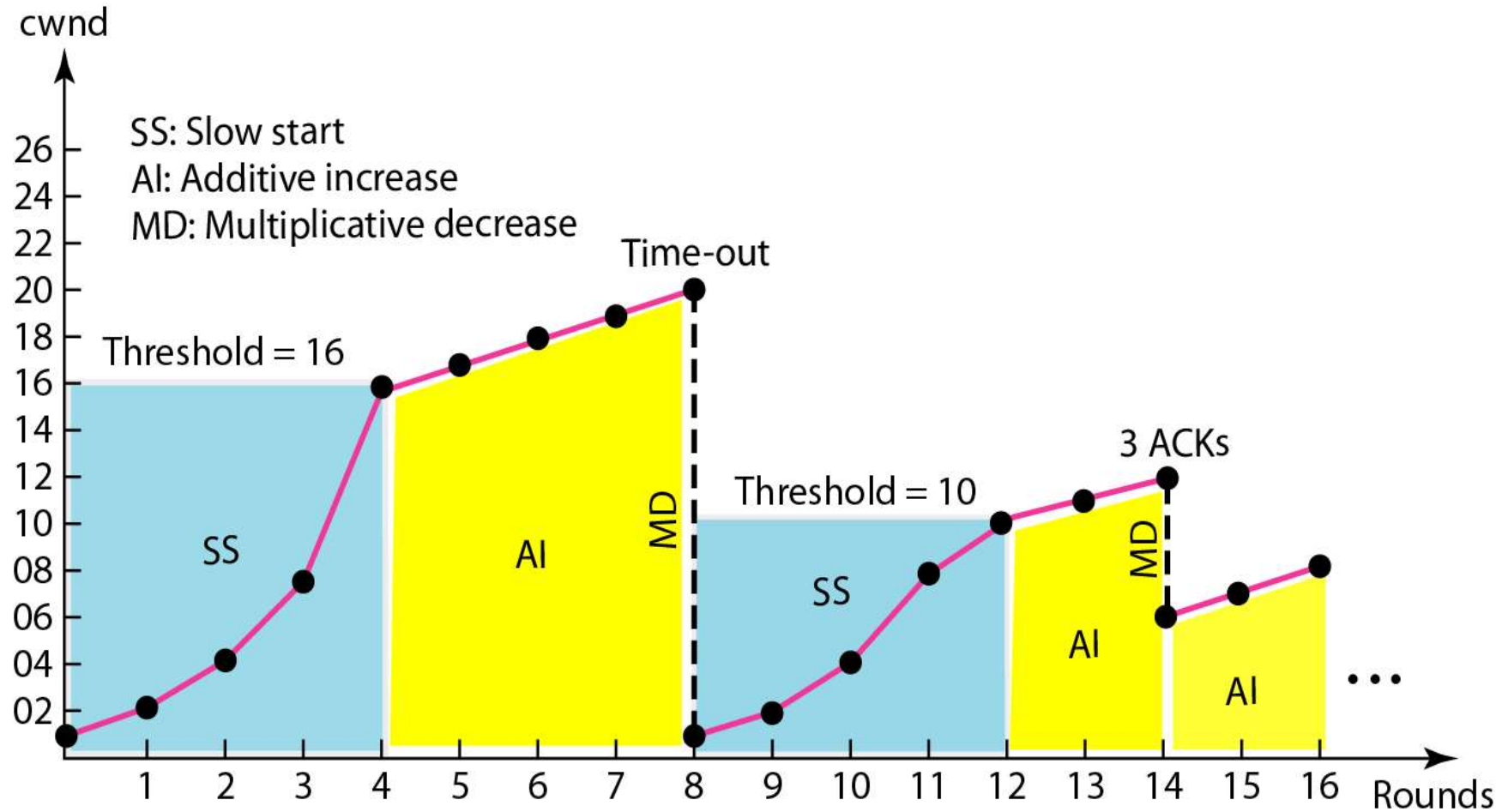
◆ $\text{cwnd} = \text{threshold}$



快速重传举例



TCP拥塞控制举例



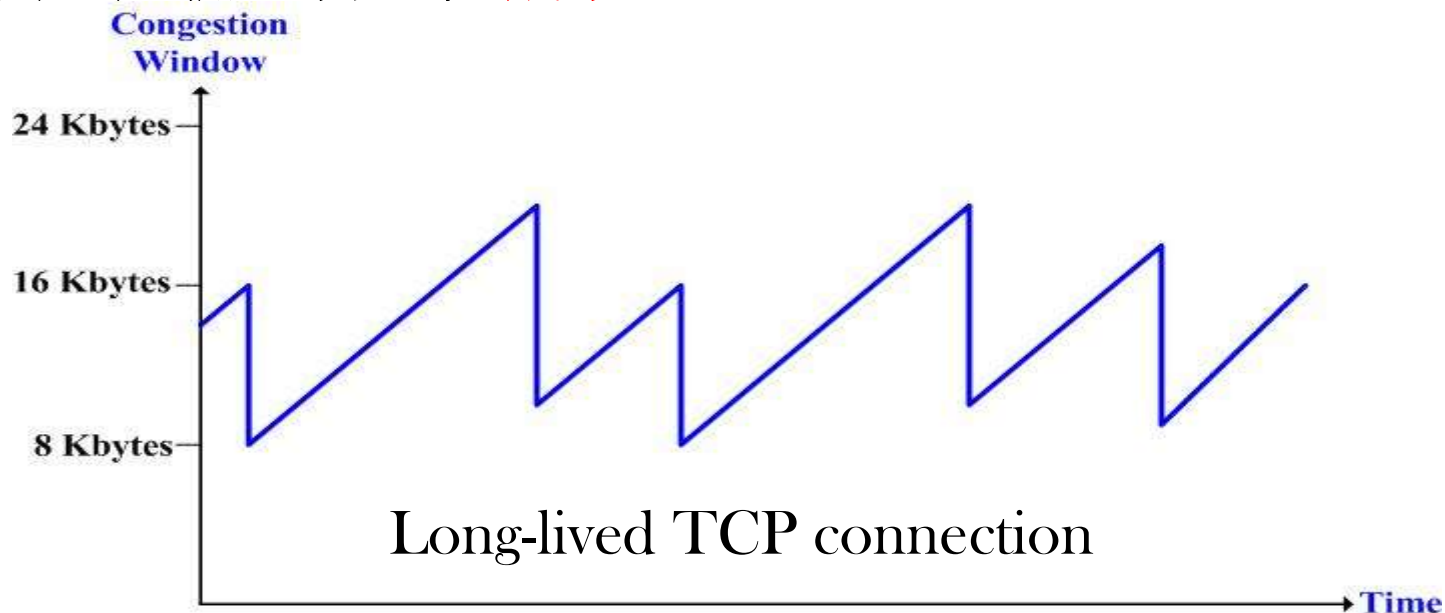
AIMD效应

■ 加法递增

- ◆ 当发送完全部窗口内的数据包而没有丢失时，拥塞窗口值加1
- ◆ 线性增加以避免拥塞

■ 乘法递减

- ◆ 丢包后拥塞窗口值减半

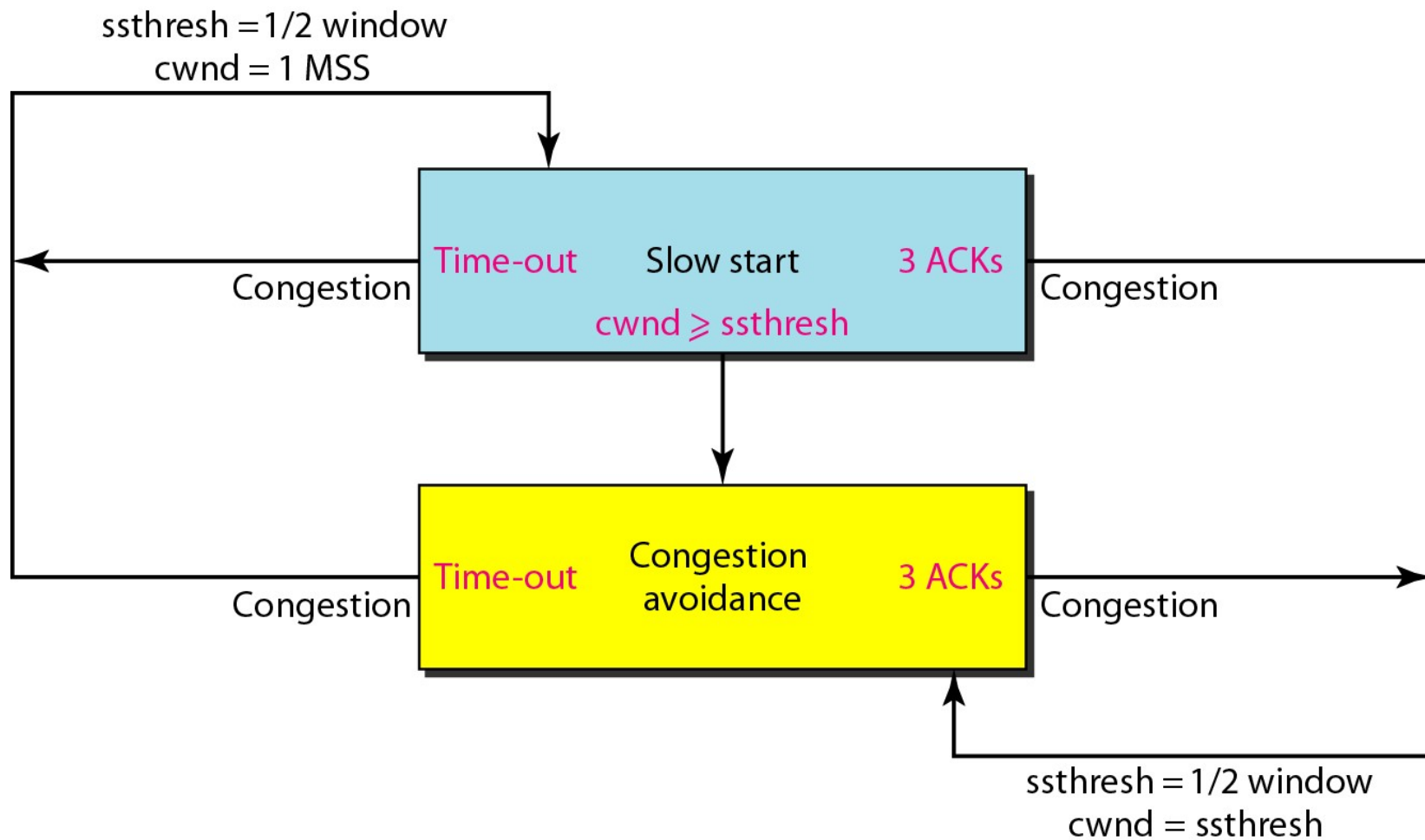


[Kurose]

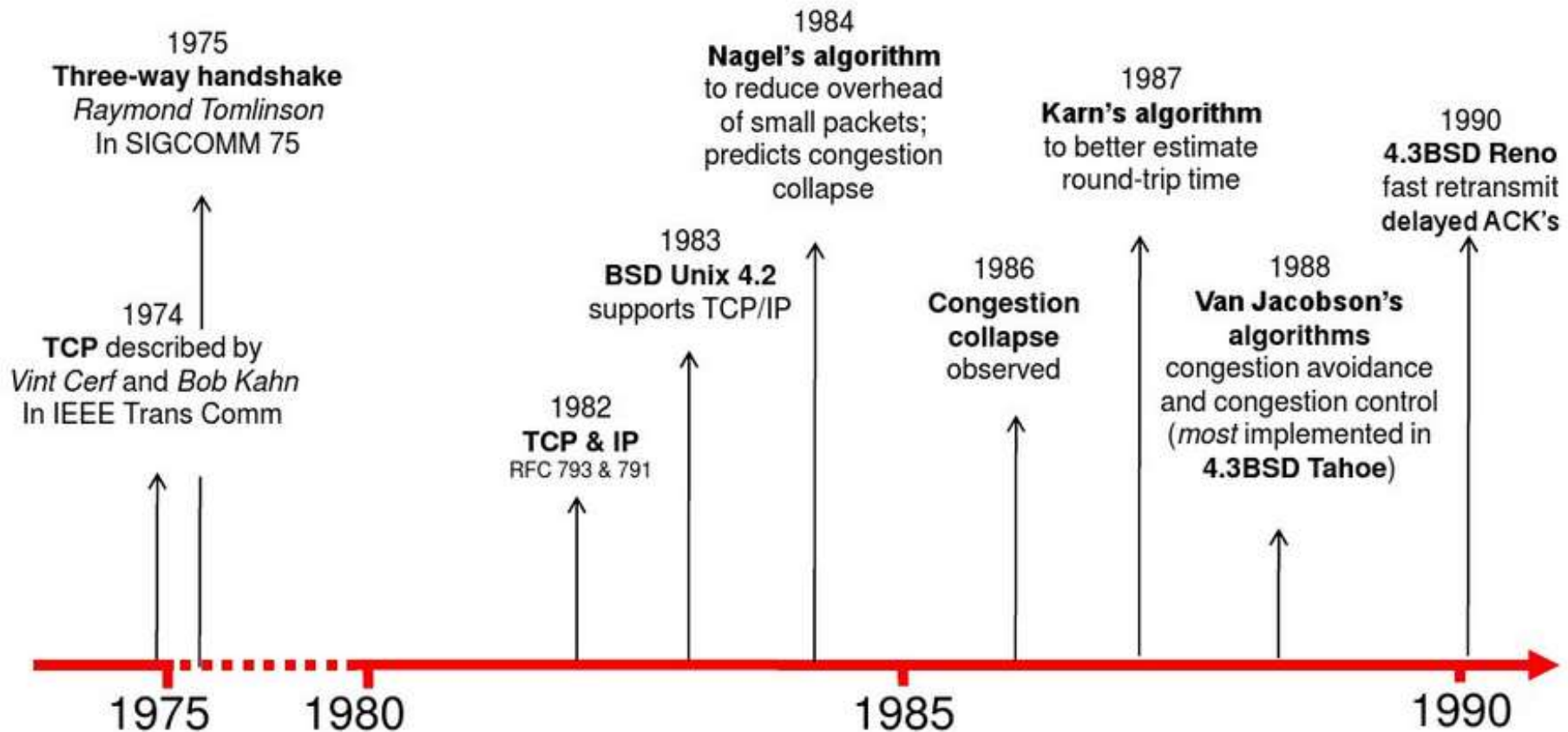
TCP拥塞控制小结 (1)

- 当拥塞窗口值(**Congwin**)小于阈值时, 发送方处于**慢启动**阶段, 窗口呈指数增长.
- 当拥塞窗口值(**Congwin**)大于阈值时, 发送方处于**拥塞避免**阶段, 窗口线性增长.
- 当收到三个重复**ACK(3-duplicate-ACK)**时, 阈值减到拥塞窗口值(**Congwin**)的一半, 并令**CongWin**值等于阈值.
- 超时时, 阈值设置为拥塞窗口值(**Congwin**)的一半, 并令**CongWin**值等于1MSS.

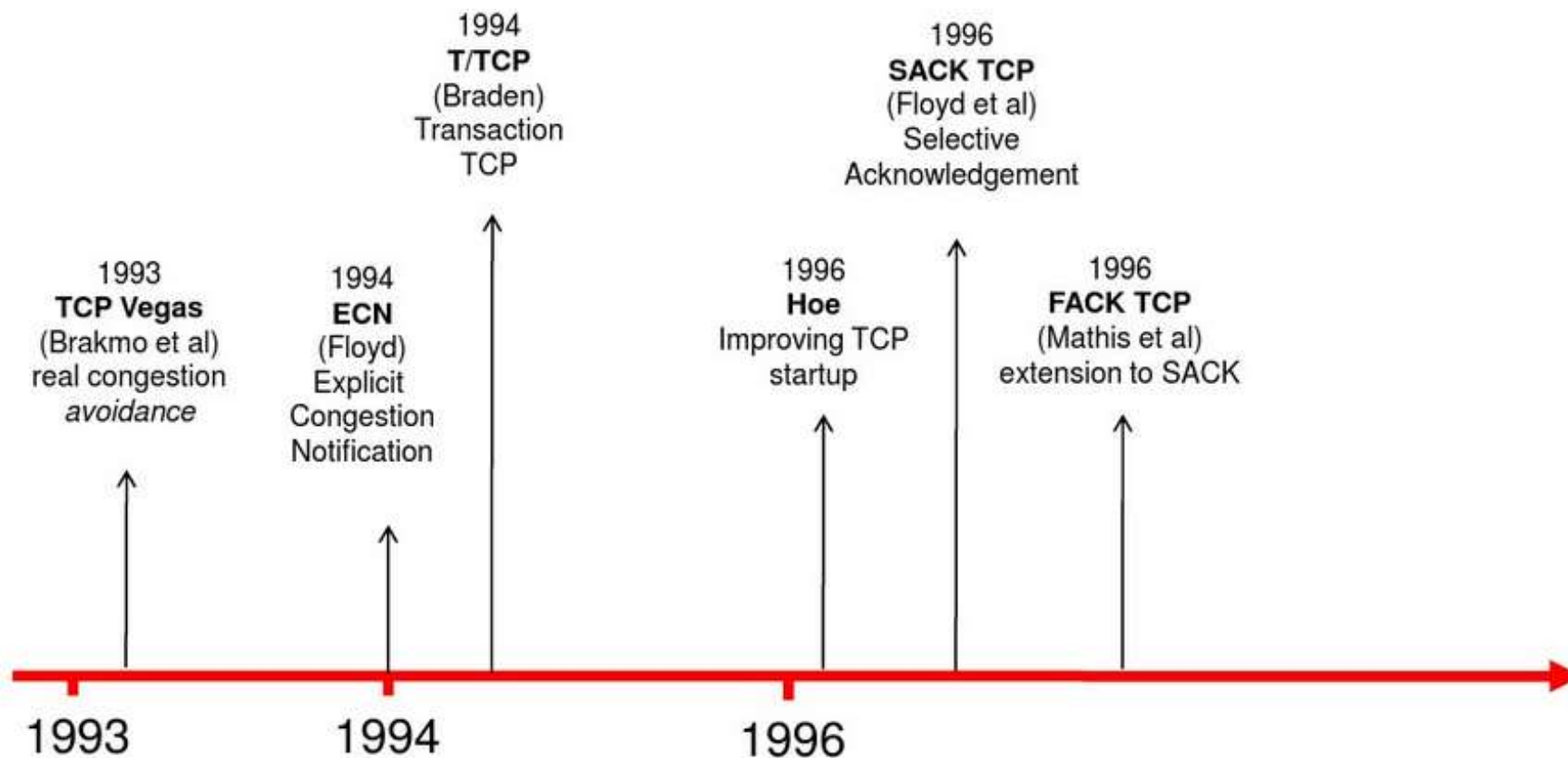
TCP拥塞控制小结(2)



TCP演进(1)



TCP演进(2)



TCP总结(1)

- 提供面向连接的字节流服务
- 采用三次握手建立连接
- 采用四步释放或三次握手关闭连接
- 采用**ARQ**机制实现差错控制
- 采用动态的滑动窗口实现流量控制
 - ◆ 减少小包，提高传输效率
 - ◆ **Nagle's**算法：发送方使用，一次尽可能发送较多的数据
 - ◆ **Clark's** 算法：接收方使用，在有较大空闲缓存时才发送窗口更新通知，解决傻瓜窗口问题
- 超时重传时间**RTO**
 - ◆ **Jacobson**算法，基于**RTT**

$$RTT \text{ 估值} = \alpha \times RTT \text{ 旧估值} + (1 - \alpha) \times RTT \text{ 采样值}$$

TCP总结(2)

■ 拥塞控制

- ◆开始时采用慢启动，拥塞窗口初值为1MSS，每轮发送成功之后加倍
- ◆达到阈值之后，采用拥塞避免，拥塞窗口按照AI递增，每轮发送成功之后增加一个MSS
- ◆一旦出现定时器超时，则采用MD，拥塞窗口降为1，开始新的慢启动，新阈值为超时发生时的拥塞窗口值的一半
- ◆当收到3个重复的ACK时，采用快速重传，从当前拥塞窗口值的一半开始新的拥塞避免，按照AI递增

本章要点(1)

■ 传输层功能与服务

- ◆ 进程-进程传输
- ◆ 可靠传输，拥塞控制

■ 设计问题

- ◆ 数据链路层与传输层
- ◆ 寻址: 端口号
- ◆ 连接管理: 三次握手
- ◆ 拥塞控制: 最大最小公平原则

■ UDP

- ◆ 快速高效, 尽力而为的服务
- ◆ IP之上的多路复用

■ TCP

- ◆ 连接建立: 三次握手
- ◆ 连接释放: 三次握手或 or 4步双向释放
- ◆ 差错控制: 校验和, ARQ, 快速重传

本章要点(2)

- 流量控制: 面向字节的滑动窗口
 - ◆ 发送窗口= $\min(\text{接收窗口}, \text{拥塞窗口})$
 - ◆ Nagles算法: 发送方的解决方案
 - ◆ 低能窗口综合症: Clark解决方案, 接收方的解决方案
- RTT 和 RTO的估算
- 拥塞控制
 - ◆ 慢启动: 拥塞窗口指数递增, 直到达到阈值
 - ◆ 拥塞避免(AI): 拥塞窗口线性递增
 - ◆ 乘法递减(MD): 当ACK超时, 开始一个新的慢启动过程
 - ◆ 快速重传: 当收到3个重复的ACK时, 开始一个新的拥塞避免过程

Terms

缩写	英文全称	中文
ACK	Acknowledgement	确认位
	addressing	编址/寻址
AIMD	Additive Increase Multiplicative Decrease	加法增乘法减
API	Application Program Interface	应用编程接口
ARQ	Automatic Retransmission reQuest	自动请求重发
	checksum	校验和/检查和
	Client-Server model	客户/服务器模型
	Congestion Avoidance	拥塞避免
	congestion control	拥塞控制
	congestion window	拥塞窗口
	connectionless service	无连接服务
	connection-oriented service	面向连接的服务

缩写	英文全称	中文
CWR	Congestion Window Reduced	拥塞窗口缩减
	Cumulative ACK	累积ACK
	datagram	数据报
ECE	Explicit Congestion Notification-ECHO	显示拥塞通知-回声
ECN	Explicit Congestion Notification	显式拥塞通知
	Fast Recovery	快速恢复
	Fast Retransmission	快速重传
	flow control	流量控制
FIN	Finish	终止(连接)位
	Internet	因特网

缩写	英文全称	中文
	Max-Min Fairness	最大最小公平性
	Maximum Segment Size	最大报文段长度
MTU	Maximum Transmission Unit	最大传输单元
	Multiplexing/Demultiplexing	复用/解复用(分用)
	packet	分组/包
	process	进程
	Pseudoheader	伪报头
PSH	push	推送(数据)位
QoS	Quality of Service	服务质量
RST	Reset	(连接)重启位

缩写	英文全称	中文
RTT	Round-Trip Time	环回时间
RTO	Retransmission Time-Out	重传超时时间
	TCP segment	TCP报文段
SACK	Selective ACK	选择确认
SAP	Service Access Point	服务访问点
SCTP	Stream Control Transmission Protocol	流控制传输协议
	Service Primitives	服务原语
	Silly Window Syndrome	傻瓜窗口症状
	sliding window	滑动窗口
	Slow Start	慢启动
	Smoothed Round-Trip Time	平滑的环回时间
SYN	Synchronize	(序号)同步位
TCP	Transmission Control Protocol	传输控制协议
UDP	User Datagram Protocol	用户数据报协议
URG	Urgent	紧急位
WAN	Wide Area Network	广域网

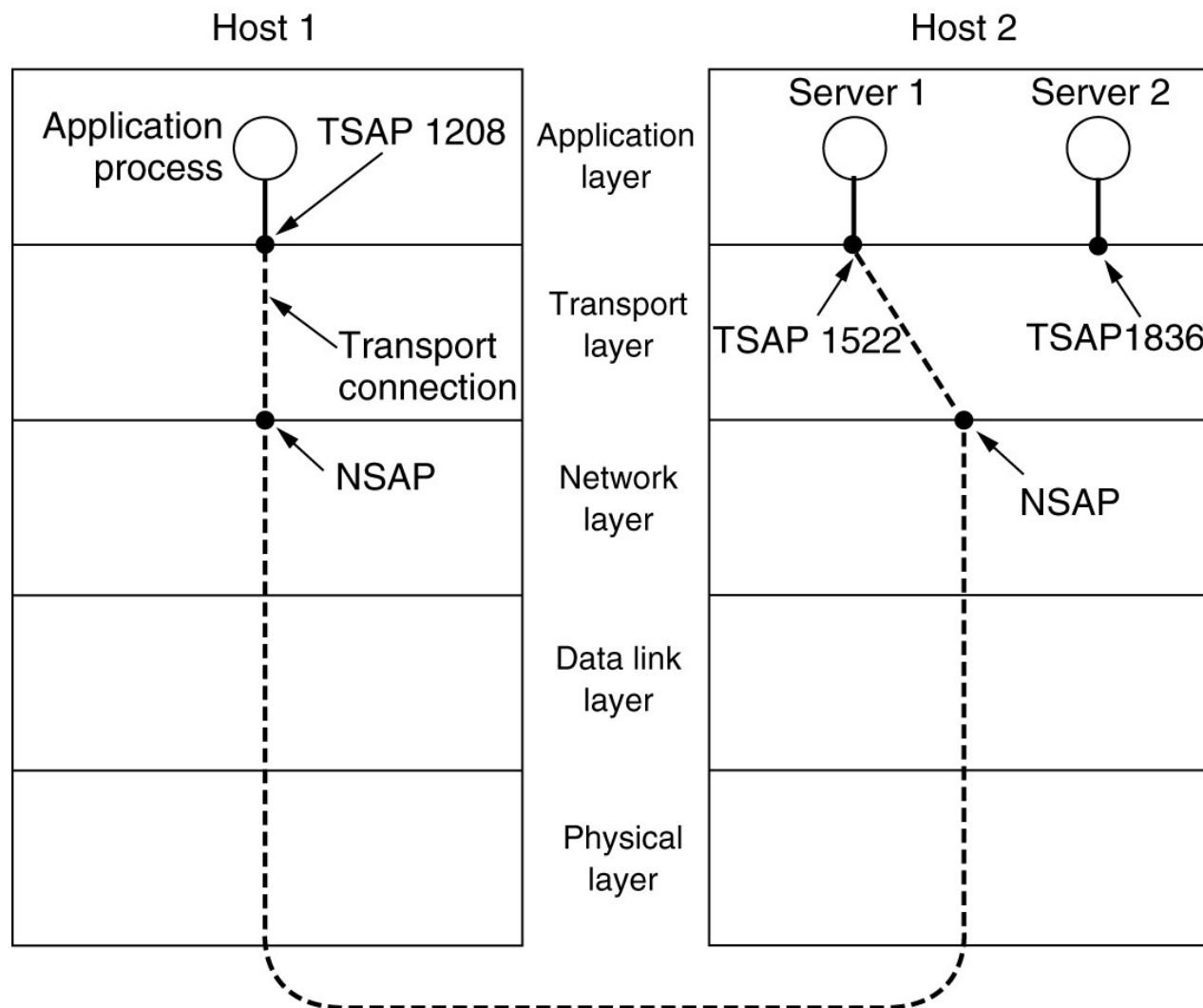
Copyright Information

- Some figures used in this presentation have been either directly copied or adapted from following materials
 - ◆ Lecture notes of book "Computer Networks, a Top-down Approach", J.Kurose and W.Ross, 3rd Edition, which are marked with [Kurose]
 - ◆ "Data communications and networking", B.A.Forouzan, which are marked with [Forouzan]
 - ◆ "计算机网络", 谢希仁, 第五版, 电子工业出版社, 2008年, which are marked with [谢]

自阅部分

寻址

传输服务访问点(TSAP)、网络服务访问点(NSAP) 和传输层连接



寻址

- 获得服务器的传输服务访问点TSAP(即端口)
 - ◆ 熟知端口
 - ◆ 端口映射器(Portmapper): 如名字服务器或目录服务器
- 进程服务器（节省资源）
 - ◆ 如果存在许多服务器进程，其中大多数很少使用，那么让每个进程都处于活动状态是一种浪费
 - ◆ 进程服务器可以作为不常用的服务器的代理
 - ◆ 仅适用于服务器进程可按需创建的场所

进程服务器

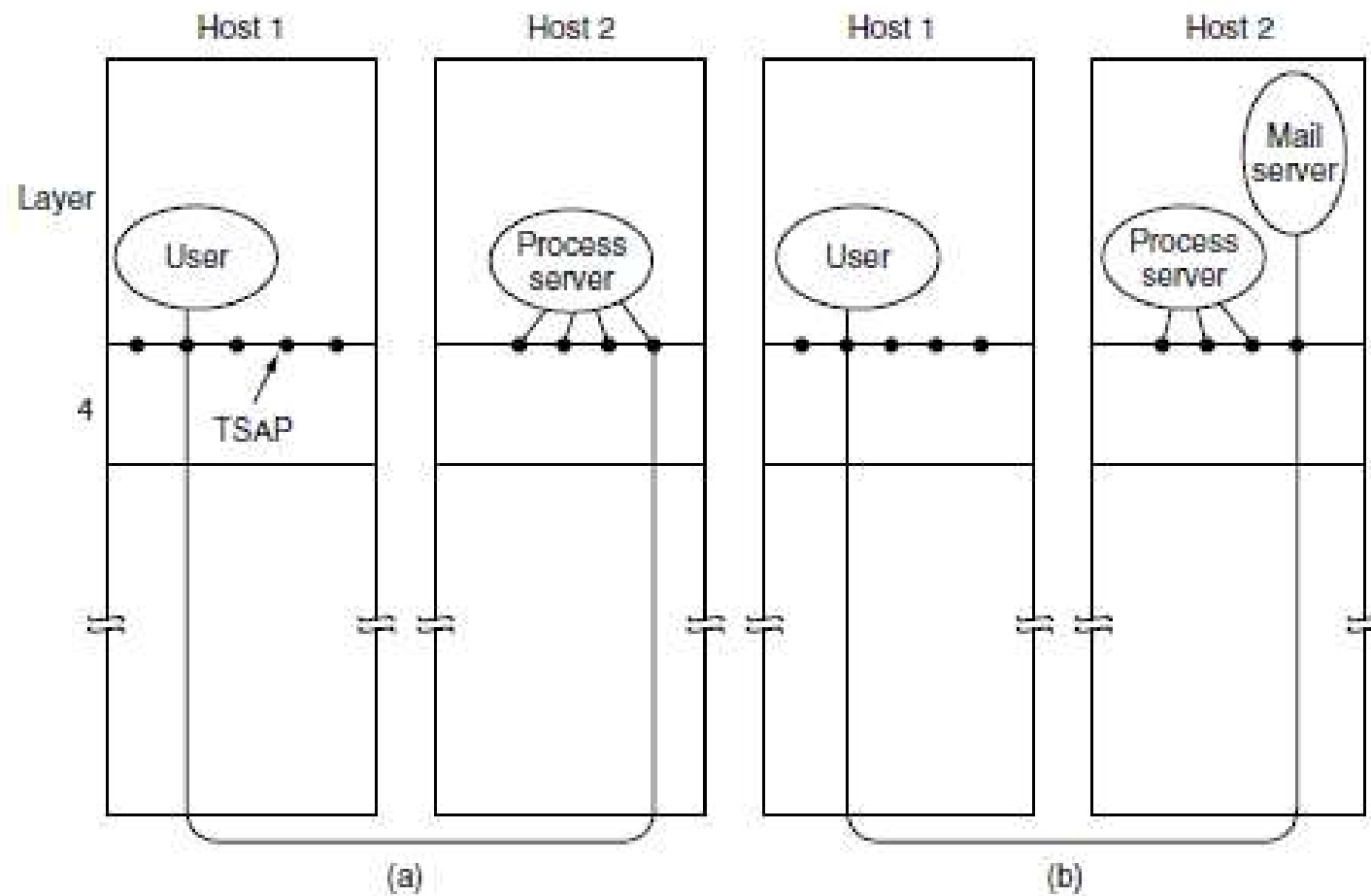
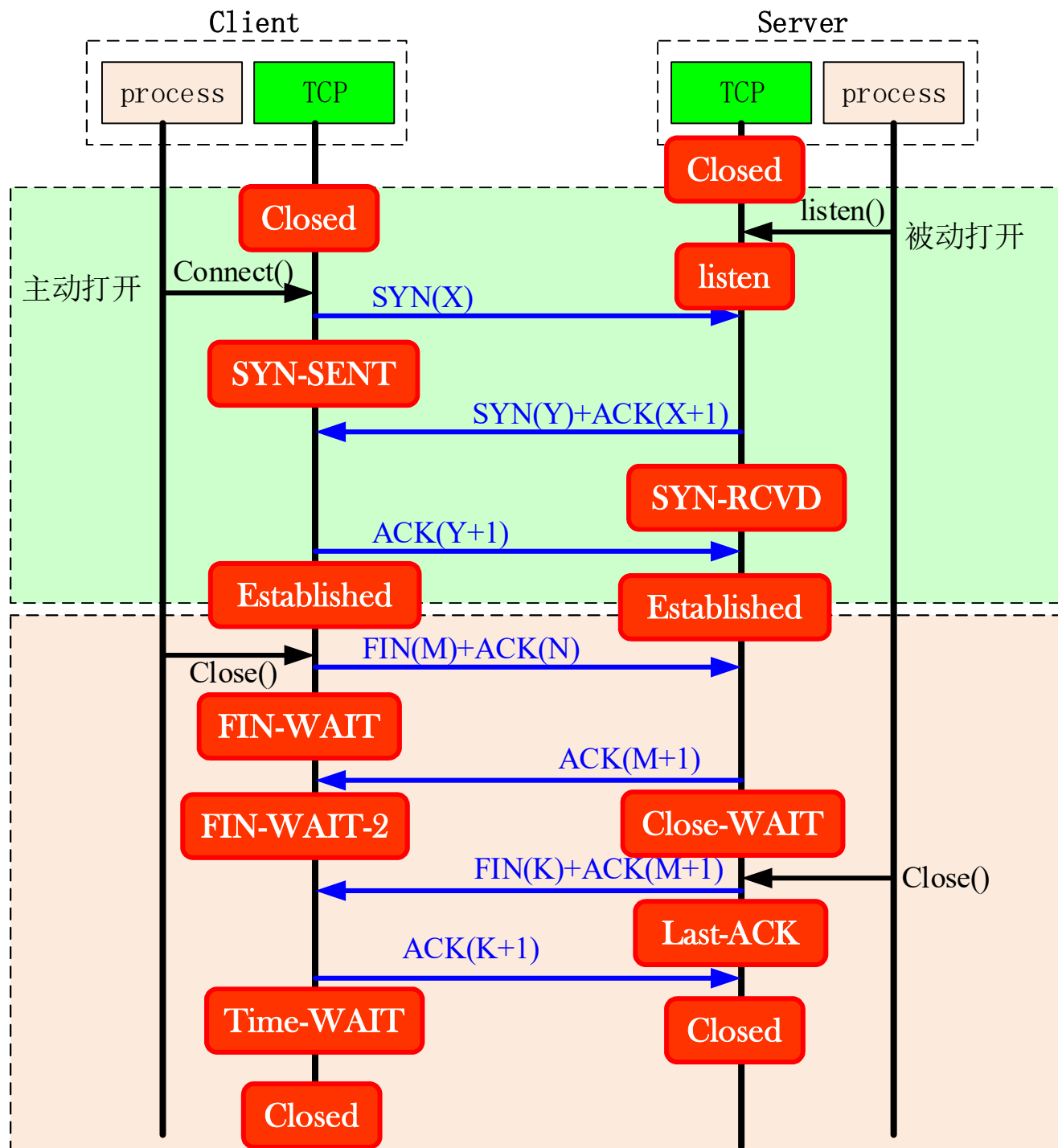


Figure 6-9. How a user process in host 1 establishes a connection with a mail server in host 2 via a process server.

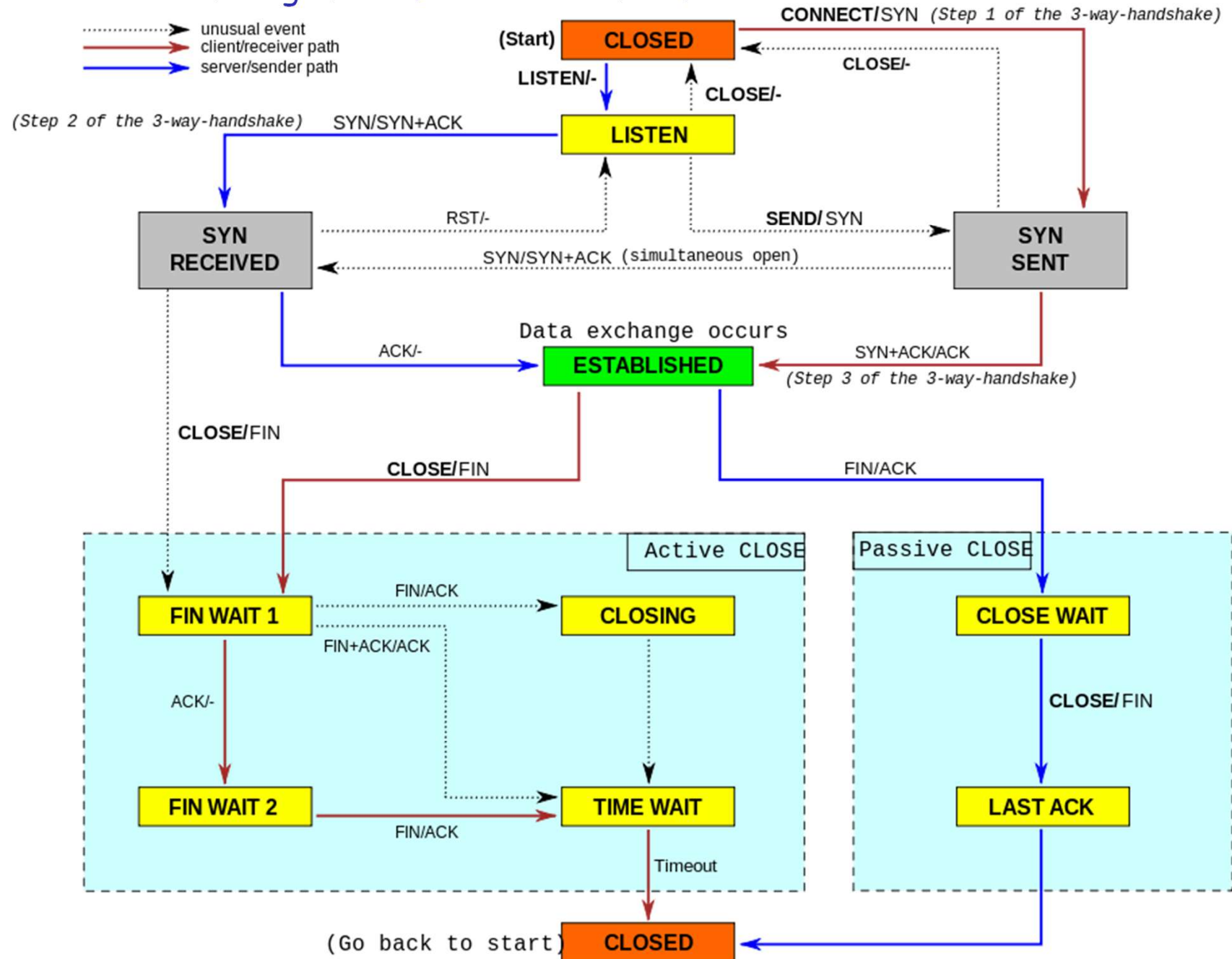
TCP 连接管理建模

State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIMED WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

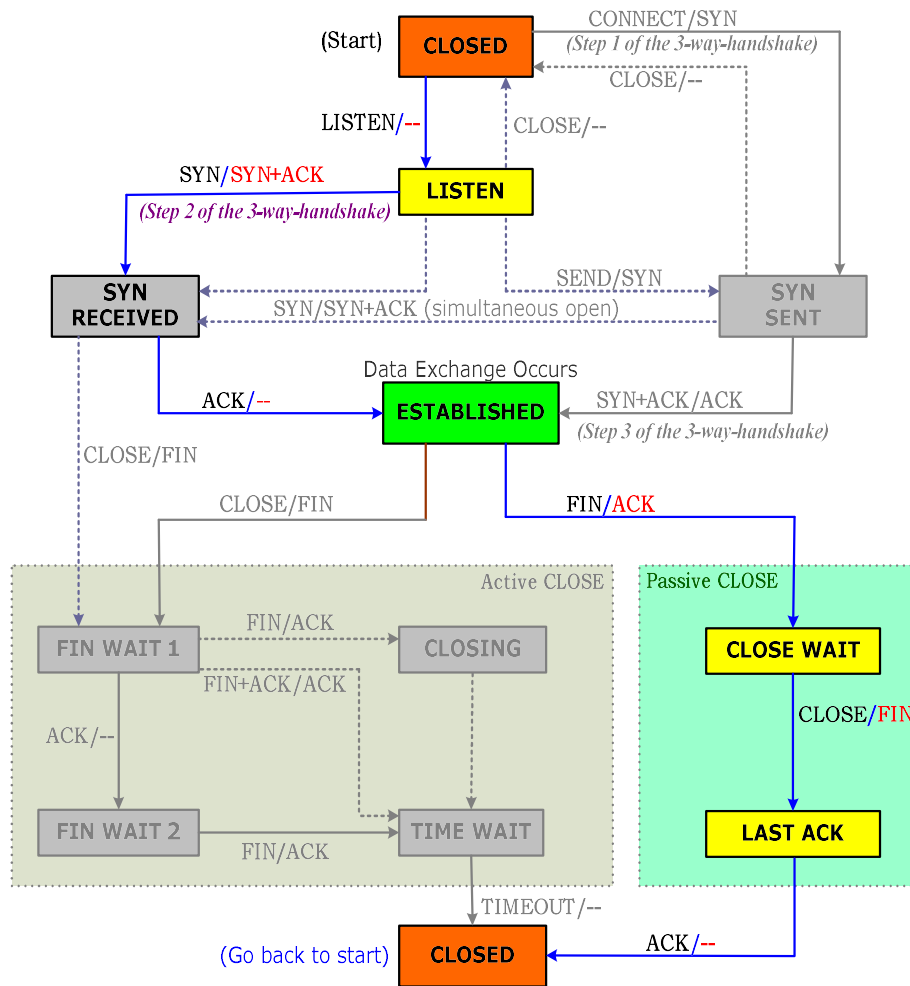
TCP连接管理有限状态机使用的状态



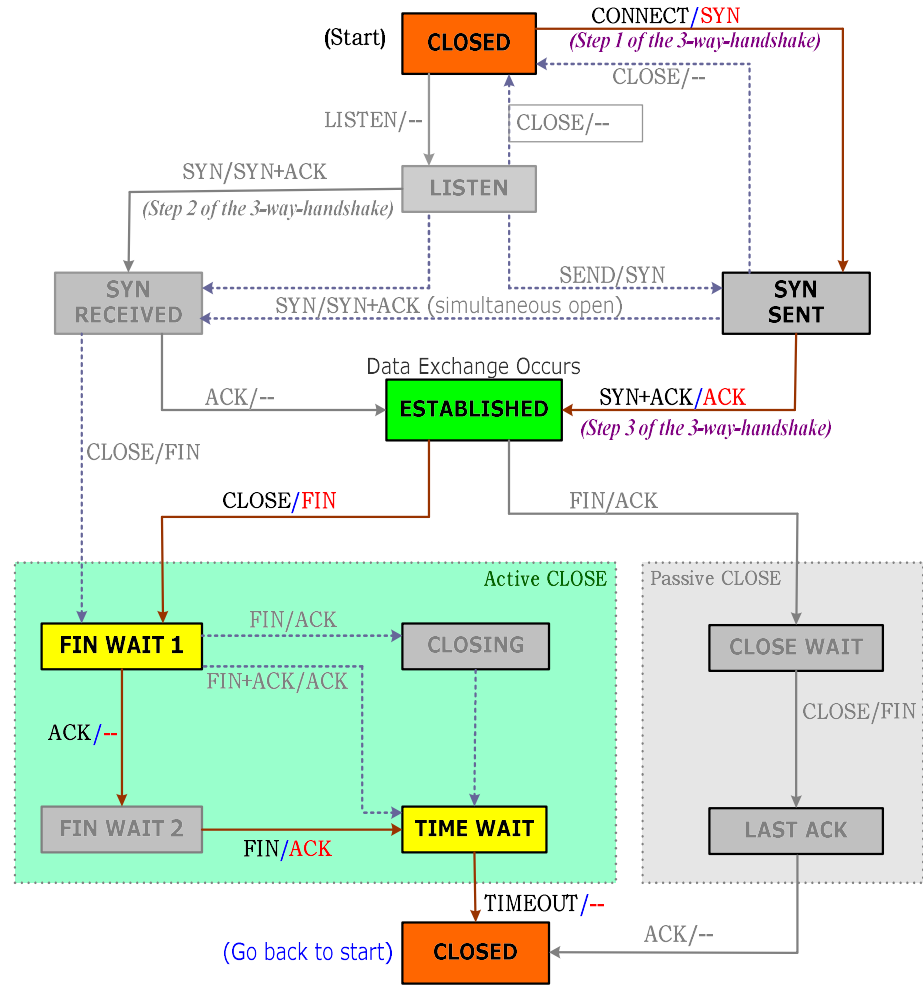
TCP connection management finite state machine.



TCP有限状态机

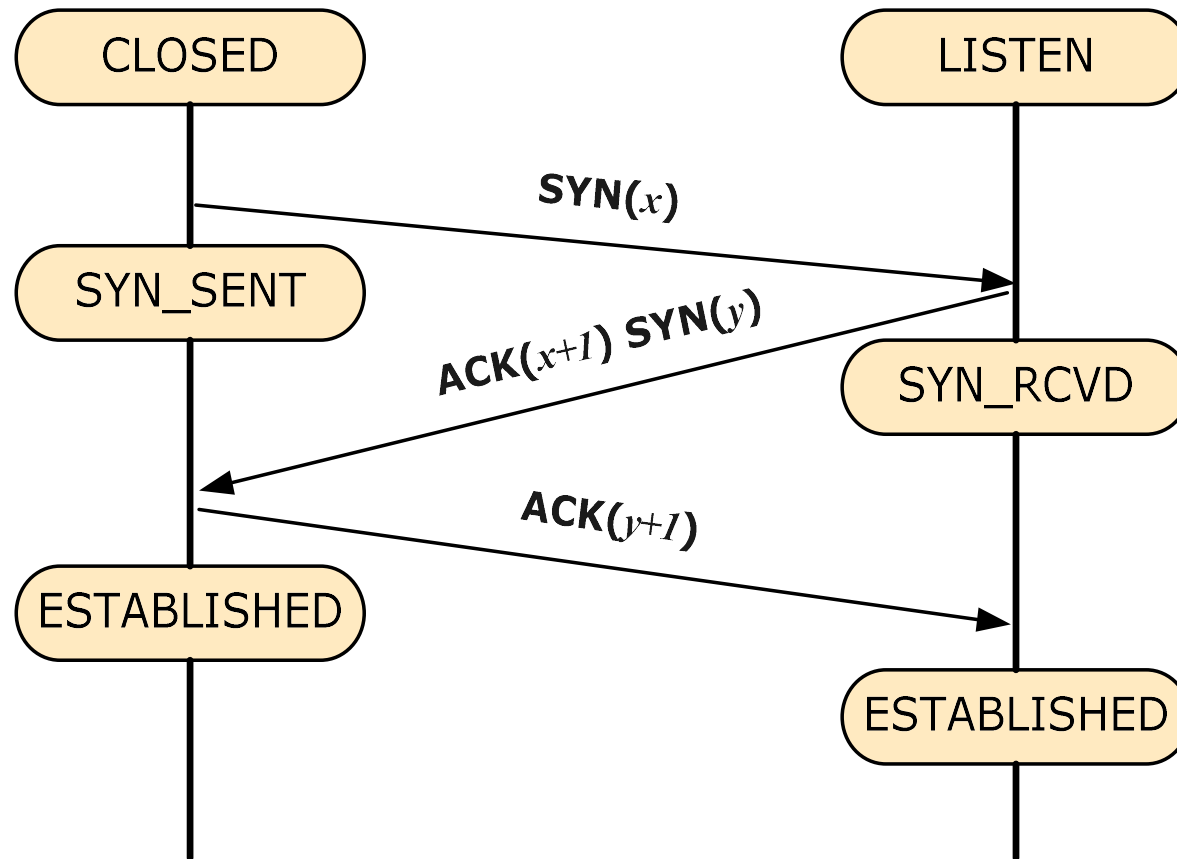


(Server Host)

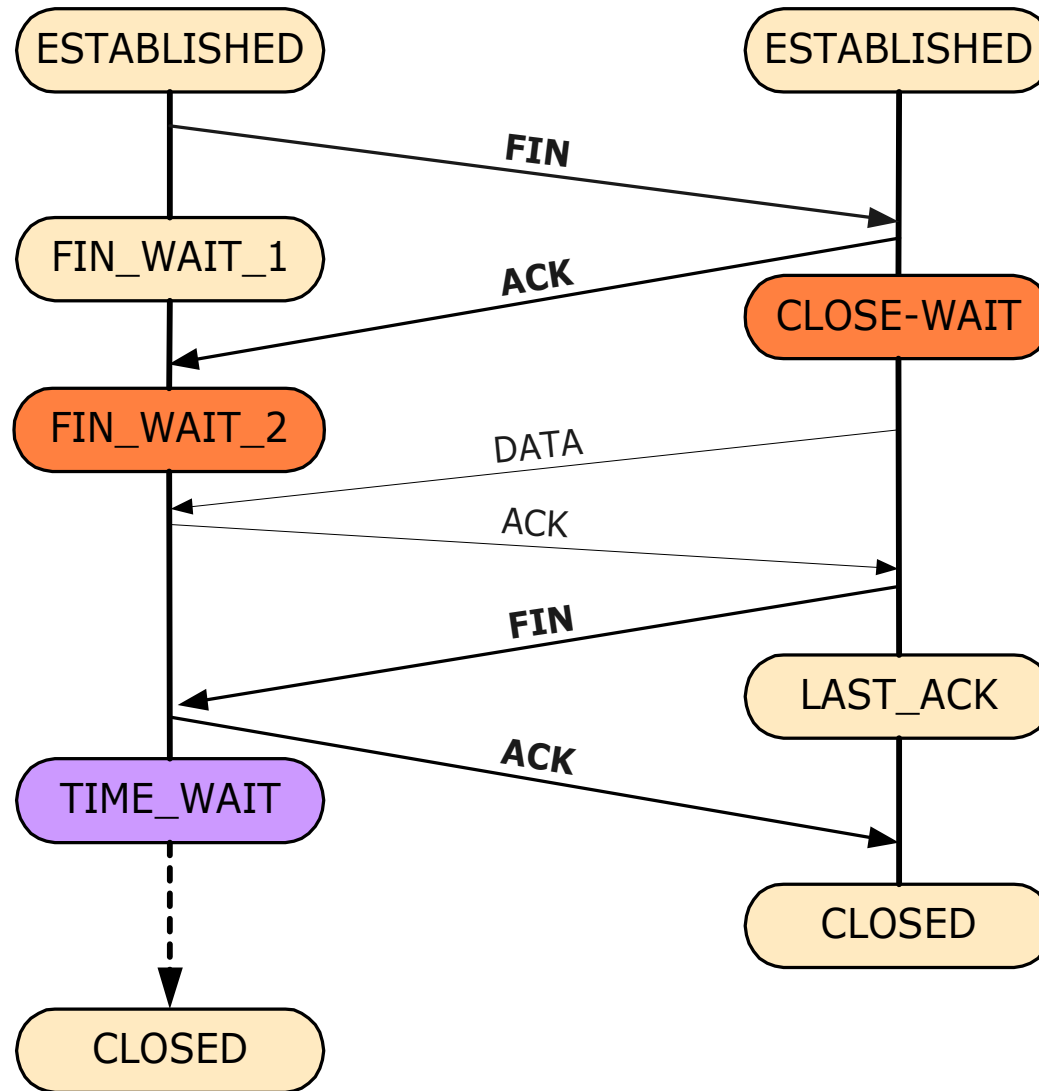


(Client Host)

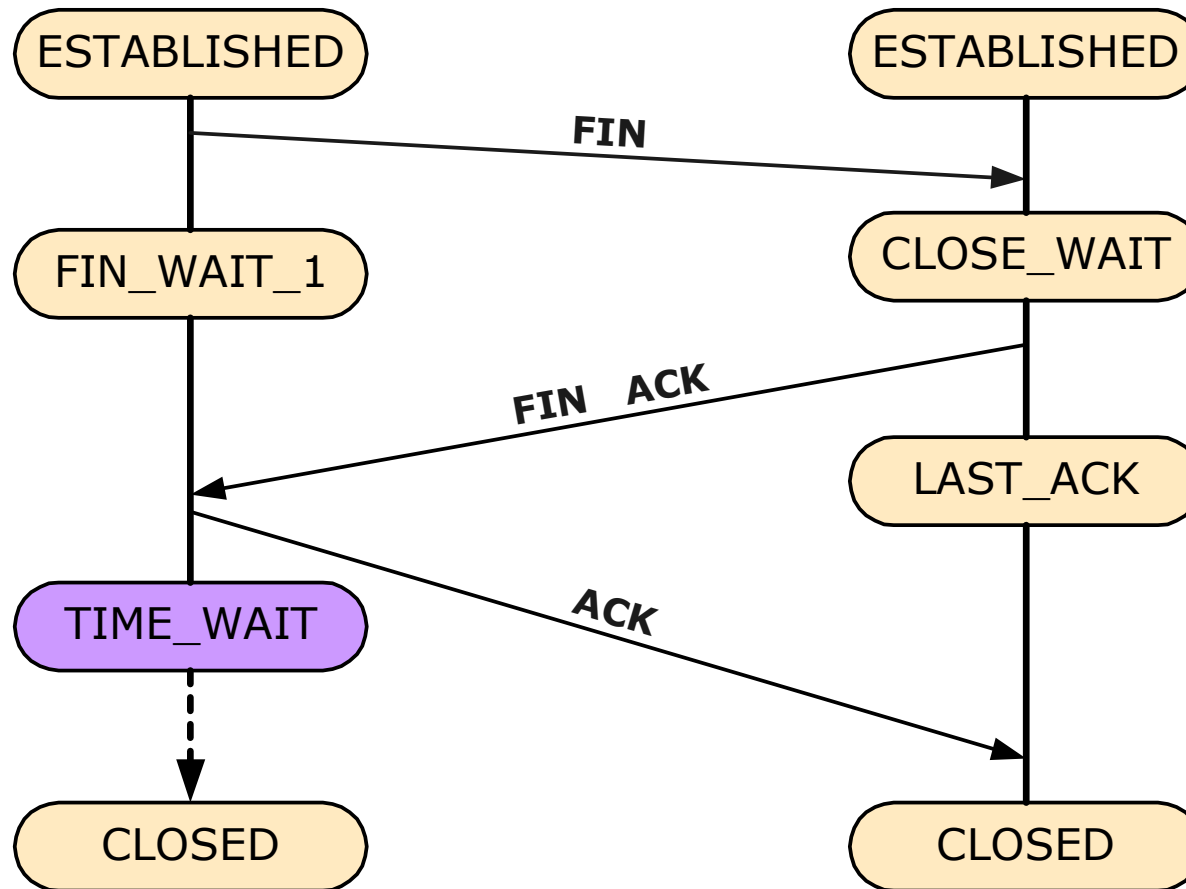
TCP连接建立



TCP 连接释放 (1)



TCP 连接释放(2)



TCP连接释放 (3)

